**VITOR HUGO DE OLIVEIRA SILVA**

# MULTITASK LEARNING FOR POINT-OF-INTEREST CLASSIFICATION AND PREDICTION TASKS: THE ROLE OF THE CHECK-IN-LEVEL REPRESENTATION

Dissertation submitted to the Computer Science Graduate Program of the Universidade Federal de Viçosa in partial fulfillment of the requirements for the degree of *Magister Scientiae*.

Orientador: Fabrício Aguiar Silva

**FLORESTAL - MINAS GERAIS**
**2026**

# Resumo

SILVA, Vitor Hugo De Oliveira, M.Sc., Universidade Federal de Viçosa, agosto de 2026. **Multitask Learning for Point-of-Interest Classification and Prediction Tasks: The Role of the Check-in-Level Representation**. Orientador: Fabrício Aguiar Silva.

Redes sociais baseadas em localização produzem registros de *check-in*, que associam um usuário, um ponto de interesse (POI) e um instante. Esses registros permitem tanto classificar a categoria de um POI quanto antecipar propriedades da próxima visita, como sua categoria e sua região. Esta dissertação investiga se o aprendizado multitarefa (MTL) pode combinar essas tarefas em um único modelo e de quais escolhas de projeto depende o sucesso desse treinamento conjunto. Embora as tarefas compartilhem dados e contexto, o compartilhamento de parâmetros também pode provocar transferência negativa. A investigação foi desenvolvida em três estudos sucessivos, organizados como um resultado negativo, seu diagnóstico e sua resolução. O primeiro estudo propôs o MTLnet, que utilizava uma representação em nível de POI e compartilhamento rígido de parâmetros para realizar classificação de categoria e previsão da próxima categoria. Apesar dessa estratégia de compartilhamento, o modelo conjunto não apresentou vantagem consistente sobre modelos dedicados a cada tarefa. O segundo estudo manteve a arquitetura do MTLnet e substituiu apenas sua entrada por codificadores espaciais, temporais e categóricos decompostos. A melhora acentuada do desempenho de categoria nos conjuntos avaliados indicou que, naquela configuração, a representação de entrada era o principal fator limitante. Com base nesse diagnóstico, o terceiro estudo passou da representação em nível de POI para uma representação em nível de *check-in* (um vetor por visita, e não por POI), trocou o compartilhamento rígido por uma topologia baseada em atenção cruzada e reuniu duas tarefas sequenciais: a previsão da próxima categoria e da previsão da próxima região. O modelo final foi avaliado em seis conjuntos de dados de diferentes contextos geográficos, incluindo um conjunto não estadunidense. O protocolo empregou validação cruzada com usuários disjuntos entre treinamento e teste estatísticos pareados. Essas análises mostraram que o modelo conjunto superou os modelos dedicados na previsão da próxima categoria em todos os conjuntos e, na previsão da próxima região, superou-os ou foi estatisticamente não-inferior dentro da margem de dois pontos de Acc@10. Em ambas as tarefas, os resultados do modelo conjunto também ficaram acima dos reportados pelos métodos externos usados como referência. Os resultados mostram que o MTL não constitui um benefício automático: sua efetividade depende da representação de entrada e da topologia de compartilhamento construída para as tarefas consideradas.

**Palavras-chave**: aprendizado multitarefa, ponto de interesse, previsão da próxima categoria, previsão da próxima região, representação em nível de check-in

# Abstract

SILVA, Vitor Hugo De Oliveira, M.Sc., Universidade Federal de Viçosa, August 2026. **Multitask Learning for Point-of-Interest Classification and Prediction Tasks: The Role of the Check-in-Level Representation**. Advisor: Fabrício Aguiar Silva.

Location-based social networks produce *check-in* records associating a user, a point of interest (POI), and a time. These records support both the classification of a POI's category and the prediction of properties of the next visit, such as its category and region. This dissertation investigates whether multitask learning (MTL) can combine these tasks in a single model and which design choices determine the success of joint training. Although the tasks share data and context, parameter sharing may also cause negative transfer. The investigation comprises three successive studies organized as a negative result, its diagnosis, and its resolution. The first study proposed MTLnet, which used a POI-level representation and hard parameter sharing for category classification and next-category prediction. Despite this sharing strategy, the joint model did not consistently outperform models dedicated to each task. The second study retained the MTLnet architecture and replaced only its input with decomposed spatial, temporal, and categorical encoders. The marked improvement in category performance across the evaluated datasets indicated that, in that configuration, the input representation was the main limiting factor. Based on this diagnosis, the third study moved from a place embedding to a check-in-level representation (one vector per visit rather than per POI), replaced hard sharing with a cross-attention-based topology, and paired two sequential tasks: next-category and next-region prediction. The final model was evaluated on six datasets from different geographic contexts, including one non-U.S. dataset. The protocol used user-disjoint cross-validation and paired statistical tests. These analyses showed that the joint model outperformed the dedicated single-task models in next-category prediction at every dataset and, in next-region prediction, either outperformed them or was statistically non-inferior within the two-point Acc@10 margin. On both tasks, the joint model's results were also above those reported by the external baselines. The results show that MTL does not provide an automatic benefit: its effectiveness depends on the input representation and the sharing topology designed for the tasks under consideration.

**Keywords**: multitask learning, point of interest, next-category prediction, next-region prediction, check-in-level representation

# List of Figures

# List of Tables

# List of abbreviations and acronyms

| | |
|---|---|
| CBIC | Congresso Brasileiro de Inteligência Computacional |
| Check2HGI | Check-in-level representation extending Hierarchical Graph Infomax |
| CoUrb | Workshop de Computação Urbana |
| DGI | Deep Graph Infomax |
| FiLM | Feature-wise Linear Modulation |
| HGI | Hierarchical Graph Infomax |
| LBSN | Location-Based Social Network |
| MobiWac | ACM International Symposium on Mobility Management and Wireless Access |
| MTL | Multitask Learning |
| POI | Point of Interest |
| SBRC | Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos |
| TOST | Two One-Sided Tests |

# Contents

# 1 Introduction

## 1.1 Context and motivation

Location-based social networks record visits through *check-ins*. Each check-in states that a user visited a place, or point of interest (POI), at a given time. Although these traces are noisy, human movement is highly regular: an analysis of large-scale mobility traces estimated the potential predictability of an individual's next location at about 93 percent [2]. Anticipating what kind of place a person will visit and where that visit will occur can support recommendation, navigation, transit planning, and the allocation of resources by area. The broader study of mobility also informs urban planning, disease-spread analysis, and pollution research [3]. Among the properties of a visit that these services rely on, the category of the place carries its semantic meaning [4], and it is one of the two properties this dissertation predicts.

This dissertation studies two properties of the next visit. *Next-category prediction* identifies the semantic category of that visit, and *next-region prediction* identifies where it will occur. These tasks differ from *next-place prediction*, which identifies the exact establishment and is outside the scope of this work. The two tasks above are sequential, because each one reads a history of visits. The first two studies also use *category classification*, a static task that reads a single place rather than a history, predicting the category of a POI held out from classifier training. The task pair changes in the final study, as explained in Section 1.2. Chapter 2 formally defines the three tasks that appear in the experiments.

Because next category and next region are predicted from the same visit history, one model may serve both tasks. Multitask learning (MTL) trains related tasks jointly so that they can share information [5]. This approach has been used in vision [6], clinical prediction [7], and language modeling [8]. For the setting studied here, its operational appeal is a single model to maintain and one forward pass that produces both predictions, instead of two dedicated single-task models. Joint training does not guarantee better predictions, however. Shared parameters can harm one task, a failure known as negative transfer. Whether MTL would help these POI prediction tasks, and which design choices would determine the result, was an open question at the start of this research.

## 1.2 Research question

**Does multitask learning help point-of-interest prediction (next category and next region), and what does the answer depend on?**

The three studies answer this question in sequence. The first reports a negative result, the

second identifies its main bottleneck, and the third tests the resulting solution. This progression is itself part of the contribution because each study narrows the explanation supported by the evidence.

Chapter 3, published at CBIC 2025, introduces MTLnet, the first joint model developed in this research. It combines a place-level graph embedding with hard parameter sharing, in which the tasks share one trunk of hidden layers and separate only at their output heads, and predicts category classification and the next category in a sequence. For that configuration, the joint model did not consistently outperform the two dedicated single-task models and required more training time. The study treated this null result as a finding and proposed three possible explanations, including an input representation that might not provide enough information for both tasks.

Chapter 4, published at CoUrb 2026, examines that explanation through a controlled comparison. The study keeps MTLnet unchanged and replaces its monolithic 64-dimensional place embedding with separate spatial, temporal, and categorical encoders. Category performance then increases substantially in every state evaluated. Because the architecture remains fixed, the study identifies the input representation as the bottleneck for that stage of the research.

Chapter 5, submitted to MobiWac 2026 and under review, develops the resolution. Any place-level embedding assigns a place the same vector on every visit; a representation that cannot tell a weekday lunch from a Saturday night out at the same place is working against both tasks at once. That observation is the hypothesis the final study tests. It moves the representation to the check-in level: one vector per visit. Acting on another of the first study's candidate explanations, it also replaces the shared hidden layers with cross-attention between task-specific streams, addressing the earlier concern about hard parameter sharing. Under a check-in level representation, static category classification is a less natural companion task than a second sequential target, so the final task pair becomes next category and next region. Both are established end targets in the mobility literature, and the next region feeds a broader family of downstream problems. The study evaluates five U.S. states and, to test whether the finding holds outside that setting, Istanbul as a non-United-States dataset, comparing the joint and dedicated models with paired superiority and non-inferiority tests. The results support a positive answer for this configuration; Chapter 5 reports the task-specific outcomes.

The task pair thus changes across the studies: the first two combine static category classification with next-category prediction, whereas the final study combines next-category and next-region prediction. The dissertation does not treat these as one unchanged experiment. Each study revised what the previous one had concluded, and each later chapter states which earlier conclusion it supersedes and under which conditions the earlier one still holds.

## 1.3 Objectives

The general objective is to determine whether multitask learning helps next-category and next-region prediction and to identify the conditions that shape the answer. Four specific objectives connect this question to the article chapters and to their consolidation:

1. Evaluate whether a joint model with hard parameter sharing benefits static category classification and next-category prediction when compared with dedicated single-task models (Chapter 3).
2. Determine how the input representation affects that result by enriching and decomposing the input while keeping the architecture fixed (Chapter 4).
3. Design and validate a check-in-level representation and a joint model for next-category and next-region prediction (Chapter 5).
4. Consolidate the evidence under the user-disjoint cross-validation, significance-testing, and non-inferiority protocol used in the final study (Chapter 5).

## 1.4 Scope and assumptions

The experiments use observational check-in traces from two sources: Gowalla data for five U.S. states (Alabama, Arizona, Florida, California, Texas) and the Istanbul subset of the Massive-STEPS benchmark. The prediction targets are the next category and the next region. In the data used here the category space has seven classes, and a region is a census tract in the United States or a *mahalle* in Istanbul. The joint model operates under a single-model constraint: one trained artifact must produce both outputs in one forward pass. These choices define the design scope. Chapter 2 specifies the output spaces and geographic units, and Chapter 6 discusses the limitations that affect how the results may generalize.

## 1.5 Organization of this dissertation

This dissertation is organized as a collection of works, where each central chapter is an independent article. The articles were re-typeset in a common format, and the second article was translated into English. Their content follows the published versions or, for the article under review, the submitted manuscript.

- **Chapter 2** defines the shared background: POI prediction tasks, mobility representations, multitask learning, datasets, metrics, and evaluation protocols.

- **Chapter 3** reproduces the first article, published in English at the XVII Congresso Brasileiro de Inteligência Computacional (CBIC 2025, DOI 10.21528/CBIC2025-1191324). This dissertation's author is its first author.
- **Chapter 4** presents an English translation of the second article, published in Portuguese at the X Workshop de Computação Urbana (CoUrb 2026, DOI 10.5753/courb.2026.22960), held with the Simpósio Brasileiro de Redes de Computadores e Sistemas Distribuídos (SBRC 2026). This dissertation's author contributed the MTLnet baseline used in the study and presented the work at the event.
- **Chapter 5** presents the third article, submitted to the 23rd ACM International Symposium on Mobility Management and Wireless Access (MobiWac 2026) and under review. This dissertation's author is its first author.
- **Chapter 6** answers the research question across the three studies, states their joint limitations, and connects each limitation to future work.

Each article chapter begins with a preface that identifies its venue and publication status. The prefaces also distinguish the conclusion supported at the time of each article from the final position of the dissertation.

## 1.6 Contributions

The contributions are organized into four groups.

**Theoretical.** The studies show that the input representation and its sharing topology determine whether MTL helps these POI prediction tasks. A null result with a place-level representation and hard sharing therefore does not conflict with a positive result obtained from a check-in-level representation and a different form of sharing. Among the works reviewed in this dissertation, none treats the next category and the next region as co-equal end targets of one joint model that does not also predict the next place.

**Software.** The work provides MTLnet (Chapter 3), Check2HGI and the joint model built on it (Chapter 5), and reproducible training and evaluation pipelines for Chapters 3 and 5. The corresponding code repositories are referenced in those chapters. Check2HGI is the reusable part: it learns one vector per visit from a four-level hierarchy and is independent of the prediction heads placed on top of it, so a task other than the two studied here can consume the same representation. The joint model separates its per-task streams from the cross-attention that connects them, which is a property of its design rather than a result measured on further tasks.

**Empirical.** The final study compares joint and dedicated models on six datasets under user-disjoint cross-validation. Each configuration has twenty fitted models: four repetitions of the complete five-fold experiment, each with a different random initialization and the same fixed folds. The analysis uses paired tests on the four per-seed means, non-inferiority tests, and a leakage audit. Because every seed uses the same fold partition, the reported intervals do not include uncertainty from resampling the user splits. Three further measurements stand on their own. Nineteen loss and gradient balancers were screened at their default configurations, at a single random initialization, on Alabama and Florida, and none improved on a tuned fixed task weighting across both tasks and both datasets. The gradients of the two tasks were measured directly and are statistically indistinguishable from orthogonal on every dataset measured, which explains the balancer result rather than merely accompanying it (Appendix D). Two external sequence baselines were adapted so that a region comparison exists at all: HMT-GRN keeps its shared multitask skeleton and gains a per-fold region-transition prior, without its graph components and hierarchical beam search, and STAN has its output layer adapted to rank region candidates. Neither adaptation reproduces its complete published system, and Chapter 5 states what was changed.

**Practical.** One model produces both predictions in one forward pass. It outperforms the dedicated models on the category task at all six datasets and on the region task at four of the six; at the other two it remains statistically non-inferior within a two-point margin (TOST).

# 2 Fundamentals

This chapter provides the concepts shared by the three studies. It first distinguishes the POI prediction tasks (Section 2.1). It then traces mobility representations from one-hot identifiers to Check2HGI (Section 2.2), explains the main forms and risks of multitask learning (Section 2.3), and defines the data and evaluation protocol (Section 2.4). Section 2.5 connects these elements to the questions addressed in the article chapters.

## 2.1 Point-of-interest prediction tasks

This section establishes what each model predicts before reviewing how those predictions are made. Location-based social networks (LBSNs) record mobility as check-ins, each linking a user, a POI, and a timestamp. Their geographic and temporal detail supports data-driven studies of cities [9]. The records also reveal recurring behavior: people often return to a small set of places and make longer trips less frequently [10]. An entropy analysis estimated the potential predictability of an individual's next location at about 93 percent [2]. Because this estimate concerns next-location prediction at a coarse spatial resolution, it shows that mobility contains learnable regularity but does not provide a reference point for the category and region metrics defined in Section 2.4.

### 2.1.1 Task boundaries and notation

Human-mobility models address distinct targets, from crowd-flow estimation to next-location prediction [3]. Because these targets are not interchangeable, model comparisons require explicit task boundaries. This dissertation studies three tasks: one static and two sequential.

#### 2.1.1.1 Check-ins and histories

Let $\mathcal{U}$, $\mathcal{P}$, $C$, and $\mathcal{R}$ denote the sets of users, POIs, category classes, and region classes. Each POI $p \in \mathcal{P}$ carries a category $c_p \in C$ and lies in a region $r_p \in \mathcal{R}$.

**Definition 2.1** (Check-in)**.** The $i$th check-in of user $u$ is the tuple

$$x_i = (u, p_i, t_i, c_i, r_i), \tag{2.1}$$

where $p_i \in \mathcal{P}$ is the visited POI, $t_i$ is its timestamp, $c_i = c_{p_i}$ is its category, and $r_i = r_{p_i}$ is its region.

**Definition 2.2** (Check-in history)**.** The check-in history of length $\ell$ preceding check-in $x_i$ is the ordered sequence

$$H_i = (x_{i-\ell}, \ldots, x_{i-1}). \tag{2.2}$$

For the two sequential tasks studied here (Definitions 2.7 and 2.8), the prediction target is $c_i$ for next-category prediction and $r_i$ for next-region prediction. These labels belong to $x_i$, not to the preceding check-ins in $H_i$: the categories and regions carried by the visits in $H_i$ are observed input, and only the attributes of $x_i$ itself are withheld.

#### 2.1.1.2 Check-in and place embedding

A task definition must also state what the model reads. Two representations recur throughout this dissertation, and the three studies vary which one supplies the model input.

**Definition 2.3** (Place embedding)**.** A place embedding assigns one vector $\mathbf{e}_p$ to each POI $p$, so every check-in at that POI enters the model with the same representation.

**Definition 2.4** (Check-in-level representation)**.** A check-in-level representation assigns one vector $\mathbf{e}_{x_i}$ to each check-in $x_i$, so two visits to the same POI may enter the model with different representations.

**Definition 2.5** (Representation map)**.** A representation map assigns to each check-in a vector,

$$\rho(x_i) \in \mathbb{R}^d, \tag{2.3}$$

and extends to histories elementwise,

$$\rho(H_i) = \big(\rho(x_{i-\ell}), \ldots, \rho(x_{i-1})\big). \tag{2.4}$$

Every model of the sequential tasks in this dissertation reads $\rho(H_i)$ rather than $H_i$ itself. Chapter 3 defines the input at the place level, $\rho(x_j) = \mathbf{e}_{p_j}$. Chapter 4 retains a vector tied to the visited POI but enriches it with separate spatial, temporal, and categorical components. Chapter 5 moves the input to the check-in level. Its semantic stream reads the per-visit vector $\rho(x_i) = \mathbf{e}_{x_i}$, while its spatial stream reads the trained vector of the visit's region node. The task definitions remain fixed, while each study uses its own form of $\rho$.

#### 2.1.1.3 The three experimental tasks

Three definitions separate the targets. The first is static and reads a place; the other two are sequential and read a history.

**Definition 2.6** (Category classification)**.** Category classification predicts the category of a POI from its representation:

$$g_{\text{cat}}(\mathbf{e}_p) \longrightarrow c_p. \tag{2.5}$$

At evaluation time, POI $p$ is held out from the classifier-training fold, so "unknown POI" means unknown to the classifier, not necessarily absent from the graph used to learn $\mathbf{e}_p$.

**Definition 2.7** (Next-category prediction)**.** Next-category prediction maps a check-in history to the category of the next visit:

$$f_{\text{cat}}(H_i) \longrightarrow c_i. \tag{2.6}$$

**Definition 2.8** (Next-region prediction)**.** Next-region prediction maps a check-in history to the region of the next visit:

$$f_{\text{reg}}(H_i) \longrightarrow r_i. \tag{2.7}$$

The task pairing changes across the studies. Chapters 3 and 4 combine category classification (Definition 2.6) with next-category prediction (Definition 2.7). Chapter 5 combines next-category prediction with next-region prediction (Definition 2.8). All three studies use seven categories: Community, Entertainment, Food, Nightlife, Outdoors, Shopping, and Travel. For the region task, the target is an administrative unit at neighborhood scale. These output spaces require different metrics and reference points, and Section 2.4 both names the unit each dataset supplies and explains the measurement.

### 2.1.2 Related sequential prediction settings

One further target must be named, because it is the task that the models this section reviews were built to solve.

**Definition 2.9** (Next-place prediction)**.** Next-place prediction maps a check-in history to the identity of the next visited POI:

$$f_{\text{place}}(H_i) \longrightarrow p_i. \tag{2.8}$$

It is named to delimit the scope of the dissertation, and no chapter reports a result for $f_{\text{place}}$.

Next place is the dominant task in the field, and methods developed for it provide the methodological basis for next-category and next-region prediction. Early approaches relied on recurrent architectures. ST-RNN uses time-specific and distance-specific transition matrices [11], while DeepMove adds attention over long and sparse visit histories [12]. HST-LSTM incorporates spatial-temporal context into a hierarchical recurrent cell [13]. Flashback retrieves past hidden states according to the temporal and spatial gaps between visits [14].

Later approaches make attention central to the architecture. STAN connects non-adjacent visits through spatio-temporal correlations [15]. GeoSAN combines a hierarchical geographic grid with self-attention [16], and GETNext introduces a global trajectory-flow map into a transformer [17]. Every model named here predicts the exact next place, so none of them is a direct baseline for the targets studied in this dissertation.

### 2.1.3 Category and region as end targets

Two strands of this literature support the task choice in this dissertation. The first concerns how a visit is represented. CTLE learns context-aware and time-aware location embeddings, allowing the same POI to have different vectors in different visits [18]. This per-visit view motivates the representation discussed in Section 2.2.

The second strand concerns the role of category and region predictions. Some methods use them to support next-place prediction rather than as final outputs. HMT-GRN uses a predicted region to constrain the search for a place [19], and CatDM predicts a category to reduce the set of candidate places [20]. Other studies make one of these properties the final target. Activity-region prediction addresses the region directly [21], while POI-RGNN predicts the next category [22]. This dissertation uses next category and next region as final outputs of equal status in one model.

## 2.2 Representations for mobility

This section traces the representations used in the dissertation and explains why the final study moves from a place embedding to a check-in-level representation.

### 2.2.1 From identifiers to graph embeddings

The representation line begins with identifiers and progresses to vectors learned from graph structure.

A one-hot identifier marks a place or category by position but encodes no relationship between identifiers. Distributed representations instead learn dense vectors whose geometry reflects relationships in the data. Skip-gram places symbols with similar contexts near one another [23]. DeepWalk transfers this idea to graphs by treating random walks as sentences [24], and node2vec adjusts the walk to emphasize homophily or structural roles [25]. Graph neural networks learn these relationships through neighborhood aggregation. Graph convolutional networks use a localized spectral rule [26], graph attention networks learn the weight of each neighbor [27], and GraphSAGE learns aggregators that can embed nodes not seen during training [28].

### 2.2.2 Hierarchical graph infomax

The graph-infomax methods supply the unsupervised representations used across the three studies. The idea these methods share is simple enough to state before any of the notation: the model learns useful vectors by being asked to tell a true pairing of two parts of the data from a corrupted one, and it needs no labels to do so, because the data itself says which pairing is the true one.

The representations used in this work are trained with no next-category and no next-region target, each visit contributes with its own description: the category of the place visited is an input feature of its node, together with the hour and the day of the week. Mutual-information estimators provide one way to learn such representations [29]. Deep InfoMax maximizes agreement between local features and a global summary [30], and Deep Graph Infomax (DGI) applies this objective to graph nodes and a graph-level summary [31]. DGI supplies the objective and the place embeddings used in the first study. Hierarchical Graph Infomax (HGI) extends that objective across POI, region, and city levels [32].

HGI is the place-level representation that the final study replaces, so its reference point matters here. The mechanism itself is set out in the original paper, and the account below is limited to what the later chapters rely on. Its own stated goal is to learn urban region representations from POIs in a fully unsupervised manner, and it reaches that goal by extending the two scales of DGI to three: POI, region, and city. The name states what is maximized. Training raises the mutual information between representations at two adjacent levels of that hierarchy, and it does so without evaluating that quantity in closed form. A bilinear discriminator combines two embeddings through a learned weight matrix and passes the result through a logistic function, and the loss rewards a high score for a true pair and a low score for a false pair. No label of any downstream task enters that comparison, so the representation is obtained without supervision.

One consequence of the design matters for the later chapters. A pretrained category encoder supplies the initial POI features, one graph convolution layer over a Delaunay graph of the POIs of the study area adds spatial context to each of them, multi-head attention aggregates the POI embeddings of a region, and an area-weighted sum over regions produces one city embedding. The training signal updates the POI encoder, the aggregation function, and the region encoder together, so a POI embedding is optimized to score high against the embedding of its own region and low against the embeddings of other regions. Region membership also enters earlier, through the edge weights of the POI graph, which decrease with the distance between two POIs and are reduced further when the two POIs lie in different regions. The POI-level output is therefore not a description of the place in isolation. It already reflects the region the place belongs to.

HGI was developed and evaluated for urban region representation. Its reported experiments estimate urban functional distributions, population density, and housing price in Xiamen Island and Shenzhen, and the object under evaluation there is the region embedding. The POI embedding is an internal stage in producing it. This dissertation repurposes that POI-level output for sequential prediction, a use the original evaluation does not cover. In this role, HGI supplies the place-level baseline in the later studies and the direct basis of Check2HGI.

The move to other study areas left one setting of that baseline open. Huang et al. reduce an edge between POIs of different regions to 0.4 of its weight in the experiments named above, and this work did not transfer that value untested. In experiments on one of the state datasets of

this dissertation, over the same five folds used elsewhere, a value of 0.7 gave the best category F1 among the values tried, and the later studies adopt it. What that comparison settles is one hyperparameter of the place-level baseline. It is not evidence about how HGI and Check2HGI compare.

### 2.2.3 From a fixed place vector to visit context

The move from HGI to Check2HGI begins with information that a fixed POI vector cannot express.

Definition 2.3 carries one limitation for this research: a weekday morning and a Saturday night at the same place have identical inputs at the representation level. CTLE shows an alternative by learning context-aware and time-aware location embeddings whose vectors change with the visit [18].

Check2HGI reaches the check-in level of Definition 2.4 by adding the check-in below the place in the hierarchy.

#### 2.2.3.1 Context encoders

A per-visit representation needs temporal and spatial context in addition to the identity of the visited POI. The second study supplies that context through separate encoders while its input stays a function of the visited POI. Time2Vec combines linear and periodic components for temporal features [33]. SIREN uses sinusoidal activation functions to represent continuous signals [34]. Space2Vec encodes location and spatial context at several scales [35], while Sphere2Vec represents coordinates on a sphere rather than on a flat surface [36]. A related approach combines spherical harmonics with sinusoidal networks [37]. Chapter 4 keeps the MTLnet architecture but replaces its single place embedding with spatial, temporal, and categorical encoders. The controlled change isolates the effect of the input representation.

#### 2.2.3.2 The check-in level

Check2HGI completes the move from contextualized inputs to one learned representation per visit. It adds a fourth level below the place and learns one vector per visit from the visits themselves, with no next-category and no next-region target in its objective. The hierarchy has three adjacent-level boundaries: check-in to place, place to region, and region to city. Its three hierarchical boundaries contribute the fixed-weight sum

$$\mathcal{L} = 0.4\,\mathcal{L}_{c2p} + 0.3\,\mathcal{L}_{p2r} + 0.3\,\mathcal{L}_{r2c}, \tag{2.9}$$

where $\mathcal{L}_{c2p}$, $\mathcal{L}_{p2r}$, and $\mathcal{L}_{r2c}$ correspond to these three boundaries. Each term uses a bilinear discriminator,

$$\mathcal{D}(\mathbf{e}_1, \mathbf{e}_2) = \sigma(\mathbf{e}_1^\top \mathbf{W} \mathbf{e}_2), \tag{2.10}$$

where $\mathbf{W}$ is learned and $\sigma(z) = 1/(1 + e^{-z})$ is the logistic function. The discriminator is linear in either embedding when the other is held fixed, and its output lies between 0 and 1. For each boundary, the loss favors a high score for a true pair and a low score for a false pair:

$$\mathcal{L}_* = -\log \mathcal{D}(\mathbf{e}^+, \mathbf{e}^+) - \log\left(1 - \mathcal{D}(\mathbf{e}^+, \mathbf{e}^-)\right), \tag{2.11}$$

Here, $\mathbf{e}^+$ belongs to a true pair, such as a check-in and the place where it occurred, whereas $\mathbf{e}^-$ is substituted from another example in the batch. No next-step target appears in these equations. Two auxiliary terms complete the objective that the final study trains: a reconstruction term at weight 0.3, which recovers a masked place's own distribution over the seven categories from its neighbors, and a term at weight 0.1, which keeps the trainable table of place vectors near the values it started from.

Where a place embedding answers "what is this place," a check-in-level representation answers "what is this visit," and Chapter 5 develops this representation and the joint model built on it. Table 1 traces the full progression, from DGI through HGI to the check-in level and the joint model that uses it.

### 2.2.4 Model lineage

Each study changes the input representation, and each also changes the architecture that consumes it. Naming both sequences together lets a later chapter be read against the step it altered.

The final joint model remains part of the MTLnet lineage. It retains the task-specific input encoders, output heads, and division between shared and task-specific parameters. MTLnet uses residual layers conditioned by Feature-wise Linear Modulation (FiLM) as its shared middle. The joint model replaces that component with cross-attention blocks, allowing each task stream to read the other while retaining its own feed-forward parameters. A private spatial path also gives the region output direct access to the region sequence before the streams interact. The models therefore differ in their sharing topology and in the private input available to the region output. Both architectures receive two inputs, and what changes between them is the source: MTLnet derives both from one place embedding, whereas the joint model reads two tables exported from the same check-in-level representation. The two tables share an origin by construction, so they are not independent views. This relationship allows the negative result in Chapter 3 and the positive result in Chapter 5 to be interpreted as two stages of one model lineage.

Table 1 – Representation and model lineage threaded through this dissertation, from the place-level graph-infomax embeddings to the check-in-level representation and the joint model built on it. Status of the Check2HGI representation and the joint model is *submitted, under review*.

| Model / method | What it added | Reference |
|---|---|---|
| DGI | Unsupervised place embeddings by maximizing mutual information between local patches and a global graph summary. | [31] |
| HGI | Extends graph infomax over the POI, region, and city hierarchy to yield region-aware place embeddings. | [32] |
| MTLnet | First joint model: place embedding, FiLM conditioning, and hard parameter sharing. Null result for that configuration. | [1] |
| ST-MTLNet | Keeps MTLnet, replaces the place-embedding input with decomposed spatial, temporal, and categorical encoders. | Chapter 4 |
| Check2HGI | Check-in-level representation: graph infomax extended to the check-in, so each visit carries its own vector. | Chapter 5 |
| Joint model | Cross-attention model on Check2HGI: one model for both tasks, next category and next region. | Chapter 5 |

## 2.3 Multitask learning

Multitask learning (MTL) trains related tasks together in the expectation that shared representations improve generalization [5]. This section explains how tasks share a model, why that sharing may fail, and how multitask optimization responds to the failure. This dissertation builds joint models for two evolving task pairs. The first pair combines static category classification from a place embedding with next-category prediction from a check-in history. The final joint model reads a check-in history to predict both the next category and the next region of a visit. Whether the shared computation helps depends on the sharing topology and the training objective.

### 2.3.1 Sharing topologies

The first design choice is where task-specific computation ends and shared computation begins.

**Definition 2.10** (Hard parameter sharing)**.** Hard parameter sharing passes all inputs through a single shared trunk before branching to task-specific output heads, so every task uses the same hidden representations [38].

**Definition 2.11** (Soft parameter sharing)**.** Soft parameter sharing gives each task its own complete network and couples the networks by penalizing differences between their parameters [38].

Definition 2.10 is the topology of the first joint model of this dissertation, and it carries the risk this section addresses. A shared trunk need not treat both tasks identically. Feature-wise Linear Modulation (FiLM) scales and shifts features according to an additional input [39], and MTLnet uses it to condition its shared layers on task identity, so the two tasks read the same parameters through different scalings.

**Definition 2.12** (Negative transfer)**.** Negative transfer occurs when joint training leaves a task worse than its dedicated single-task model.

Because hard sharing is rigid and soft sharing requires many parameters, several architectures explore intermediate topologies. Cross-stitch units learn linear combinations of activations from different task networks [40]. The multi-gate mixture-of-experts shares expert subnetworks but learns a separate routing gate for each task [41]. Progressive layered extraction separates shared and task-specific experts across successive gates [42], while DSelect-k makes expert selection sparse and differentiable [43]. These methods selectively share computation instead of forcing every task through one fixed trunk.

The joint model in Chapter 5 builds a partial hard-shared interaction subsystem based on this principle, using bidirectional cross-attention rather than routing gates. The tasks do not reuse the same weight matrices, but their streams cross within this jointly optimized module. Each stream queries the other to read its features, which couples their gradients without forcing their representations into a single bottleneck.

### 2.3.2 Gradient conflict

The second design choice depends on a quantity rather than on a preference: whether the tasks are asking for the same update at all. Gradient conflict describes one source of negative transfer, and it is a measured quantity rather than a figure of speech.

**Definition 2.13** (Gradient conflict)**.** Let $\mathbf{g}_i$ and $\mathbf{g}_j$ be the gradients of two task losses with respect to the shared parameters, and let $\varphi_{ij}$ be the angle between them, so that

$$\cos \varphi_{ij} = \frac{\mathbf{g}_i^\top \mathbf{g}_j}{\|\mathbf{g}_i\| \, \|\mathbf{g}_j\|}. \tag{2.12}$$

The two tasks conflict at that point when $\cos \varphi_{ij} < 0$ [44].

The cosine between their gradients reads how the two requested updates are aligned and ignores how large they are: $+1$ means the tasks ask for the same update, $-1$ for opposite ones, and 0 that the requests are orthogonal, each leaving the other's objective unchanged to first order. PCGrad changes a direction only when this cosine is negative. Other methods may still react to differences in gradient magnitude or task loss. In Chapter 5, two such methods

exceed equal weighting on one dataset but not on the others. Appendix D evaluates the final joint model and finds the task gradients equivalent to orthogonal within a margin of five hundredths at every dataset measured there. A paired test separates the mean from zero at three datasets, but those small departures remain within the equivalence margin. The combined result is that the deviations are detectable but too small to represent meaningful directional conflict at the scale considered by the balancers.

### 2.3.3 Multi-objective optimization

The third design choice is how several task losses determine one parameter update. A measured conflict has to be acted on somewhere, and the place where it acts is the step that turns two losses into one direction. Shared parameters can produce the negative transfer of Definition 2.12, so task relationships must be measured rather than assumed [45]. The training problem has multiple losses, and a standard scalar objective writes them as

$$\mathcal{L}_{\text{total}}(\boldsymbol{\theta}) = \sum_{k=1}^{K} w_k \, \mathcal{L}_k(\boldsymbol{\theta}), \tag{2.13}$$

over $K$ tasks and shared parameters $\boldsymbol{\theta}$, where $\mathcal{L}_k$ is the loss of task $k$. A balancing method changes the weights $w_k$ or the update direction that they induce. The scalar sum does not remove the multi-objective nature of the problem [46]. One parameter setting exhibits Pareto dominance over another when it is no worse on every task loss and better on at least one. A setting is Pareto optimal when no other setting dominates it, and the corresponding loss vectors form the Pareto front.

#### 2.3.3.1 Guarantees and their limits

The guarantees of balancing methods are usually weaker than Pareto optimality. A Pareto-stationary point is one at which a convex combination of the task gradients is zero. This condition is necessary but not sufficient for Pareto optimality [47]. Nash-MTL proves convergence of a subsequence to such a point and reaches Pareto optimality only under an additional convexity assumption that does not hold for a deep network [47]. CAGrad has Pareto-stationary fixed points [48], and Aligned-MTL converges to one when task weights are fixed in advance [49]. PCGrad makes no Pareto claim of its own. Under conditions on two task gradients, loss curvature, and step size, it guarantees that one projected update does not produce a higher multitask loss than the original gradient [44]. Reaching the Pareto front would still leave the balance between tasks unspecified [48, 49]. This dissertation therefore claims no Pareto property for its models. It judges each task against its dedicated single-task model under the tests defined in Section 2.4.

### 2.3.4 Loss-balancing methods

Balancing methods differ in whether they modify loss weights or task-gradient directions.

The first class sets the weights $w_k$ of Equation 2.13 from different signals. Kendall et al. weight each loss by the task's homoscedastic uncertainty [50]. Chen et al. introduce GradNorm, which balances gradient magnitudes and training rates [51].

S. Liu et al. introduce Dynamic Weight Average alongside their attention architecture, deriving each weight from the rate of change of that task's loss, measured as the ratio of its two previous loss values [52], and B. Liu et al. propose FAMO, which tracks loss changes without computing every task gradient at each step [53].

The second class changes the update direction instead. The multi-objective reading begins with Sener and Koltun, who cast multitask learning as multi-objective optimization [46]. Yu et al. introduce PCGrad, which projects away a conflicting component in the sense of Definition 2.13 [44]. B. Liu et al. introduce CAGrad, which protects the worst per-task improvement within a neighborhood of the average gradient [48]. Navon et al. introduce Nash-MTL, which treats gradient combination as a bargaining problem [47], and Senushkin et al. introduce Aligned-MTL, which uses the condition number of the linear system of task gradients as a stability criterion and aligns that system's orthogonal components [49].

These methods must be compared with simple alternatives. Random loss weighting can be competitive [54]; specialized optimizers often fail to improve on a tuned fixed-weight baseline [55]; and plain loss summation with standard regularization can match or improve on them [56]. Surveys of general and dense-prediction MTL summarize the same design space [57, 58]. For this dissertation, a balancing method is useful only if it improves on a tuned fixed weighting.

### 2.3.5 Multitask learning for mobility prediction

Prior mobility applications show several ways to connect auxiliary targets to an exact-place recommendation.

In mobility, MTL has been used almost entirely in the service of next place. MCARNN jointly predicts activity and place through a context-aware recurrent unit [59]. CSLSL instead predicts time, then activity, and then location in a cascade [60]. iMTL couples next-POI recommendation with the handling of uncertain check-ins [61]. Halder et al. develop multitask next-POI recommenders that account for queue time and, in the journal extension, user interest [62, 63]. TME instead applies tree-guided multitask embedding to static semantic POI annotation [4]. HAMTL sets location prediction as its main task and category prediction as an auxiliary task, and its hierarchical decoder predicts the category of the next destination first and then refines the prediction of the specific location using that predicted category [64]. The same

group had already published IeMTLF, an interaction-enhanced multitask framework for next location prediction [65].

These methods confirm that category information and auxiliary tasks can support mobility prediction, but they do not instantiate the pair studied in the final chapter. Among the works reviewed here, none treats next category and next region as co-equal end targets of one joint model.

This dissertation addresses that gap starting with MTLnet, which evaluates hard parameter sharing over a static place embedding [1]. That initial configuration fails to outperform dedicated single-task models, motivating the representation and topology refinements developed in subsequent chapters.

## 2.4 Datasets and evaluation

This section defines the datasets, metrics, reference points, and validation protocol used to interpret the results. It also states which statistical tests support the comparison verbs used in the final study.

### 2.4.1 Datasets

The geographic scope combines five state-level datasets with one city outside the United States.

The experiments use two sources of check-ins. Gowalla is the main source, and the dissertation evaluates Alabama, Arizona, Florida, California, and Texas [10]. Istanbul check-ins come from Massive-STEPS, which also draws attention to the field's continued dependence on old datasets [66]. The two sources supply different administrative units for the region target: a census tract in the five United States datasets, and a *mahalle*, a municipal neighborhood, in Istanbul. The Foursquare dataset for New York and Tokyo is another common benchmark [67], but it provides context only and is not used in the experiments reported here.

### 2.4.2 Metrics and reference points

Each sequential target has its own metric, and the scores are not combined into one claim of task performance.

#### 2.4.2.1 Next category

Next-category prediction uses macro-F1 because the class distribution is imbalanced. Food accounts for roughly one third of the check-ins in a representative state, so plain accuracy

can hide poor performance on less frequent classes. For $C$ classes, macro-F1 is

$$\text{MacroF1} = \frac{1}{C} \sum_{c=1}^{C} \frac{2P_c R_c}{P_c + R_c}, \tag{2.14}$$

where $P_c$ and $R_c$ are precision and recall for class $c$. The unweighted mean gives every class equal importance [68]. It does not show which individual classes improve, and it can be low even when overall accuracy is high. The models use unweighted cross-entropy rather than a reweighted loss. In Chapter 5, class weighting lowered both category macro-F1 and region accuracy.

#### 2.4.2.2 Next region

Next-region prediction uses accuracy at 10 (Acc@10):

$$\text{Acc@10} = \frac{1}{N} \sum_{i=1}^{N} \mathbb{I}[y_i \in \text{Top10}(\hat{\mathbf{p}}_i)], \tag{2.15}$$

where $y_i$ is the true region and $\text{Top10}(\hat{\mathbf{p}}_i)$ contains the ten regions with the largest predicted scores. The metric is the fraction of test check-ins for which the correct region appears in this set. It does not distinguish first place from tenth and does not measure the probability assigned to the true region. Regions absent from the training partition count as errors. The reported out-of-distribution discounted, or OOD-discounted, Acc@10 is therefore the Acc@10 measured on the in-distribution check-ins, multiplied by one minus the fraction of check-ins whose region is out of distribution. Chapter 5 gives the complete evaluation procedure.

#### 2.4.2.3 Joint-model selection and floors

Dedicated single-task models evaluate each target independently using its respective metric. Evaluating a joint model, however, requires a single validation signal that considers the performance of both outputs simultaneously. The validation protocol achieves this by combining category macro-F1 and region Acc@10 into a scalar selection score:

$$S_{\text{joint}} = \sqrt{\text{MacroF1} \times \text{Acc@10}}. \tag{2.16}$$

Under this *joint-best* convention, both reported task scores come from the same validation-selected checkpoint, reflecting the balanced performance of a single deployable system.

To interpret these scores, model performance is evaluated between lower reference floors and upper benchmark ceilings. At the lower bound, basic baselines establish performance floors: a majority-class predictor for category classification, and a first-order Markov model over training visits for sequential transitions [69]. At the upper bound, the primary reference ceiling

for evaluating multitask gains is the dedicated single-task model for each output. Theoretical limits from prior work, such as the 93 percent potential predictability reported for coarse location prediction [2], do not serve as ceilings for seven-class macro-F1 or region ranking.

### 2.4.3 Preparation and data split

How the check-ins are divided determines which population a result covers.

The studies use stratified cross-validation [70], but the split changes across chapters. Chapters 3 and 4 stratify samples, so one user's check-ins may occur in both training and validation. Chapter 5 instead uses a grouped, stratified splitter that keeps each user on one side of a fold [71]. A seed in that chapter is one complete repetition of the five-fold experiment with a different random initialization. Four seeds over one fixed set of five folds produce twenty fitted models per configuration. The fixed folds isolate random-initialization variation but do not measure uncertainty from resampling the users.

### 2.4.4 Comparison and statistical decisions

The validation design determines which generalization claim and comparison verb each result can support.

Chapters 3 and 4 report fold means and standard deviations without significance tests. The inferential rules therefore apply to Chapter 5. Its superiority verdict uses a paired $t$ test on the four per-seed means. A Wilcoxon signed-rank test on the twenty paired fold results is reported alongside it and reaches the same decisions [72]. Holm correction controls family-wise error across the six dataset comparisons [73]. A superiority test cannot establish that a difference is small enough to accept. For Alabama and Arizona, two one-sided tests establish that the joint model is statistically non-inferior within a two-point margin [74]. Accordingly, *outperforms* is reserved for paired superiority and *matches* for statistical non-inferiority within that margin.

## 2.5 Relevance

The preceding sections introduce the prediction targets, the representations, multitask learning, and the evaluation protocol one at a time. They are parts of a single problem. This section states how they combine, before the three studies put them to work.

### 2.5.1 The parts of one problem

The argument begins with the targets: Next place, next category, and next region have different output spaces and different meanings for a correct answer. This dissertation treats next

category and next region as final outputs rather than as intermediate steps toward next-place prediction.

Although this dissertation reports on those two targets, its three studies do not all train the same pair. Chapters 3 and 4 pair next-category prediction with category classification, the static task that labels a place from its own representation. Chapter 5 keeps next category but replaces that static task with next region. The replacement follows from the representation rather than from a change of interest: a place embedding gives a static classifier one fixed vector per place, whereas a check-in-level representation describes a single visit, and the static task is a less natural fit for that input than a second sequential target.

Two choices increase what the input representation must carry: treating next category and next region as final outputs, and replacing the static task. A place embedding can encode semantic and geographic relationships, but it assigns the same vector to every visit to a place, so it carries nothing about the visit being predicted. Check2HGI answers that limit by adding the visit as a level of its own.

Multitask learning is the mechanism that would let two tasks use what they have in common, and it is not a free gain. Hard sharing can leave a task below its dedicated single-task model. The balancing methods proposed to prevent that outcome often do not improve on a tuned fixed-weight baseline. For the pair of tasks studied here, the gradients are close to orthogonal on the shared parameters, which leaves such a method little to correct. Whether joint training helps therefore cannot be assumed. It has to be measured against the dedicated single-task model for each output, under a protocol that does not favor the joint model.

Section 2.4 defines that protocol, and the final study applies it in full. Chapters 3 and 4 instead use sample-stratified splits and report fold means without significance tests. In the final study, user-disjoint cross-validation limits leakage between a user's training and test visits. Macro-F1 and OOD-discounted Acc@10 describe different outputs and are read against explicit reference points. Paired superiority tests support the verb *outperforms*, while non-inferiority tests within a two-point margin support *matches*. These rules prevent an observed mean difference from being reported as a conclusion that its test does not support.

### 2.5.2 Gaps, questions, and answers

Whether joint training helps this task pair is still open, and two gaps in the literature explain why. The field has a place-level representation, but not a check-in-level one built for these targets. It studies multitask learning, but it has mostly treated the region as an intermediate signal on the way to a place rather than as an end target.

Each study addresses one question. Chapter 3 asks whether a place-level representation and hard parameter sharing are sufficient for a joint model. For that configuration, the joint model does not outperform the dedicated single-task models. Chapter 4 keeps the model fixed, asks

whether the input representation, rather than the sharing topology, is the bottleneck, and finds that it is. Chapter 5 asks what a check-in-level representation and a different sharing topology enable for two sequential targets. There, the joint model outperforms the dedicated model on next category at all six datasets, and on next region it outperforms at four of them and matches the dedicated model at the other two. Chapter 5 reports which datasets fall on each side.

Those answers also decide something practical. Anticipating what kind of place a person will visit, and where that visit will occur, can support recommendation, navigation, transit planning, and the allocation of resources by area. A service that needs both predictions can now take them from a single model to maintain and one forward pass, instead of two dedicated single-task models. The gain is operational, not computational. Chapter 5 reports the joint model as the larger artifact, and a forward pass through it costs more than running the two dedicated models, so the reduction is in the number of models to train and maintain.

# 3 Multitask Learning for POI Category and Next-POI Prediction

*This chapter reproduces the article "An Investigation into Multi-Task Learning for Point-of-Interest Category Classification and Next-POI Prediction", published at the Congresso Brasileiro de Inteligência Computacional (CBIC 2025), DOI 10.21528/CBIC2025-1191324, with Vitor H. O. Silva as first author. The text was reformatted from the two-column conference layout to the dissertation format, and typographical and tabulation errata were corrected. Its conclusions are the conclusions of the time, for the configuration studied here: with a place-level embedding and hard parameter sharing, multitask learning did not consistently improve on the dedicated single-task models. The two chapters that follow revise that verdict, and they change different things to do it. Chapter 4 keeps this architecture and replaces the input representation, which is what identifies the representation as the binding constraint. Chapter 5 then changes the architecture as well, moving from hard parameter sharing to a cross-attention sharing topology, and changes the task pair with it. The chapter's preference for the Nash-MTL optimizer is likewise a conclusion of the time, weakened by a later finding about the optimizer implementation, and Chapter 5 does not rely on it. A note on terminology: the term "Next-POI Prediction" as used in the reproduced article denotes the frame's* next category *task, that is, predicting the category of the next visited place, not the exact place itself; the dissertation reserves* next place *for the exact-POI task, which is not studied here.*

## 3.1 Introduction

Multitask Learning (MTL) is a machine learning paradigm where multiple related tasks are learned jointly, sharing representations and inductive biases to improve generalization performance [5]. By using shared information, MTL can potentially mitigate overfitting and yield more robust feature extractors compared to single-task approaches. Its success in domains like computer vision [6], natural language processing [8] and recommendation systems [61] motivates its application to Location-Based Social Networks (LBSNs), where understanding user interactions with Points-of-Interest (POIs), specific physical locations that someone might find useful or interesting, such as restaurants, shops, or landmarks, is fundamental.

In this chapter, we investigate the application of MTL to two critical POI-related tasks:

1. **POI Category Classification**: Classify the semantic category (e.g., shopping, travel) of a POI based on its features and context within a user's trajectory.

2. **Next-POI Prediction**: Predicting the category of the next POI a user will visit, given the sequence of their prior visits.

These tasks are ostensibly related, as both draw upon the same underlying user mobility data. Accurate POI category representations could inform next-location predictions, and conversely, sequential patterns could provide context for category classification. However, the tasks possess fundamentally different natures. POI Category Classification is primarily a **static classification task** that relies on the intrinsic, context-rich features of individual locations. In contrast, Next-POI Prediction is a **dynamic, sequential task** that depends on capturing temporal patterns and transitions.

This dichotomy raises a critical question: can a single, shared representation effectively serve two tasks with such distinct underlying characteristics? The assumption that MTL will be beneficial is not guaranteed. It is possible that forcing a shared encoder to learn features for both a static and a sequential task could result in **negative transfer**, where the joint training process actually hinders performance by learning a "compromise" representation that is not specialized enough for either task.

Therefore, the central hypothesis of this study is that a standard hard parameter-sharing MTL architecture will face significant limitations when applied to this specific task pairing, failing to achieve substantial and consistent improvements over well-tuned, specialized single-task models. Our work is thus framed as an empirical investigation into the boundary conditions of MTL's effectiveness, exploring whether the presumed task relatedness is sufficient to overcome their inherent structural differences.

To test this hypothesis, we conducted experiments on the Gowalla LBSN dataset [10, 75], comparing our MTL model against strong single-task baselines (HMRM [76] and MHA+PE [77]). Our results lend weight to our hypothesis. While the proposed models perform well, particularly in POI category classification, the MTL framework does not deliver the consistent gains often expected. The performance differences between the MTL and single-task models were frequently marginal and fell within standard deviations, suggesting their statistical performance was largely comparable. This outcome indicates that, for this problem, the architectural constraints and task dissimilarities may have offset the potential benefits of joint learning.

The main contributions of this chapter are as follows:

- We introduce an MTL framework to simultaneously address POI category classification and next-POI prediction.[1]

[1] The complete source code is publicly available at <https://github.com/VitorHugoOli/PoiMtlNet> for full reproducibility.

- We design task-specific heads to capture category co-occurrence and sequential transition dynamics, respectively.
- We provide a critical analysis of the limitations of a hard parameter-sharing MTL approach for this task pairing. Our findings highlight that significant task dissimilarity can challenge the effectiveness of MTL, serving as a case study on the importance of task compatibility in model design.

The rest of this chapter is organized as follows. In Section 3.2, we review the theoretical foundations of MTL and prior work on POI modeling. Section 3.3 describes our joint architecture, data preprocessing, and training protocol. Section 3.4 presents experimental results. Finally, Section 3.5 summarizes our findings and discusses future research directions.

## 3.2 Theoretical Foundations and Related Work

### 3.2.1 POI Classification and Next-POI Prediction

POI Category Classification refers to the task of inferring the semantic category of a location based on contextual information such as geographic coordinates, visit frequency, or user behavior.

Next-POI Prediction, in contrast, aims to predict which specific location a user is likely to visit next, given their past movement history.[2] It is a sequential task that requires modeling temporal and spatial patterns within user trajectories.

The prediction and classification of POIs are fundamental challenges in applications such as recommendation systems and urban planning. Xu et al. [4] propose the Tree-guided Multitask Embedding (TME) model, which aims to improve the semantic annotation of POIs using a hierarchical structure of categories and mobility-based representation learning.

POI Category Classification has been extensively addressed in location-based recommendation systems, with various techniques applied to capture patterns and improve prediction accuracy. In this context, Lim et al. [19] propose the Hierarchical Multitask Graph Recurrent Network (HMT-GRN), which uses multitask learning to simultaneously predict the next POI and its geographic region.

Although these approaches have made significant progress in the POI classification and next-location prediction fields, they tend to focus only on single-task objectives. None of the cited models propose a unified multitask learning (MTL) framework that jointly optimizes both

[2] This states the general formulation of the task as the literature poses it. The variant studied in this chapter predicts the *category* of the next visited place rather than the place itself, as defined in Section 3.1 and in the chapter preface.

POI category classification and next-POI prediction tasks. This motivates the development of solutions that use shared representations across related POI tasks.

### 3.2.2 Multitask Learning

MTL was formalized by Caruana [5] as an inductive transfer mechanism that enhances generalization by learning shared representations across related tasks. Early neural architectures employed *hard parameter sharing*, where a common set of hidden layers is used for all tasks, yielding strong regularization and reduced training time.

Recent surveys [58, 78] organize contemporary MTL research along five methodological dimensions: (i) *parameter sharing* (hard vs. soft); (ii) *relationship learning* (discovering task affinity or hierarchy); (iii) *feature routing* (e.g., cross-stitch, sluice networks, attention gating); (iv) *optimization* (conflict-aware gradient techniques); and (v) *pre-training and instruction tuning*. MTL has evolved from small, homogeneous task sets to dozens of heterogeneous objectives spanning computer vision, natural language processing, and recommender systems.

A fundamental architectural challenge in MTL is deciding *how much*, *where*, and *when* to share parameters across tasks. Three canonical sharing schemes have emerged:

#### Hard Parameter Sharing

A single encoder is shared by all tasks, followed by task-specific heads. This remains the simplest and most popular baseline, providing effective regularization [5].

#### Soft Parameter Sharing

Each task has its own encoder, but networks exchange information through learned cross-connections, such as Cross-Stitch units [40] or Sluice networks [79].

#### Mixture-of-Experts (MoE)

Multi-Gate Mixture-of-Experts (MMoE) architectures maintain a pool of shared expert subnetworks; each task learns a gating function to combine experts, outperforming monolithic sharing in large-scale recommendation settings [41].

To mitigate task interference and loss imbalance, several optimization techniques have been proposed: GradNorm equalizes gradient magnitudes across tasks [51]; MGDA finds a direction that decreases every task loss, unless the current point is already Pareto-stationary [46]; PCGrad removes conflicting gradient components [44]; and Dynamic Weight Averaging (DWA) reweights losses based on their relative learning speeds [52].

Despite these successes, MTL still faces several key challenges:

- **Negative Transfer**: Unrelated or adversarial tasks can degrade shared representations, harming individual task performance [79, 78].
- **Gradient Conflict**: Simultaneous optimization may produce opposing gradient directions, slowing or destabilizing convergence [44, 46].
- **Data Heterogeneity**: Variations in modality, label granularity, and dataset size complicate sampling strategies and minibatch construction [47, 45].
- **Scalability**: Routing complexity, memory footprint, and evaluation costs often grow superlinearly as the number of tasks increases [78, 58].

Recent work addresses these issues via task clustering [45], dynamic curricula, conflict-aware optimizers, and parameter-efficient adapters [58]. Nevertheless, principled criteria for task grouping, theoretical guarantees of Pareto efficiency, and energy-efficient training of very large MTL foundation models remain open research directions.

### 3.2.3 Multitask learning applied in POI

MTL has been explored in problems related to Points of Interest (POIs), primarily due to its ability to improve generalization by sharing information across correlated tasks. In POI modeling scenarios, such as user trajectory prediction and recommendation, MTL allows the learning of complementary objectives simultaneously, allowing for more accuracy.

Early MTL-based approaches such as MCARNN [59] employ recurrent neural networks with temporal attention mechanisms to jointly predict user activities and future visited locations. Similarly, the iMTL framework [61] uses an LSTM architecture to model next-activity prediction, incorporating temporal dynamics in user behavior modeling.

More recent works have begun incorporating contextual and semantic signals. TLR-M [62], for example, applies Transformer-based architectures to simultaneously predict the next POI and queue waiting time, demonstrating the value of attention-based models in capturing spatio-temporal dependencies. MTPR [80] combines LSTMs and adversarial learning to address uncertainty in check-ins and improve multitask POI recommendation both location and temporal context with a generative component.

Despite these advances, existing MTL approaches often focus on joint modeling of next-POI prediction and temporal or behavioral signals, while the relationship between POI category classification and next-POI prediction remains underexplored. Some Models such as TME [4] address category annotation using graph-based encoders, but treat prediction and classification separately.

This creates a gap in the literature regarding the joint modeling of semantic and sequential POI tasks. In contrast, our proposed model introduces a unified MTL framework that jointly

performs POI category prediction and next-POI classification using a hard parameter-sharing architecture. By modeling both tasks simultaneously, our approach promotes knowledge transfer across shared spatio-temporal representations, enabling more robust predictions even in data-sparse environments.

## 3.3 Methodology

This section details the proposed methodology for Point-of-Interest (POI) category classification and next-POI prediction using a MTL approach. We first describe the generation of POI embeddings and the preparation of data for both tasks. Subsequently, we present the architectures of the single-task baselines, which are also used as task-specific heads within our MTL framework. Finally, we elaborate on the proposed MTL architecture, including the hard parameter-sharing strategy and the Nash-MTL optimizer employed for gradient aggregation.

### 3.3.1 POI Embeddings and Data Preparation

This chapter computes POI embeddings using graph attention convolution along with contrastive learning based on Deep Graph Infomax (DGI). These embeddings are the input for the MTL model discussed in Section 3.3.2.

#### 3.3.1.1 POI Embeddings

A point of interest is defined as a tuple $\langle id, lat, long, cat \rangle$, where $id$ serves as a unique identifier, $lat$ and $long$ represent its coordinates, and $cat$ denotes its category, such as restaurant or pharmacy. A place suffers from complementarity effect [81], implying that a location is affected by its neighbors (e.g. leisure zones are often near residential districts). Hence, we propose a graph-based contrastive method to capture this spatial locality behavior, generating a POI embedding to serve as input to our MTL model.

To capture the spatial locality and capture neighborhood information, we model the weighted graph $G(V, E)$ as a Delaunay Triangulation, where each vertex $v \in V$ represents a POI and edge $e_{ij} \in E$ defines the connection between POI $p_i$ and a POI $p_j$. Besides that, following [82] the weight of an edge $e_{ij}$ is defined as $w_{ij} = \log((1 + D^{1.5}/1 + d_{ij}^{1.5}))$, where D is the diagonal length of bounding box that encloses the coordinates of POIs, and $d_{ij}$ is the geodesic distance between $p_i$ and $p_j$. The node feature matrix is based on category one-hot encoding of each POI, represented by $X \in \mathbb{R}^{n_c}$, where $n_c$ is the number of possible categories.[3]

[3] The released implementation feeds the network the mean of the one-hot vectors of a POI's graph neighbors, with the POI's own vector excluded, rather than the POI's own one-hot vector (`research/embeddings/dgi/preprocess.py`), a departure from the published description. The distinction matters for how the embedding should be read: the input describes a POI's neighborhood, so the static task the embedding supports is spatial homophily rather than recall of the POI's own label.

Therefore, to encode the Delaunay graph we apply a graph attention convolution layer [27], yielding an embedding $h_i \in \mathbb{R}^d$, where $d$ is the dimension of the output. In addition, we utilize a Deep Graph Infomax (DGI) [31] as a contrastive learning method to maximize the mutual information between local representations (node embeddings) and the global graph embedding compared to a corrupted version of a graph, which is guided by the Equation 3.1.

$$\mathcal{L} = \frac{1}{2|V|}\Big(\sum_{i=1}^{|V|}\mathbb{E}_{(X,G)}[\log \mathcal{D}(\vec{h}_i, \vec{s})] + \sum_{j=1}^{|V|}\mathbb{E}_{(\tilde{X},G)}[\log(1 - \mathcal{D}(\tilde{\vec{h}}_j, \vec{s}))]\Big) \tag{3.1}$$

where $X$ and $G$ denote the positive samples for the node feature matrix and graph adjacency, respectively, while $\tilde{X}$ is the corrupted version of the feature matrix. To corrupt the features, we employ a random shuffling method. Additionally, $\vec{s}$ is the global graph embedding derived by applying global mean pooling to the embedding matrix $H = [h_i, h_{i+1}, \ldots, h_n]$ in our model. In addition, $\mathcal{D}$ is a discriminator that differentiates each positive example from its corrupted version, learning from their dissimilarities; in our POI Embedding, this discriminator is implemented as a linear model.

#### 3.3.1.2 Data Processing for POI Category Classification

Each POI is represented by its 64-dimensional DGI embedding $\mathbf{e} \in \mathbb{R}^{64}$. The dataset therefore consists of pairs $(\mathbf{e}, c)$, where $c$ is the POI's ground-truth category to be predicted.

#### 3.3.1.3 Data Processing for Next-POI Category Prediction

This task predicts the category of the next POI a user will visit from their recent check-in history.

1. **Trajectory building.** Check-ins (user, POI, timestamp) are ordered chronologically per user; users with fewer than five visits are discarded.
2. **Sequence extraction.** From each trajectory we draw non-overlapping windows of length $L_h = 9$: $(p_1, \ldots, p_9)$ as context and $p_{\text{target}}$ as the next POI.
3. **Feature/label construction.**
   - The input is the concatenation of the 64-dimensional embeddings of $p_1$–$p_9$, yielding a $9 \times 64 = 576$-dim vector. Histories shorter than nine are padded by a special POI whose embedding is $\mathbf{0}$.
   - The label is the category of $p_{\text{target}}$.

This format allows the model to infer the category of the next POI solely from the embeddings of the previous visits.

### 3.3.2 Multitask Learning Architecture

Our proposed Multitask Learning (MTL) architecture, depicted in Figure 1, is built upon a hard parameter-sharing scheme for the joint training of POI Category Classification and Next-POI Prediction. This design integrates Feature-wise Linear Modulation (FiLM) layers to condition the shared representations, a technique aimed at modulating task interactions and mitigating negative transfer [39, 45].

#### Architecture Overview

Input features for POI Category Classification ($\mathbf{x}^{(c)}$) and Next-POI Prediction ($\mathbf{x}^{(n)}$) are first processed by separate, task-specific encoders. These encoders, implemented as MLPs, map the raw input features into a shared latent space of dimension $d_{\text{shared}}$.

#### Feature-wise Linear Modulation (FiLM)

To allow for task-specific adaptation within the shared pathway, we employ FiLM layers [39]. A unique, learnable task embedding $\mathbf{e}_t$ for each task $t$ is used to generate a scaling vector $\gamma_t$ and a shifting vector $\beta_t$. These vectors modulate the encoded features $\mathbf{h}_{\text{enc}}^{(t)}$ from each task-specific encoder:

$$\text{FiLM}(\mathbf{h}_{\text{enc}}^{(t)} \mid \mathbf{e}_t) = \gamma_t \odot \mathbf{h}_{\text{enc}}^{(t)} + \beta_t$$

This lightweight operation conditions the features before they enter the fully shared layers of the network.

#### Shared Processing Layers

The FiLM-modulated representations are processed by a sequence of shared layers. This module begins with a linear transformation followed by a LeakyReLU activation, LayerNorm, and dropout, and continues through $L_{\text{shared}}$ residual blocks. This hard parameter-sharing approach regularizes the model by forcing it to learn a common, robust feature representation for both tasks [83].

#### Task-Specific Heads

Finally, the processed shared representations are passed to dedicated, unshared task-specific heads. These heads generate the final predictions for each task, allowing for specialized output generation and isolated gradient computation during backpropagation.

### Rationale for Hard Parameter Sharing

The choice of hard parameter sharing is motivated by its efficiency and regularization benefits, which are critical for our target application. This approach is supported by several factors:

- **Computational Efficiency:** Unlike soft-sharing methods that often introduce task-specific parameters and increase computational overhead, hard sharing maintains a compact model. This is crucial for deployment on resource-constrained edge devices (e.g., NVIDIA Jetson platforms).
- **Implicit Regularization:** By constraining the hypothesis space, hard sharing acts as a regularizer, often leading to more generalizable models, especially when tasks are related [83]. Seminal work by Caruana demonstrated significant error reduction with this approach [5].
- **Empirical Performance:** In practice, sharing one network across tasks reduces inference cost, since one network is evaluated instead of one per task [45].[4]

This architecture strategically balances model capacity with knowledge transfer, prioritizing efficiency for potential real-time recommendation scenarios.

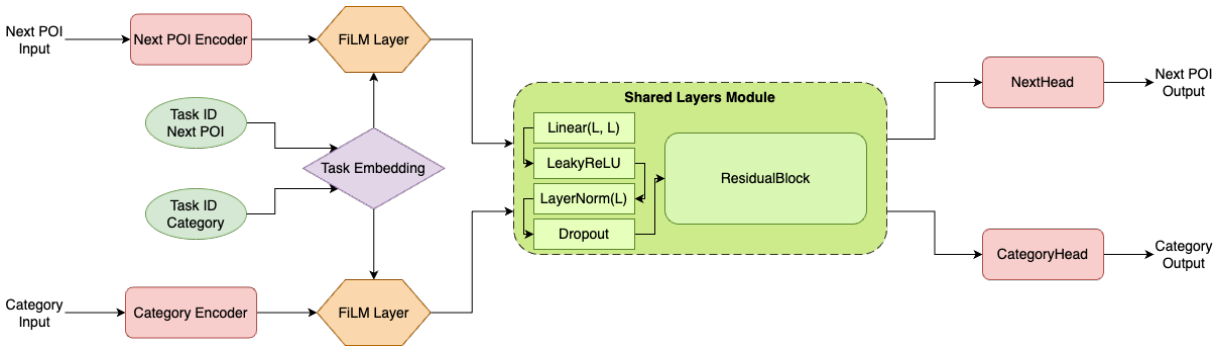


Figure 1 – The proposed Multitask Learning (MTL) architecture. Task-specific encoders process inputs for POI Category Classification and Next-POI Prediction. Feature-wise Linear Modulation (FiLM) layers adapt these features using learnable task embeddings. The modulated representations are then processed by a shared path of residual blocks. Finally, dedicated task-specific heads generate the respective outputs.

[4] The published sentence read "hard parameter sharing frequently matches or exceeds the performance of more complex architectures on many benchmarks, while offering faster training and inference". The accuracy and training-speed halves are corrected here rather than reproduced. The cited work names improved accuracy and reduced training time among the benefits joint training may have "in theory", and then argues the other way empirically, reporting that joint training "often leads to inferior overall performance as task objectives can compete"; its contribution is a framework for assigning tasks to several networks so that competing tasks are separated, and it describes that framework as costly at training time.

### 3.3.3 Nash-MTL Optimizer

MTL necessitates a unified update direction that benefits all tasks, even when their gradients conflict. Nash-MTL formulates this as a cooperative $K$-player bargaining game [47], where each task acts as an "agent" seeking to maximize its utility (loss reduction). The optimizer selects a step that is (i) beneficial to all tasks, (ii) scale-invariant to arbitrary loss re-weightings, and (iii) proportionally fair, ensuring no alternative direction increases one task's utility without decreasing the product of all utilities [47]. The unique solution satisfying these axioms is the Nash Bargaining Solution (NBS). This approach allows Nash-MTL to "negotiate" at each iteration, yielding a compromise descent direction that improves every loss while maximizing the joint progress of the entire system.

**Formal Formulation.**

Given task-specific objectives $\{\mathcal{L}_k(\boldsymbol{\theta})\}_{k=1}^{K}$ and their gradients $\mathbf{g}_k = \nabla_{\boldsymbol{\theta}}\mathcal{L}_k$ at current parameters $\boldsymbol{\theta}$, Nash-MTL frames gradient aggregation as a bargaining game to determine the common update direction [47]. The utility for each task $k$ is its signed improvement, $u_k(\Delta\boldsymbol{\theta}) = \mathbf{g}_k^{\top}\Delta\boldsymbol{\theta}$, where $\Delta\boldsymbol{\theta}$ is the parameter update within an $\varepsilon$-ball [47]. The NBS is found by maximizing the weighted geometric mean of utilities:

$$\Delta\boldsymbol{\theta}^{*} \;=\; \arg\max_{\Delta\boldsymbol{\theta}\in\mathcal{B}_{\varepsilon}} \sum_{k=1}^{K} \log\big(u_k(\Delta\boldsymbol{\theta})\big) \quad \text{s.t. } u_k(\Delta\boldsymbol{\theta}) > 0,\ \forall k. \tag{3.2}$$

By setting $\Delta\boldsymbol{\theta} = \sum_k \alpha_k \mathbf{g}_k$ with positive weights $\boldsymbol{\alpha}$, the solution simplifies to solving the nonlinear system $\big(\mathbf{G}^{\top}\mathbf{G}\big)\boldsymbol{\alpha} \;=\; \boldsymbol{\alpha}^{-1}$ [47], where $\mathbf{G} = \big[\mathbf{g}_1 \ \ldots \ \mathbf{g}_K\big]$ [47]. The parameter update is then $\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta\,\mathbf{G}\boldsymbol{\alpha}$, with $\eta$ chosen to ensure monotonic loss decrease and convergence to a Pareto-stationary point [47].

**Practical Aspects.**

Nash-MTL is architecture-agnostic.[5] Its invariance to the scale of the individual task gradients, the second of the axioms listed above, balances heterogeneous tasks without manual re-weighting. For efficiency, task weights can be updated less frequently, significantly reducing runtime while maintaining performance [47].

[5] The published sentence continued "and requires only two matrix-vector products per iteration". That cost claim is corrected here rather than reproduced: it is not supported by the method's own paper, and it understates the cost.

**Parameter Partition.**

In our model (Figure 1), we explicitly expose two disjoint parameter sets:

$$\Theta_{\text{shared}} = \{\texttt{task\_emb}, \texttt{film}, \texttt{shared\_layers}\},$$
$$\Theta_{\text{specific}} = \{\texttt{category\_encoder}, \texttt{next\_encoder}, \texttt{CategoryHead}, \texttt{NextHead}\}.$$

This partition enables independent learning-rate scheduling or regularization if domain shifts between tasks emerge during fine-tuning.

The Nash-MTL optimizer was employed for its ability to aggregate tasks gradients in a balanced and principled manner, formulating the optimization process as a bargaining problem and aiming for joint progress across all tasks. In our evaluation, Nash-MTL was compared with different strategies, including PCGrad [44] and an approach with no optimizer. Among these, it was found that Nash-MTL consistently yielded a better overall performance of the MTL model, with a lower combined multitask loss, which supported our decision to adopt it as our choice of optimizer.

## 3.4 Experimental Setup and Results

### 3.4.1 Dataset and Evaluation Metrics

To evaluate our model, we use a subset of the public *Gowalla* check-in dataset, originally collected by Cho *et al.* [10]. We focus our analysis on the data from the U.S. state of Florida, which is one of the most active regions in the original dataset. This subset comprises 20,301 users, 65,009 unique Points-of-Interest (POIs), and 990,518 check-ins.

Following previous work, each POI is mapped to one of seven high-level categories: *Community*, *Entertainment*, *Food*, *Nightlife*, *Outdoors*, *Shopping*, and *Travel*. These categories will be used for the POI category prediction task.

**Evaluation protocol**

To ensure a robust evaluation of our models, all experiments were conducted using a 5-fold cross-validation methodology. The folds are formed by a stratified splitter over the samples rather than over the users, so the check-ins of one user may appear in both training and validation. For the category task the sample unit is the place, so no place spans two folds. The code of record pins a single random seed, so the five folds constitute one repetition of the experiment rather than several, and the means and standard deviations reported below are the spread across those five folds at that seed rather than across independent repetitions. Within a

fold, training runs for the full number of epochs configured, without early stopping, and each task is read at the epoch of its own highest validation macro-F1, measured on the same fold the score is reported on. For every category, we report precision, recall, and $F_1$-score, and utilize the macro-average of these metrics to summarize overall performance. Class-wise metrics are crucial because category frequencies are highly skewed in real LBSN data.

### 3.4.2 Discussion and Comparison with Established Models

In this section, we discuss the performance of our proposed MTL model and its single-task (Single) counterpart. For a comprehensive evaluation, we selected two state-of-the-art approaches as baselines. The Human Mobility Representation Model (HMRM), introduced by Chen et al. (2020) [76], is designed for POI category classification. HMRM calculates POI embeddings by considering users' temporal visiting patterns and the co-occurrence of locations within an individual's historical trajectory. To quantify the relationship between locations and visiting times, it employs Point-wise Mutual Information (PMI). The model then utilizes matrix factorization techniques to generate latent representations (embeddings) from various inputs, and these embeddings are subsequently used to train a Support Vector Machine (SVM) for predicting POI categories.

For the task of next POI category prediction, we compare our models against MHA+PE, a solution proposed by Zeng et al. (2019) [77]. This model enhances a recurrent neural network with a Multi-Head Attention (MHA) mechanism, originally developed for machine translation by Vaswani et al. (2017) [84], and incorporates Positional Encoding (PE). The MHA mechanism allows the model to extract correlations from different parts of a sequence, effectively capturing the relevance of various elements within a user's trajectory, while positional encoding helps to propagate the order of records in a sequence.

#### 3.4.2.1 POI Category Classification

As shown in Table 2, both our MTL and Single models outperform HMRM [76] in every POI category in terms of F1-score, precision, and recall. For instance, in the 'Shopping' category, our MTL model achieves an F1-score of $62.51 \pm 0.94$ compared to HMRM's $46.69 \pm 0.81$. Similarly, for 'Food', the MTL model scores $57.43 \pm 1.46$ against HMRM's $28.44 \pm 0.42$. While both MTL and Single models are competitive, the Single model shows marginally better F1-scores in several categories like 'Community' ($53.11 \pm 0.58$) and 'Outdoors' ($47.75 \pm 0.89$), whereas the MTL model excels in 'Food' and 'Shopping'. Overall, our approaches demonstrate a substantial improvement over the HMRM baseline for this task.

Table 2 – POI category classification results for Florida (per-category F1-score, precision, and recall, mean ± standard deviation over the 5 folds; the better of the MTL and Single values per row in bold, as in the published table).

| | | MTL | Single | HMRM |
|---|---|---|---|---|
| F1 | Community | 52.13 ± 0.90 | **53.11 ± 0.58** | 20.25 ± 0.52 |
| | Entertainment | 41.44 ± 1.29 | **41.73 ± 1.65** | 13.07 ± 1.14 |
| | Food | **57.43 ± 1.46** | 56.70 ± 0.84 | 28.44 ± 0.42 |
| | Nightlife | 29.94 ± 2.27 | **34.22 ± 2.08** | 25.54 ± 1.73 |
| | Outdoors | 47.19 ± 1.01 | **47.75 ± 0.89** | 16.11 ± 0.68 |
| | Shopping | **62.51 ± 0.94** | 61.88 ± 1.01 | 46.69 ± 0.81 |
| | Travel | 43.16 ± 1.43 | **43.20 ± 1.94** | 15.25 ± 1.19 |
| Precision | Community | **51.39 ± 1.77** | 49.54 ± 1.70 | 21.13 ± 0.75 |
| | Entertainment | 44.08 ± 1.46 | **44.47 ± 3.66** | 14.52 ± 1.34 |
| | Food | 58.05 ± 1.23 | **59.04 ± 1.12** | 48.94 ± 0.88 |
| | Nightlife | **36.07 ± 2.05** | 32.52 ± 1.83 | 18.34 ± 1.34 |
| | Outdoors | 46.39 ± 1.12 | **47.74 ± 1.09** | 13.00 ± 0.48 |
| | Shopping | 60.24 ± 1.21 | **61.59 ± 1.44** | 39.63 ± 0.82 |
| | Travel | **46.48 ± 1.12** | 44.69 ± 4.89 | 24.58 ± 1.91 |
| Recall | Community | 52.93 ± 1.35 | **57.37 ± 2.46** | 19.48 ± 0.99 |
| | Entertainment | 39.14 ± 1.69 | **39.88 ± 4.93** | 11.90 ± 1.08 |
| | Food | **56.99 ± 3.71** | 54.60 ± 2.01 | 20.05 ± 0.32 |
| | Nightlife | 25.80 ± 3.54 | **36.77 ± 5.86** | 42.13 ± 3.28 |
| | Outdoors | **48.09 ± 2.11** | 47.79 ± 1.51 | 21.27 ± 1.78 |
| | Shopping | **65.04 ± 2.69** | 62.28 ± 2.83 | 56.82 ± 0.98 |
| | Travel | 40.39 ± 2.73 | **43.08 ± 7.15** | 11.07 ± 0.95 |

#### 3.4.2.2 Next POI Category Prediction

The results for next POI category prediction, detailed in Table 3, indicate a competitive performance landscape. The MHA+PE model [77] shows strong F1-scores in several categories, notably 'Food' (43.47±0.50) and 'Shopping' (43.53±1.66), outperforming our MTL and Single models in these instances. However, our MTL model achieves the best F1-score in 'Travel' (64.61 ± 1.11) and 'Nightlife' (22.07 ± 0.52), surpassing MHA+PE in these two categories. The Single model also shows competitive results, leading in 'Entertainment' (26.06 ± 1.01) and 'Outdoors' (22.45 ± 0.81). This suggests that while MHA+PE is highly effective for certain POI categories, our MTL and Single models offer superior or comparable performance in others, particularly where different sequential patterns might be exploited. One further point concerns how these leads are distributed. The MTL and Single models do not lead consistently across the categories, each obtaining the better F1-score in some of them, and this uneven distribution could make either model appear worse in the categories where the other one leads.

Overall, while our proposed models show strong performance in POI category classification against HMRM [76], the next POI category prediction task presents a more competitive

Table 3 – Next POI category prediction results for Florida (per-category F1-score, precision, and recall, mean ± standard deviation over the 5 folds; best value per row in bold, second-best underlined).

| | | MTL | Single | MHA+PE |
|---|---|---|---|---|
| F1 | Community | 34.79 ± 1.05 | 34.90 ± 0.61 | **35.83 ± 1.70** |
| | Entertainment | 24.21 ± 1.75 | **26.06 ± 1.01** | 25.48 ± 0.86 |
| | Food | 28.06 ± 3.55 | 26.35 ± 2.97 | **43.47 ± 0.50** |
| | Nightlife | **22.07 ± 0.52** | 21.79 ± 0.30 | 1.55 ± 1.50 |
| | Outdoors | 21.61 ± 0.50 | **22.45 ± 0.81** | 12.56 ± 1.94 |
| | Shopping | 42.58 ± 2.06 | 42.18 ± 1.13 | **43.53 ± 1.66** |
| | Travel | **64.61 ± 1.11** | 64.32 ± 0.88 | 26.91 ± 0.97 |
| Precision | Community | 30.22 ± 1.36 | 30.04 ± 1.08 | **42.59 ± 1.48** |
| | Entertainment | 32.64 ± 2.55 | 28.63 ± 2.85 | **38.72 ± 1.56** |
| | Food | 39.05 ± 0.70 | **39.87 ± 1.48** | 35.91 ± 0.81 |
| | Nightlife | 17.71 ± 2.40 | 15.27 ± 0.67 | **39.67 ± 23.94** |
| | Outdoors | 19.63 ± 2.93 | 19.50 ± 2.08 | **46.83 ± 1.82** |
| | Shopping | 42.57 ± 0.78 | **43.47 ± 1.08** | 39.04 ± 1.06 |
| | Travel | 60.64 ± 1.87 | **63.21 ± 0.67** | 36.64 ± 1.74 |
| Recall | Community | 41.54 ± 5.27 | **41.92 ± 3.49** | 31.16 ± 3.20 |
| | Entertainment | 19.52 ± 3.18 | **24.16 ± 2.16** | 19.05 ± 1.25 |
| | Food | 22.19 ± 4.32 | 19.94 ± 3.75 | **55.12 ± 1.46** |
| | Nightlife | 32.13 ± 9.47 | **38.35 ± 3.11** | 0.80 ± 0.79 |
| | Outdoors | 25.06 ± 4.26 | **27.07 ± 3.40** | 7.30 ± 1.32 |
| | Shopping | 42.87 ± 4.56 | 41.05 ± 2.33 | **49.37 ± 3.74** |
| | Travel | **69.36 ± 4.15** | 65.52 ± 2.25 | 21.30 ± 1.03 |

landscape when compared against a sophisticated sequential model like MHA+PE [77]. Comparing the MTL and Single models directly reveals that the multitask learning approach did not yield the anticipated substantial improvements over the single-task baseline across both tasks. Crucially, many of these observed differences in F1-scores are minor and often fall within the reported standard deviations, suggesting that the performance of the MTL and Single models is largely comparable, without a clear or consistent advantage for the multitask learning setup in these experiments.

### 3.4.3 Convergence Comparison

To further evaluate the practical implications of the MTL approach compared to single-task models, we conducted a convergence experiment. We measured the wall time, number of epochs, and Mega Floating Point Operations (MFLOPs) required for the MTL model and the individual single-task models (SingleClass and SinglePred) to reach predefined target average F1-scores. The target F1-score for the *Category* prediction task was set to 47, and for the *Next* POI category classification task, it was set to 32.2. These specific F1 values were chosen

because they are close to the best-achieved results for each respective task, thereby representing a comparable and significant level of predictive performance for the models to attain. To ensure robust measurements for this experiment, all models were evaluated through a 5-fold cross-validation process. The specific metrics are detailed in Table 4.

The convergence measurements show a clear wall-time disadvantage for the joint model, while the MFLOPs column does not follow the same pattern. Table 4 reports the three metrics for each model.

Table 4 – Convergence metrics to reach the target F1-score for each model (wall time in seconds, number of epochs, and MFLOPs; 5-fold cross-validation).

| **Model** | **Time (s)** | **Epochs** | **MFLOPs** |
|---|---|---|---|
| Category | 16.26 | 3.8 | 2.315 |
| Next | 18.71 | 3.2 | 0.012 |
| MTL | 80.88 | 3.2 | 0.234 |

The results from this experiment were contrary to the potential expectation that an MTL framework might offer training efficiencies. Our findings indicated that the MTL model took substantially longer to converge to the target F1-scores. Specifically, the MTL approach required 80.88 s of wall time, about 2.3 times the cumulative 34.97 s of the individual single-task models. In terms of MFLOPs, in contrast, the table does not show a higher cost for the MTL setup: the MTL model required 0.234 MFLOPs, against 2.315 MFLOPs for the Category model and 0.012 MFLOPs for the Next model. This suggests that while attempting to learn multiple tasks simultaneously, the MTL model, in this configuration, incurred a significantly higher overhead in wall time. Given these observations, for deployment scenarios where convergence speed is a critical factor, employing separate, optimized single-task models would likely be a more convenient strategy.[6]

## 3.5 Conclusion and Future Work

In this chapter, we introduced a Multitask Learning (MTL) framework employing hard parameter-sharing, task-specific encoders, and FiLM layers to jointly address POI Category Classification and Next-POI Prediction using DGI-based POI embeddings.

Essentially, the MTL approach did not consistently demonstrate superior performance over its single-task counterparts and exhibited higher computational demands in terms of convergence wall time. These findings indicate that the anticipated benefits of MTL are not universally guaranteed and are highly dependent on task compatibility and architectural design.

[6] Experiments conducted on Apple M2 Pro (10-core CPU, 16-core GPU) with 32GB unified memory running macOS 15.5. Key software versions: Python 3.9.6, PyTorch 2.6.0.dev20241014+cpu, NumPy 1.26.4, scikit-learn 1.5.2, CVXPY 1.5.2. Training utilized PyTorch MPS backend for Apple Silicon acceleration.

The lack of significant improvement from the MTL model prompts an analysis of the potential underlying causes, especially since optimizers like Nash-MTL mitigated overt gradient conflicts. We hypothesize three primary factors contributed to this outcome:

- **Subtle Negative Transfer due to Task Dissimilarity:** Although related, the two tasks possess fundamentally different natures. POI Category Classification is a static task that relies on the intrinsic, context-rich features of a single POI embedding. In contrast, Next-POI Prediction is a dynamic, sequential task that depends on capturing temporal patterns and transitions within a user's trajectory. The shared encoder may have been forced to learn a "compromise" representation that was not specialized enough for either task, thus failing to outperform the focused single-task models.
- **Task Difficulty and Representation Mismatch:** The performance gap between the two tasks (with category classification achieving higher F1-scores) suggests an imbalance in difficulty. The representation learned by the shared layers might have become biased towards the features required for the simpler, static classification task, inadvertently hindering its effectiveness for the more complex sequential prediction task. The shared representation may not have been rich enough to simultaneously encode both semantic properties and sequential dynamics effectively.
- **Architectural Restrictiveness:** Our choice of a hard parameter-sharing architecture, while efficient and strongly regularizing, may have been too restrictive for the tasks. The distinct nature of the tasks might require more flexible computational pathways. A single shared block may be insufficient to learn a representation that benefits both, suggesting that soft-sharing or expert-based models could be more appropriate.

These insights contribute to the broader understanding of MTL's practical challenges. Future research will proceed along several directions motivated by these hypotheses. We plan to explore alternative parameter-sharing mechanisms, such as **soft sharing (e.g., Cross-Stitch Networks) or Mixture-of-Experts (MoE) models**, to test the hypothesis that the hard-sharing architecture was overly restrictive. In addition, investigating **advanced multitask optimizers and loss-balancing schemes** beyond gradient conflict mitigation could address the task difficulty imbalance. Finally, a deeper analysis of **task relatedness**, perhaps using techniques to measure feature representation overlap, could help quantify the degree of negative transfer and guide future architectural choices.

# 4 ST-MTLNet: Spatio-Temporal POI Representations for Multitask Learning

*This chapter is a translated reproduction of the paper "ST-MTLNet: Representações Espaço-Temporais de Pontos de Interesse para Aprendizado Multitarefa," published in Portuguese in the Anais do X Workshop de Computação Urbana (CoUrb 2026, an SBRC workshop), pages 323 to 336, DOI 10.5753/courb.2026.22960 [85]. This study and Chapter 3 share one evaluation protocol, which stratifies the cross-validation split by sample rather than by user; the user-disjoint cross-validation of Chapter 5 arrives only with the final study. The conclusions reported here are those of the time, for that configuration. This chapter isolates the representation effect with MTLnet as its only baseline; it does not revisit the MTL-versus-single-task question, which Chapter 5 reopens. As in Chapter 3, the term "Next-POI Prediction" used here denotes the frame's* next category *task (the category of the next visited place), not the exact-place task the dissertation calls* next place*. After publication, we established that the input to this chapter's static task contains the label it predicts: the venue-type feature maps one-to-one onto the seven top-level categories across the Gowalla state subsets used, so the reported static-task accuracy measures that lookup rather than learned semantic inference. This does not affect the sequential task, whose input is a check-in history and whose target category is never present in that history; every claim this chapter makes about the sequential task, including the improvement the decomposed representation produces, stands as published.*

## 4.1 Introduction

Location-Based Social Networks (LBSNs), such as Gowalla [10, 75], record user check-ins with geographic location, visit time, and category of the visited place. In this context, a Point of Interest (POI) corresponds to a semantically identifiable physical place, such as restaurants, stores, airports, or parks. The analysis of these data is relevant to applications such as recommendation and urban mobility. In this scenario, two tasks stand out:

1. **POI Category Classification**: classify the semantic category (e.g., *Food*, *Shopping*, *Travel*) of a POI from its features.
2. **Next-POI Prediction**: predict the category of the next POI that a user will visit, given the historical sequence of their check-ins.

In principle, these tasks may be treated as related, since they share the same mobility dataset and the same semantic space of categories, which makes multitask learning (MTL) a promising strategy to exploit information shared between them [5]. However, they impose distinct demands. POI Category Classification is a non-sequential task, since it depends mainly on the attributes of the POI itself and on its spatial context. Next-POI Prediction is explicitly sequential, requiring the modeling of temporal order, recurrence, and transitions along the user trajectory. Thus, the information useful for one task may not benefit the other in the same way.

MTLnet, proposed in [1], exploits this potential by modeling the two tasks in a single multitask architecture with hard parameter sharing. However, the model uses exclusively vector representations based on the graph structure, generated by *Deep Graph Infomax* (DGI), which encodes spatial and categorical information in a single 64-dimensional vector, without explicitly incorporating continuous geographic coordinates or temporal visitation patterns.

Given this, the question arises of whether decomposing the input into specialized encoders for space, time, and category produces a base more suitable for the two tasks than the monolithic embedding of DGI. The hypothesis is that decoupled representations capture complementary dimensions of human mobility that are not explicitly modeled by the original approach.

In this sense, this chapter proposes ST-MTLNet (*Spatial-Temporal MTLNet*), which integrates three independent representations: a continuous spatial representation for geographic coordinates, a temporal representation (Time2Vec) for visitation patterns, and a hierarchical categorical representation for regional and structural relationships between POIs. In the spatial dimension, we evaluate two architectures with distinct assumptions, SIREN and Sphere2Vec-M, originally validated in geospatial tasks of remote sensing and ecology [86], but still little explored in multitask learning of POIs in LBSNs.

The experiments were carried out with the public Gowalla dataset [75] in the states of Florida, California, and Texas. In categorical classification, ST-MTLNet outperforms MTLnet in all evaluated category-state combinations, with average gains per state of 20.2 to 22.0 percentage points, considering the better of the two spatial encoders in each combination. In Next-POI Prediction, the proposed model outperforms the baseline in most scenarios, with emphasis on categories such as *Food*. The comparison between SIREN and Sphere2Vec-M also shows that, although the modular approach is consistently superior to the baseline, the most suitable spatial encoder depends on the geographic distribution of the POIs in each territory.

Therefore, the main contributions of this chapter are:

- The proposal of ST-MTLNet, which incorporates continuous spatial, temporal, and hierarchical categorical representations into MTLnet.
- The comparative evaluation of SIREN and Sphere2Vec-M in three states with distinct

territorial structures.

- The public release of the source code and the experimental pipeline.[1]

This chapter is organized as follows: Section 4.2 presents the related work. Section 4.3 describes the proposed methodology. Section 4.4 discusses the experimental results. Finally, Section 4.5 presents the conclusions and future work.

## 4.2 Related Work

POI prediction and classification are fundamental challenges in location-based recommendation systems and in urban mobility analysis. This section organizes the related work into four axes: graph-based representations, spatial encoders, temporal representations, and multitask learning applied to POIs.

### 4.2.1 Graph-Based POI Representations

Modeling POIs through graphs allows capturing spatial and semantic neighborhood relationships between places. The POI2Vec model [87] adapts the Word2Vec architecture to generate POI embeddings that incorporate the geographic influence between places through a geographic binary tree structure. CATAPE (*Category-Aware Location Embedding*) [88] extends this idea by incorporating categorical and sequential information, capturing the geographic influence between POIs based on the temporal sequence of user visits.

In a self-supervised approach, *Deep Graph Infomax* (DGI) [31] maximizes the mutual information between local (node) and global (graph) representations to generate embeddings without explicit supervision. HGI (*Hierarchical Graph Infomax*) [32] advances this line by operating at multiple hierarchical levels (POI, region, and city), producing representations enriched with regional information through graph convolutions and multi-attention mechanisms.

However, these approaches encode spatial information implicitly through the graph topology, without directly using the geographic coordinates as an input signal for the representation.

### 4.2.2 Spatial Representations

The direct use of geographic coordinates to generate latent spatial features is an approach distinct from graph-based representations. The TorchSpatial framework [86] establishes a unified benchmark for evaluating spatial representation strategies, allowing systematic comparison between different architectures.

[1] The complete source code is available at <https://github.com/TarikSalles/Spatial_Embeddings>

The SIREN methodology (*Sinusoidal Representation Networks*) [34], applied to the geographic context [37], models continuous functions through sinusoidal activations, being able to represent high-frequency signals in spatial data. Sphere2Vec [36] operates directly on spherical coordinates with multiple resolution scales, preserving geodesic distance properties that are lost in planar projections.

Although these strategies have been evaluated on geospatial tasks such as species prediction and population estimation, their application can be extended to POI prediction and classification models as proposed in this chapter.

### 4.2.3 Temporal Representations

The temporal dimension is central to POI prediction tasks, since user visitation patterns present regularities (e.g., meal times, weekly movements). The LSTPM model (*Long- and Short-Term Preference Modeling*) [89] captures short- and long-term preferences using a Geo-Dilated RNN that relates non-consecutive POIs through temporal sequences of visits and geographic information. TransTARec [90] proposes an adaptive temporal translation model that unites representations of the POI, the timestamp, and user preferences for Next-POI Prediction.

These models incorporate the temporal dimension mainly in the context of the user's visit sequence. In this sense, Time2Vec [33] proposes a temporal representation that combines linear terms with sinusoidal functions of learnable frequency and phase, capturing periodic and non-periodic patterns from continuous temporal values.

### 4.2.4 Multitask Learning for POI Tasks

Multitask learning (MTL) [5] has been explored in POI tasks mainly to combine Next-POI Prediction with temporal, behavioral, or regional signals. MCARNN [59] integrates sequential dependencies and temporal regularities to simultaneously predict future activities and places. MTPR [80] jointly models location and temporal context through geographic LSTMs and adversarial learning. In a more recent line, Halder et al. [63] propose a multitask model based on *Transformers* to recommend the next POI and simultaneously predict the waiting time.

In categorical classification, TME [4] explores a hierarchical category structure for semantic annotation of POIs, while HMT-GRN [19] combines Next-POI Prediction and prediction of its geographic region in a recurrent model with hierarchical graphs. Together, these works indicate that MTL is promising in POI problems, but there is still little exploration of architectures that jointly address *POI Category Classification* and *Next-POI Prediction* with continuous spatial and explicit temporal representations. This gap motivates the investigation proposed in this chapter.

### 4.2.5 The MTLnet framework

The baseline of this study is MTLnet, a joint architecture for this task pair [1]. MTLnet addresses POI Category Classification and Next-POI Prediction in one model with hard parameter sharing: task-specific encoders project the input embeddings into a shared latent space, FiLM modulation conditions the shared layers on the task identity, and specialized heads produce the two outputs, with Nash-MTL balancing the task gradients during training. Its input is a single 64-dimensional embedding per POI, produced by Deep Graph Infomax over a spatial graph.

That evaluation [1] found that this joint model performed on par with the dedicated single-task models at a higher training cost, a conclusion of the time for that configuration, and hypothesized that the shared input representation might not be rich enough for the two tasks. This chapter takes that hypothesis as its starting point: it keeps the MTLnet architecture unchanged and replaces only its input representation, as detailed in Section 4.3.

## 4.3 Methodology

As discussed in Section 4.2, MTLnet [1] uses a single vector $\mathbf{E}_{DGI} \in \mathbb{R}^{64}$ as input for both tasks, jointly encoding spatial and categorical information through the graph topology. One limitation of this monolithic approach is that it does not incorporate continuous geographic coordinates or temporal visitation patterns, factors that, as shown by works on spatial [86] and temporal [33] encoding, carry information complementary to the topological relationships of the graph.

The central design decision of this chapter is to decompose the unimodal DGI representation into three independent 64-dimensional components, trained separately and integrated by concatenation: a continuous spatial encoder, a temporal encoder (Time2Vec), and a hierarchical categorical encoder (HGI). The underlying hypothesis is that a decoupled representation, with specialized encoders trained with loss functions suited to each dimension of the information, produces more expressive representations than a unimodal representation that encodes all dimensions jointly. To evaluate this hypothesis and investigate the impact of the spatial encoding strategy, two encoders with distinct architectural assumptions are compared: SIREN and Sphere2Vec-M.

The proposed methodology is illustrated in Figure 2. The following describes the MTLnet model, the baseline with DGI, the rationale and operation of each proposed encoder, and the strategy for integrating the embeddings.

MTLnet [1] is a multitask network built on hard parameter sharing [5], which simultaneously processes the POI Category Classification and Next-POI Prediction tasks. The architecture uses task-specific encoders implemented as MLPs with LayerNorm and Dropout, which project the input embeddings into a shared latent space of dimension $d_{\text{shared}} = 256$. Next, FiLM

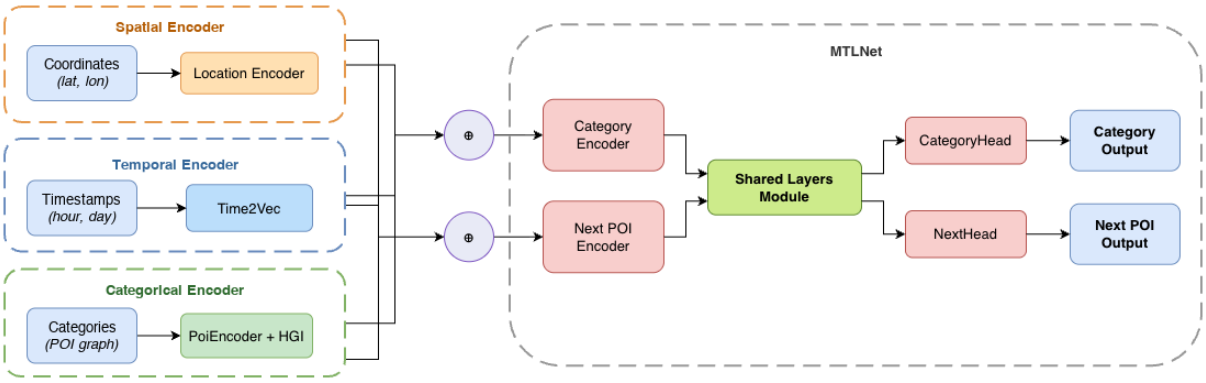


Figure 2 – Architecture based on MTLnet [1]. The spatial, temporal, and categorical encoders are trained in a decoupled manner and generate their respective embeddings, which are integrated as input to the model. The integrated input is processed by the model's shared layers (*Shared Layers Module*) and by the task-specific layers, producing the *Category Output* and *Next POI Output*.

modulation (*Feature-wise Linear Modulation*) [39] is applied, conditioned by a task-identifier embedding $\mathbf{e}_t$, which generates scale $\gamma_t$ and shift $\beta_t$ parameters to modulate the encoded representations:

$$\mathrm{FiLM}(\mathbf{h}_{\mathrm{enc}}^{(t)} \mid \mathbf{e}_t) = \gamma_t \odot \mathbf{h}_{\mathrm{enc}}^{(t)} + \beta_t \tag{4.1}$$

The modulated representations are processed by four shared residual blocks with LayerNorm, LeakyReLU, and Dropout, allowing the model to learn a common representation for both tasks [83].

At the output, each task has a specialized head. The category classification module uses a set of three parallel MLPs with varying depths (2, 3, and 4 layers), whose outputs are concatenated and projected to the 7 POI classes. The next-POI head employs a *Transformer encoder* with 8 attention heads and 4 layers, a causal mask for autoregressive processing, and attention-weighted pooling using attention weights learned over the timesteps of the input sequence.

Multitask training uses the Nash-MTL regularizer [47], which formulates gradient balancing as a cooperative Nash bargaining game among $K$ tasks. Given the gradients of each task, Nash-MTL seeks the update direction that maximizes the product of the utilities of all tasks. Away from a Pareto-stationary point, meaning a point at which some convex combination of the task gradients is zero, and under the method's assumption that the gradients are linearly independent there, that direction is a descent direction for every task, avoiding the dominance of one task over the other.

### 4.3.1 Baseline: MTLnet with DGI

The baseline model corresponds to the original MTLnet proposed in [1], whose internal architecture is kept unchanged in this chapter. In this model, each POI is represented by a single monolithic embedding $\mathbf{E}_{DGI} \in \mathbb{R}^{64}$, obtained via *Deep Graph Infomax* (DGI) [31] over a spatial graph built by Delaunay triangulation and categorical node attributes, as described in detail in [1]. Thus, DGI jointly encodes spatial and categorical relationships in a single vector, without incorporating explicit temporal components. In contrast, the proposed approach replaces this monolithic representation with the concatenation of three specialized encoders, totaling 192 input dimensions.

### 4.3.2 Data Preparation

For the *POI Category Classification* task, each POI is represented by the embedding resulting from the concatenation of the three components, $\mathbf{E}_{cat} = [\mathbf{E}_{HGI} \| \mathbf{E}_{loc} \| \mathbf{E}_{time}] \in \mathbb{R}^{192}$, forming pairs $(\mathbf{E}_{cat}, c)$ where $c$ is the real category of the POI. In the case of the baseline, $\mathbf{E}_{DGI} \in \mathbb{R}^{64}$ is used directly.

For the *Next-POI Prediction* task, the check-ins are ordered chronologically per user, and users with fewer than five visits are discarded. Non-overlapping windows of size $L_h = 9$ are extracted: the check-ins $(p_1, \ldots, p_9)$ form the context, and $p_{\text{target}}$ is the next POI whose category must be predicted. Each check-in in the window is represented by the corresponding embedding, resulting in an input of $9 \times 192 = 1728$ dimensions for ST-MTLNet and $9 \times 64 = 576$ dimensions for the baseline. Shorter sequences are filled with padding of zero vectors.

### 4.3.3 Spatial Encoder

The original MTLnet model encodes spatial information implicitly through the topology of the Delaunay graph, without directly using geographic coordinates as an input signal. However, as discussed in Section 4.2, continuous spatial encoders have shown effectiveness in various geospatial tasks [86], although their application in multitask POI models remains little explored. This chapter selects two encoders that represent distinct spatial encoding paradigms: SIREN [37], which models continuous functions through sinusoidal activations with controllable frequencies, and Sphere2Vec-M [36], which operates directly on spherical coordinates preserving geodesic distance properties. This diversity allows evaluating how different geometric and architectural assumptions impact performance on POI tasks.

All encoders produce $\mathbf{E}_{loc} \in \mathbb{R}^{64}$ and are trained with the same binary contrastive loss based on the geographic distance between pairs of coordinates:

$$\mathcal{L}_{\text{loc}}(i,j) = -\left[ y_{ij} \log \sigma\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right) + (1 - y_{ij}) \log\left(1 - \sigma\left(\frac{\text{sim}(z_i, z_j)}{\tau}\right)\right)\right] \tag{4.2}$$

in which $\sigma$ denotes the sigmoid function, $\text{sim}(\cdot, \cdot)$ is the cosine similarity, and $y_{ij} \in \{0, 1\}$ indicates whether the pair is positive ($\Delta d_{ij} \leq 10$ km) or negative ($\Delta d_{ij} \geq 70$ km), with temperature $\tau = 0.15$. Using the same loss function for the two spatial encoders allows a controlled comparison, isolating the effect of the encoding architecture.

#### 4.3.3.1 SIREN

The SIREN model (*Sinusoidal Representation Networks*) [37] models a continuous function $f_\theta : \mathbb{R}^2 \to \mathbb{R}^{64}$ that directly maps normalized geographic coordinates into a vector embedding. Each layer of the network uses sinusoidal activations with controllable frequencies, allowing high-frequency signals in spatial data to be represented. The output is the embedding $\mathbf{E}_{loc} \in \mathbb{R}^{64}$.

#### 4.3.3.2 Sphere2Vec-M

Sphere2Vec-M [36] is a multi-scale encoder formulated to operate directly on spherical coordinates ($\lambda$, $\phi$), generating representations that preserve distance properties on the surface of the sphere. The *sphereM* variant is used, in which $S = 16$ multi-resolution scales are employed with geometrically spaced factors between 10 km and 10,000 km. Interaction terms between $\lambda$ and $\phi$ are constructed by keeping one of the components at the highest scale level, allowing the angular resolution of latitude and longitude to be controlled independently. The resulting vector is projected by an MLP to produce $\mathbf{E}_{loc} \in \mathbb{R}^{64}$.

### 4.3.4 Temporal Encoder

Human mobility patterns exhibit cyclical regularities, such as meal times and weekly movements [10], and models of the next visit read them through explicit temporal representations [89]. However, the original MTLnet model does not incorporate explicit temporal representations to capture this information.

To fill this gap, Time2Vec [33] is used, a generic and decoupled temporal representation capable of capturing periodic and non-periodic patterns without the need for manual feature engineering. The architecture combines linear and periodic terms to map the temporal values of each check-in (hour of day and day of week, in a coupled manner) into an embedding vector. For an embedding of dimension $D$, the function is defined as:

$$\mathbf{f}(\tau)[i] = \begin{cases} \omega_0 \tau + \phi_0 & \text{if } i = 0 \\ \sin(\omega_i \tau + \phi_i) & \text{if } 1 \leq i \leq D - 1 \end{cases} \tag{4.3}$$

in which $\omega_i$ and $\phi_i$ are trainable parameters (weights and bias) that capture the periodicity of the temporal visitation patterns. The linear term ($i = 0$) captures global temporal trends, while

the sinusoidal terms model cyclical patterns at different frequencies.

The model is trained using the discrete temporal values (hour of day and day of week) in a coupled manner for each check-in, with the same binary contrastive formulation employed in the spatial encoder, denoted $\mathcal{L}_{tempo}$. The final output is the embedding $\mathbf{E}_{time} \in \mathbb{R}^{64}$, which represents the timestamp of each check-in.

### 4.3.5 Categorical Encoder

The DGI used in the original model encodes categorical information through a *one-hot* matrix of 7 classes processed by the graph structure, without explicitly capturing hierarchical or regional relationships among POI categories. To enrich this dimension, the generation of categorical embeddings is carried out in two complementary phases. The POI Encoder learns vector representations that capture co-occurrences among categories from random walks on the spatial graph, and the HGI model [32] incorporates hierarchical information at multiple levels (POI, region, and city), producing representations that reflect both the local and the regional context.

#### 4.3.5.1 POI Encoder

The category embedding $\mathbf{E}_{\text{POI\_category}} \in \mathbb{R}^{64}$ is generated by the POI Encoder, which is trained independently. The POIs are organized into a spatial graph built by Delaunay triangulation over the geographic coordinates, with edges weighted by logarithmic decay of the Haversine distance and penalization for connections between distinct counties (GEOIDs).

Over this graph, random walks are executed following the Node2Vec methodology [25]. Each walk is converted into a sequence of secondary categories, the fine classes, resulting in local and global co-occurrences among categories. The model learns the embeddings using the *skip-gram* strategy with *negative sampling* [23, 91]:

$$\mathcal{L}_{\text{SGNS}}(i, j) = -\log \sigma\left(\mathbf{e}_j^\top \mathbf{e}_i\right) - \sum_{k=1}^{K} \log \sigma\left(-\mathbf{e}_{n_k}^\top \mathbf{e}_i\right),$$

in which $\mathbf{e}_i$ is the embedding of the target class, $\mathbf{e}_j$ the positive context embedding, and $\mathbf{e}_{n_k}$ represent the negative samples. The implementation incorporates a hierarchical regularization term [4] between category and fine class: $\mathcal{L}_{\text{hier}} = \sum_{(c,s)\in\mathcal{H}} \|\mathbf{e}_s - \mathbf{e}_c\|_2^2$, in which $\mathcal{H}$ contains the (category, fine class) pairs. The total loss is $\mathcal{L} = \mathcal{L}_{\text{SGNS}} + \lambda_{\text{hier}} \mathcal{L}_{\text{hier}}$. In the end, the resulting embedding is generated per category and remapped to each POI.

#### 4.3.5.2 Hierarchical Graph Infomax (HGI)

After the generation of the embeddings $\mathbf{E}_{\text{POI\_category}}$, they are used as input to the Hierarchical Graph Infomax (HGI) model [32], which enriches the representation by incorporating regional geographic information. The model optimizes mutual information at two hierarchical levels (POI-region and region-city):

$$\mathcal{L} = \alpha\, \mathcal{L}_{\text{POI–region}} + (1 - \alpha)\, \mathcal{L}_{\text{region–city}},$$

in which $\alpha = 0.5$ controls the balance between the hierarchical levels. Both losses use structural negative sampling to reinforce the separability between regions and between POIs. The output is the enriched embeddings $\mathbf{E}_{\text{HGI}} \in \mathbb{R}^{64}$, which capture both local and regional categorical relationships.

### 4.3.6 Embedding Integration

The three representation components are trained independently and concatenated to form the input of both MTLnet tasks:

$$\mathbf{E}_{input} = [\mathbf{E}_{HGI} \| \mathbf{E}_{loc} \| \mathbf{E}_{time}] \in \mathbb{R}^{192} \tag{4.4}$$

The choice of concatenation as the integration strategy preserves the individual information of each component without imposing premature interactions between the representations, delegating to MTLnet the task of learning how to combine these dimensions through its shared layers and FiLM modulation. The input dimensionality of ST-MTLNet ($\mathbb{R}^{192}$) is higher than that of the baseline ($\mathbb{R}^{64}$), a natural consequence of the decomposition into three specialized components.

However, the MTLnet architecture projects any input to the same shared latent space of dimension $d_{\text{shared}} = 256$ through the task-specific encoders, so that the capacity of the shared layers and task heads remains unchanged across the evaluated models. Even so, the difference in input dimensionality may influence part of the observed gains. For this reason, an additional experimental control equalizing the dimensionality of the representations would allow validating more precisely whether the gains occur mainly from the semantic specialization of the encoders, and not only from the increase in input dimensionality.

Two variants of the proposed model are evaluated, named ST-MTLNet (*Spatial-Temporal MTLNet*), each using a different spatial encoder. The baseline MTLnet [1] uses only the monolithic embedding $\mathbf{E}_{DGI} \in \mathbb{R}^{64}$ as input for both tasks. The proposed variants replace this representation with the concatenation of the decoupled embeddings, keeping the temporal (Time2Vec) and categorical (HGI) encoders fixed and varying the spatial encoder between

SIREN and Sphere2Vec-M.

All models share the same hyperparameters of the MTLnet architecture, differing only in the input embeddings. This configuration allows isolating the effect of the representation strategy on performance, evaluating both the gain of the modular approach relative to the monolithic baseline and the impact of the choice of the spatial encoder.

## 4.4 Results

### 4.4.1 Experimental Setup

The performance of the models is evaluated using the Average F1-Score per category, reported as mean and standard deviation over 5 folds with a stratified split, using 80% of the data for training and 20% for validation, preserving the proportion between categories. The split is stratified by sample, not by user, so the check-ins of one user may appear in both training and validation. The released code of record pins a single random seed, so the five folds constitute one repetition of the experiment rather than several, and the reported standard deviations are the spread across folds at that seed rather than across independent repetitions. Within a fold, training runs for the full number of epochs configured, without early stopping, and each task is read at the epoch of its own highest validation macro-F1, measured on the same fold the score is reported on.

The experiments were conducted with the Gowalla dataset [10, 75] in the states of Florida, California, and Texas, whose statistics are presented in Table 5. Texas concentrates the largest volume of check-ins of the three states, while Florida provides the smallest. The choice of these states is due to the high volume of check-ins and the presence of distinct urban patterns and spatial distributions, which allows evaluating the behavior of the models in scenarios with different levels of density and geographic dispersion.

Table 5 – Total number of check-ins, POIs, and users for Florida, California, and Texas.

| States | Check-ins | Points of Interest | Users |
|---|---|---|---|
| Florida | 990,518 | 65,009 | 20,301 |
| California | 2,535,573 | 148,314 | 36,106 |
| Texas | 3,355,419 | 135,570 | 37,522 |

### 4.4.2 POI Category Classification

The results of the POI Category Classification task are presented in Table 6.

In the three evaluated states, the two variants of ST-MTLNet outperform the original MTLnet in all 21 category-state combinations, showing that replacing the monolithic embedding

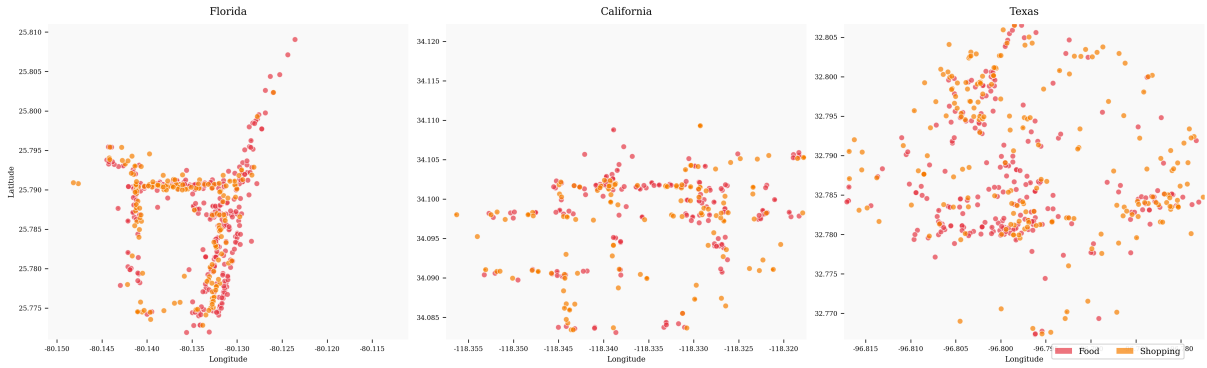


Figure 3 – Spatial distribution of POIs of the Food (red) and Shopping (orange) categories in the densest sub-region of Florida, California, and Texas. The sub-regions were selected by density in a grid, with about 100 POIs per region. Each panel plots the POI coordinates (longitude on the horizontal axis, latitude on the vertical axis) for one state; the dense, overlapping clusters of the two categories illustrate the co-location pattern discussed in the text.

of DGI with decoupled representations consistently enriches the characterization of the POIs. The average gains per state are 20.2 to 22.0 percentage points, considering the better of the two spatial encoders in each combination. The gains are particularly substantial in categories such as *Nightlife* and *Travel*. In *Nightlife*, for example, ST-MTLNet with Sphere2Vec-M reaches 62.44 ± 2.00 in Florida, 60.78 ± 1.63 in California, and 64.57 ± 1.03 in Texas, always well above the baseline. In *Travel*, the best results reach 64.89 ± 1.20 in Florida with SIREN, 63.59 ± 1.45 in California with SIREN, and 64.73 ± 0.72 in Texas with Sphere2Vec-M.

The improvements observed in *Food* and *Shopping* reinforce that the geographic location carries discriminative signals relevant to the semantic category of the POI. Figure 3 shows that these categories tend to form dense clusters in the analyzed sub-regions, favoring the capture of spatial patterns by the continuous encoders. Although the two proposed variants present close performance, a complementary behavior is observed: SIREN obtains more outstanding results in Florida and California, while Sphere2Vec-M stands out more frequently in Texas. Together, these results show that the modular approach consistently outperforms the monolithic paradigm of DGI in this task.

### 4.4.3 Next-POI Prediction

The results of the Next-POI Prediction task are presented in Table 7: the proposed variants keep the higher Average F1-Score per category in most category-state combinations, while the baseline retains six of them.

Unlike classification, this task presents a more heterogeneous scenario, but still favorable to the proposed variants: counting the better of the two spatial encoders per combination, the spatio-temporal models outperform the original MTLnet in 15 of the 21 evaluated combinations, with one additional technical tie in *Outdoors* in Florida, where the baseline mean exceeds the best

Table 6 – Average F1-Score (%) per model and state for the POI Category Classification task.

| | | **MTLnet** | **ST-MTLNet**$_{SIREN}$ | **ST-MTLNet**$_{Sphere2Vec-M}$ |
|---|---|---|---|---|
| **Florida** | Community | 51.86 ± 0.73 | **70.00 ± 0.81** | 69.84 ± 0.98 |
| | Entertainment | 41.24 ± 1.83 | **64.45 ± 1.82** | 63.92 ± 1.61 |
| | Food | 55.47 ± 1.55 | 72.92 ± 0.59 | **73.22 ± 0.66** |
| | Nightlife | 32.59 ± 1.96 | **62.60 ± 1.96** | 62.44 ± 2.00 |
| | Outdoors | 47.71 ± 1.60 | 65.64 ± 1.73 | **66.35 ± 1.51** |
| | Shopping | 62.96 ± 0.62 | **77.48 ± 0.36** | 77.47 ± 0.72 |
| | Travel | 45.49 ± 1.20 | **64.89 ± 1.20** | 64.77 ± 1.54 |
| **California** | Community | 53.22 ± 0.37 | **70.21 ± 0.17** | 70.08 ± 0.44 |
| | Entertainment | 40.89 ± 1.32 | **63.60 ± 0.83** | 63.21 ± 0.76 |
| | Food | 60.68 ± 0.69 | **75.78 ± 0.39** | 75.74 ± 0.19 |
| | Nightlife | 26.81 ± 1.71 | 60.59 ± 1.35 | **60.78 ± 1.63** |
| | Outdoors | 51.59 ± 1.15 | **67.94 ± 1.52** | 67.37 ± 1.95 |
| | Shopping | 60.43 ± 1.01 | 76.84 ± 0.29 | **77.00 ± 0.33** |
| | Travel | 38.88 ± 1.32 | **63.59 ± 1.45** | 63.25 ± 0.99 |
| **Texas** | Community | 57.37 ± 0.69 | 73.45 ± 0.34 | **76.20 ± 0.46** |
| | Entertainment | 46.69 ± 0.46 | 64.09 ± 1.24 | **67.73 ± 1.43** |
| | Food | 54.31 ± 0.42 | 69.80 ± 0.80 | **73.10 ± 0.17** |
| | Nightlife | 34.44 ± 1.21 | 60.62 ± 1.32 | **64.57 ± 1.03** |
| | Outdoors | 45.99 ± 1.70 | 64.21 ± 0.42 | **68.72 ± 0.70** |
| | Shopping | 62.70 ± 0.29 | 76.40 ± 0.44 | **79.67 ± 0.14** |
| | Travel | 39.37 ± 1.29 | 57.53 ± 1.08 | **64.73 ± 0.72** |

variant by 0.02 percentage points, a gap within one standard deviation. The largest gains occur in *Food*, a category in which both variants outperform the baseline in the three states. The best result is obtained by Sphere2Vec-M in Texas, with 48.67 ± 0.66, while in California the same model reaches 51.28 ± 0.57, outperforming DGI by a wide margin. Consistent improvements also appear in *Shopping* and *Community*, suggesting that the combination between continuous spatial information and temporal context favors the modeling of recurrent transitions between categories.

The co-location pattern shown in Figure 3 helps to interpret this behavior, since the presence of dense clusters of categories such as *Food* and *Shopping* favors frequent local transitions along the user's trajectory. Even so, the baseline maintains an advantage in some cases, especially in *Travel* in Florida and California, in addition to specific categories in California, such as *Entertainment*, *Nightlife*, and *Outdoors*.

This behavior may be related to the nature of the *Travel* category itself, which tends to involve sparser movements between distant regions. In these cases, the graph topology used by DGI may be more efficient for preserving relationships between geographically distant POIs. On the other hand, spatial encoders such as SIREN and Sphere2Vec-M are coordinate-based, being more suitable for capturing local patterns, showing that continuous coordinate encodings

Table 7 – Average F1-Score (%) per model and state for the Next-POI Prediction task.

| | | **MTLnet** | **ST-MTLNet$_{SIREN}$** | **ST-MTLNet$_{Sphere2Vec-M}$** |
|---|---|---|---|---|
| **Florida** | Community | 34.33 ± 1.33 | **38.31 ± 0.90** | 37.98 ± 1.22 |
| | Entertainment | 25.34 ± 1.07 | **31.38 ± 1.72** | 31.24 ± 2.41 |
| | Food | 27.12 ± 2.79 | **41.91 ± 1.93** | 41.65 ± 3.15 |
| | Nightlife | 21.33 ± 1.61 | 23.32 ± 2.74 | **23.98 ± 1.94** |
| | Outdoors | **21.61 ± 0.99** | 21.29 ± 1.59 | 21.59 ± 1.91 |
| | Shopping | 39.62 ± 3.40 | 44.00 ± 2.04 | **44.66 ± 2.38** |
| | Travel | **64.47 ± 1.02** | 45.00 ± 1.10 | 44.93 ± 1.11 |
| **California** | Community | 32.14 ± 0.89 | **37.86 ± 0.20** | 37.80 ± 1.11 |
| | Entertainment | **19.38 ± 0.94** | 17.28 ± 1.73 | 17.01 ± 1.39 |
| | Food | 29.26 ± 1.67 | 50.42 ± 1.15 | **51.28 ± 0.57** |
| | Nightlife | **18.24 ± 0.53** | 16.34 ± 1.95 | 15.10 ± 2.70 |
| | Outdoors | **25.01 ± 0.81** | 24.84 ± 1.70 | 24.07 ± 2.46 |
| | Shopping | 38.41 ± 2.64 | **39.04 ± 1.72** | 37.82 ± 1.31 |
| | Travel | **46.05 ± 0.84** | 36.94 ± 1.70 | 37.82 ± 1.04 |
| **Texas** | Community | 34.22 ± 0.43 | 35.99 ± 0.93 | **36.97 ± 0.81** |
| | Entertainment | 25.56 ± 1.56 | **28.30 ± 1.37** | 27.59 ± 1.64 |
| | Food | 29.56 ± 4.10 | 47.48 ± 2.42 | **48.67 ± 0.66** |
| | Nightlife | 25.46 ± 1.13 | 26.06 ± 1.36 | **26.14 ± 2.66** |
| | Outdoors | 23.02 ± 1.03 | **24.62 ± 0.46** | 23.04 ± 1.44 |
| | Shopping | 38.79 ± 4.12 | **45.40 ± 1.30** | 43.18 ± 2.19 |
| | Travel | 29.71 ± 1.32 | **34.26 ± 0.90** | 33.05 ± 1.51 |

and graph-based representations capture complementary aspects of mobility. Even with this limitation, the general performance indicates that the proposed representations are more effective in most sequential prediction scenarios.

## 4.5 Conclusion and Future Work

This chapter investigated whether replacing the monolithic embedding of DGI with decoupled and specialized representations could produce a base more suitable for the tasks of POI Category Classification and Next-POI Prediction. The results obtained in the three evaluated states indicate that it can. By incorporating a continuous spatial encoder, a temporal encoder (Time2Vec), and a hierarchical categorical encoder (HGI), ST-MTLNet consistently outperforms the baseline based only on DGI.

In the POI Category Classification task, the proposed variants outperform MTLnet in all 21 category-state combinations, with average gains per state of 20.2 to 22.0 percentage points, considering the better of the two spatial encoders in each combination. In the Next-POI Prediction task, the performance is also mostly favorable to ST-MTLNet, which outperforms the baseline in 15 of the 21 evaluated combinations, with one additional technical tie, with

emphasis on categories such as *Food*. Taken together, these results show that decomposing the representation into spatial, temporal, and categorical components allows capturing patterns that DGI, by itself, does not model sufficiently.

The comparison between SIREN and Sphere2Vec-M also shows that there is no single universally superior spatial encoder. SIREN presents better performance more frequently in Florida and California, while Sphere2Vec-M stands out in Texas, suggesting that the suitability of the encoder depends on the geographic distribution of the POIs in each territory. Thus, the main gain of this chapter lies not only in the choice of a specific spatial architecture, but in the very adoption of decoupled representations as an alternative to the monolithic paradigm of DGI.

A relevant limitation remains in the *Travel* category of the Next-POI Prediction task, in which the original MTLnet still obtains the best results in part of the scenarios. This suggests that long-distance movements and topological relationships between regions are still better captured by the graph structure used by DGI. In addition, since the three proposed components are used together, this chapter does not isolate the individual contribution of each encoder, which restricts a more detailed analysis of the relative weight of each source of information.

Another limitation is related to the exclusive use of Gowalla, a dataset commonly used in the literature but composed of mobility records collected between February 2009 and October 2010. Consequently, the results should be interpreted with caution regarding their generalization to current urban mobility patterns.

As future work, we highlight the investigation of more flexible multitask architectures, the exploration of more sophisticated fusion strategies between the encoders, and the combination of graph-based representations and continuous coordinate encodings. These directions may broaden the observed gains and help reduce the limitations still present in categories such as *Travel*.

# 5 A Check-in-Level Multitask Study of Next Category and Region

*This chapter reproduces the article "Predicting the Next Category and Region of a Visit: A Check-in-Level Multi-Task Study on Mobility Data," submitted to MobiWac 2026, under review at the time of writing (EDAS #1571313639). It closes the investigation that Chapters 3 and 4 open: with a check-in-level representation and a redesigned sharing topology, a single joint model outperforms the dedicated single-task category model at all six datasets studied (five U.S. states and Istanbul) and the dedicated region model at four of the six, and it matches the dedicated region model (statistically, within two points) at the other two. The article was reformatted from its two-column original into the dissertation layout: sections, figures, and tables were renumbered, and the figures were regenerated at single-column width. No result, claim, or conclusion was altered.*

## 5.1 Introduction

Location-based social networks, such as Gowalla [10], let people share the places that they visit as check-ins, which together record how people move through a city. Individual mobility is highly predictable in principle [2], so a service that anticipates the next move can prepare ahead of time, caching content where a user is heading [92] or provisioning capacity at the destination before demand arrives. Handover predictions already let cellular services adapt in advance at the network level [93]; at the city scale, check-in mobility analysis is likewise a design input for mobility-aware services [94].

Predicting the next place exactly, a single point of interest (POI), is hard and often more than a service needs. Two coarser questions are usually enough: the next category, the type of place such as food or shopping, and the next region, the part of the city that a user is heading to (a neighborhood-scale unit such as the U.S. census tract). The first captures intent, the second captures geography; we study whether one model should learn both at once.

Sharing one representation across tasks has a cost: in multitask learning (MTL), one model does several jobs at once by sharing most of its parts, so the shared parameters can converge to a compromise optimal for neither task, helping one while hurting the other [5]. Our earlier work reported no consistent multitask advantage for the paired category tasks and attributed it, in part, to this effect [1]; the useful question is where sharing helps, what it costs,

and how to share so the gains hold and the cost stays small.

We propose two enhancements. First, we build a check-in-level representation: instead of one fixed vector per place, each check-in gets its own vector that holds its context (the time, nearby places, and recent visits). This adds a fourth level, the check-in, beneath the place, region, and city levels of hierarchical graph infomax [32, 31] (Figure 4). Second, we train one model that predicts the next category and the next region in a single forward pass, with a shared trunk (a cross-attention stack where the two tasks exchange semantic context) and a private spatial path for the region task (Figure 5). We evaluate on two settings chosen to differ: five U.S. states of different sizes (Gowalla) and one non-U.S. city (Istanbul).[1]

Our contributions are as follows.

- A check-in-level representation that extends hierarchical graph infomax from the place to the individual visit, improving next-category prediction over a standard place embedding by about +28 to +40 points of macro-F1 (a balanced score that counts every category equally) across all six datasets (Table 9). A controlled test attributes most of the gain to the per-visit context, not to extra training signal. The gap is large because a single fixed vector per place cannot separate places that serve several purposes, which per-visit context recovers.

- A single model for joint next-category and next-region prediction, sharing semantic context across tasks while keeping a private spatial path for the region task; to our knowledge, the first work to treat fine-grained region as an end target of equal standing (Section 5.2.2). In one forward pass, it outperforms a dedicated single-task category model on every dataset (by about +5 to +9 points of macro-F1), even though each dedicated model is tuned per dataset. On region, it outperforms at four of the six datasets and stays statistically non-inferior within a two-point margin (TOST, a test of equivalence) at the remaining two (Table 10).

- An empirical account, across five U.S. states and one non-U.S. city, of how the joint gain scales with the number of regions. The category gain holds on all six datasets. Across the U.S. states, the region result moves monotonically with the region count: the joint model matches the dedicated model (TOST, ±2 percentage points, or pp) at the small region counts and outperforms it at the large region counts, with the state with the most regions (California) showing the largest region gain (Figure 7). At Istanbul, the dataset with the fewest regions, the joint model is also ahead on region (+0.19 Acc@10). All joint and dedicated results average four random initializations (Section 5.6.2).

[1] Code (model, representation, baselines, statistical tests): <https://github.com/VitorHugoOli/PoiMtlNet/tree/mobiwac>. Both data sources are public: the category-annotated Gowalla dump (<https://figshare.com/articles/dataset/gowalla_data/22126586>) and Massive-STEPS [66].

The remainder of this chapter presents related work (Section 5.2), the problem (Section 5.3), the method and setup (Sections 5.4 and 5.5), results (Section 5.6), and discussion and conclusions (Sections 5.7 and 5.8).

## 5.2 Background and Related Work

### 5.2.1 From place embeddings to per-visit embeddings

A place embedding transforms a POI into a vector so that similar or nearby places sit close together in the vector space. Deep Graph Infomax (DGI) [31], a general self-supervised method for graphs, learns such vectors on a place network linked by similarity, time, and distance, training them to tell the real network from a copy with shuffled node features. Hierarchical Graph Infomax (HGI) [32] adds geographic structure, organizing the network as place, then region, then city. Both produce one vector per place, so two visits to the same coffee shop look identical to the model. Contextual check-in representations instead give each visit its own vector: CTLE [18], the closest example, learns a vector per visit by masking and reconstructing parts of a user's check-in sequence. Our representation, Check2HGI, also works at the check-in level but through a different construction. CTLE is a sequence model, a Transformer that reads the check-in sequence itself; Check2HGI remains a network model, keeping the hierarchical place-to-region-to-city network and the same infomax objective, now extended one level deeper, from individual places down to individual check-ins (Figure 4). The order of a user's visits still enters the network, as links between consecutive check-ins (Section 5.4.1).

The novelty is this specific combination: per-visit context inside a hierarchical graph-infomax representation. We compare against CTLE and a feature-concatenation control to separate the combination from contextualization alone and from feature injection.

### 5.2.2 Predicting the next category and the next region

A large body of work predicts the next place that a user will visit, the exact POI, with recurrent and attention models such as DeepMove [12], STAN [15], and GETNext [17]. In contrast, we target two properties of the next visit, its category and its region, rather than the exact place. We choose them because they are what a mobility-aware service can act on, not to make the task easier; the region alone spans hundreds to several thousand classes. Predicting over a partition of the map is also the standard formulation in the human-mobility literature, with a grid cell as the target [3]; our next-region task substitutes official neighborhood-scale units for grid cells. Our earlier work [1] paired static category classification with next-category prediction and found no consistent multitask gain; this chapter introduces the next-region task and adds the check-in-level representation, on which sharing helps instead of hurting (Section 5.6.2).

The field increasingly models several granularities at once; in those systems, category and region are auxiliary signals that help a primary next-place task (MCMG [95], HMT-GRN [19]). We study the pair as the object itself. To our knowledge, fine-grained region as an end target of equal standing, rather than an auxiliary coarse grid cell, is underexplored. The nearest exceptions do not study our exact pairing: DRRGNN [21] forecasts a person's next activity region jointly with a mobility-intention label, but over regions discovered per person rather than a fixed citywide partition; a generative recommender [96] predicts category and region together, but only as auxiliary steps toward its next-place ranking.

### 5.2.3 Multitask learning for mobility

MCARNN [59] jointly predicts activity and location, and a recent hierarchy-aware model continues the pattern [64]; in both, the location target is the exact place. The closest alternative to our setting is the cascade: predict the category first, then the place given the category [97]. CSLSL [60] is its current form, predicting in a chain (when, then what, then where), with location as the primary output and category as an intermediate step; CatDM [20] likewise uses the predicted category only to filter candidate places. We instead predict category and region in parallel, as end targets of equal standing in one model with a single forward pass, and we drop the next-place target entirely, so neither task is an intermediate step toward a third. Since the cascade is the pattern that the field uses, we test the choice directly (Section 5.6.2). On optimization, we are conservative by design: joint training with a fixed loss weighting is standard practice [5], not itself our contribution. Reports find that gradient-balancing methods, which re-weight the tasks' updates during training (PCGrad [44], Nash-MTL [47]), rarely improve on a well-tuned fixed weighting with two tasks [55]. We confirm this at scale: of nineteen loss and gradient balancers screened at their default configurations at a single seed on two datasets, Alabama and Florida, including the two methods named above, none improved on a tuned fixed task weighting across both tasks and both datasets. Two exceed equal weighting on next-category at Alabama, Nash-MTL by 0.68 points and scale normalization by 0.19, and neither holds that position elsewhere: Nash-MTL loses it on the second dataset, and scale normalization gains on next-category there while collapsing on next-region. The reason is visible in the gradients. Measured during development on the same joint architecture (on an earlier preparation of the data), the cosine similarity between the next-category and next-region updates on the shared trunk averages +0.001 across training (four seeds each on four Gowalla states: Alabama, Arizona and Florida, which are three of the five U.S. datasets reported here, and Georgia, which is not among the datasets this study reports; the largest per-dataset mean in absolute value is +0.0032). A balancer therefore has no conflict to resolve, consistent with the mechanism test in Section 5.6.2; this is a finding for this pair of tasks, not a general rule. The same quantity, measured on the final model across seven datasets, is positive at every one of them, so the absence of conflict is not an artifact of this smaller set of runs.

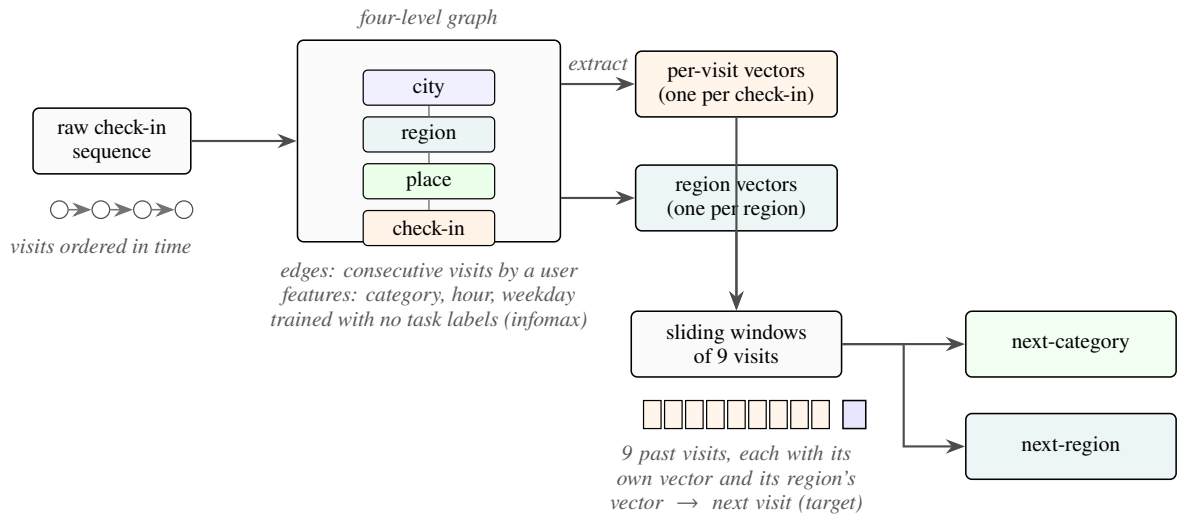


Figure 4 – From a check-in sequence to two predictions: each visit is embedded in a four-level graph (check-in, place, region, city) trained without task labels. The graph yields two sets of vectors, one per visit and one per region; sliding nine-visit windows of each feed a single model that outputs the next category and the next region.

## 5.3 Problem and Tasks

Given a user's time-ordered check-in history, we predict two properties of the next visit. The first is its *category*, one of seven fixed labels: Community, Entertainment, Food, Nightlife, Outdoors, Shopping, and Travel. Istanbul's source collection maps its places onto the same seven labels. The second is its *region*, the place's census tract (for Istanbul, the *mahalle*, a municipal neighborhood). We do not predict the exact next place; both properties are easier to learn and, for most uses, enough. Region, unlike category, is a large label set: we frame next-region as classification over candidate regions, from 520 classes (Istanbul) to 8,501 (California; Table 8).

The motivation is practical: the category says what type of place comes next, hence what a user will want; the region says where, hence where to prepare. The mobility literature supports such preparation: city services have long been built on location-based social network traces [9]. A recent analysis of tourist check-in mobility finds that visits concentrate on a few hub places, argues that crowds and demand therefore concentrate, and ties this structure to infrastructure planning and adaptive strategies [94]. That analysis reads past visits; we predict two properties of the next one, and aggregated over users, those predictions point to where demand is heading before it arrives. A census tract is a neighborhood, not a radio cell. We therefore scope our claims to neighborhood-level preparation: demand and load anticipation, caching content ahead of time, and capacity planning. Cell association and handover are radio-level decisions and remain out of scope. We build and evaluate no such service here; Section 5.7 quantifies what these predictions would give such a service.

## 5.4 Method

### 5.4.1 The check-in-level representation

We want each visit, not each place, to have its own vector; we call the resulting representation Check2HGI (Section 5.2.1). Figure 4 shows the full data flow, from check-ins to the two outputs.

We build a graph with four levels: the check-in, its place, the place's region (a census tract), and the city. Edges connect each level to the one above it and link a user's consecutive check-ins with a weight that decays as the time gap between the visits grows; same-place visits connect through their shared place node one level up, and nearby places are linked at the place level. Each visit's category, time of day, and day of week enter as input features of its node, not as edges. This keeps a hierarchical graph's place-to-region-to-city geography and adds the check-in as a bottom level. We train the graph mainly with an infomax objective, under which each vector learns to match its real neighborhood and reject a shuffled one [31, 32]. Two small label-free auxiliary terms are added (weights 0.3 and 0.1): a masked reconstruction of each place's aggregated category features, and an anchor to a place embedding pre-trained, label-free, on the same data. The training uses no task label: it never sees the next category or the next region. From the trained graph, we extract one 64-dimensional vector per check-in (its region nodes have trained vectors as well), so a model sees a sequence of per-visit vectors rather than repeated per-place ones.

### 5.4.2 The single multitask model

We then train one model that reads a window of recent per-visit vectors and predicts the next visit's category and region at once. Figure 5 shows the design. Each task has its own input. The category task reads the window of per-visit vectors (the semantic stream); the region task reads the same window of visits, each visit now represented by the trained vector of its region node from the same graph (the spatial stream). Both pass through private per-task encoders (a small input network per task, with no shared weights) into the shared trunk, a cross-attention stack of two blocks, so each prediction still uses both inputs. In each block, attention lets each stream read the other's features, while each keeps its own feed-forward weights. The tasks therefore share by exchanging information between per-task streams, not by owning hidden layers in common. The trunk then feeds a category output and a region output; the region output also keeps a private spatial path: a small branch inside the one model (not a second model) that reads the spatial input window, bypassing the shared trunk. The category task does not touch this branch. Together, the region task's encoder, its output, and this private path form the region pathway.

The training loss is a fixed-weight sum of the two outputs,

$$L = 0.75\, L_{\text{cat}} + 0.25\, L_{\text{reg}}, \tag{5.1}$$

where $L_{\text{cat}}$ and $L_{\text{reg}}$ are plain unweighted cross-entropy losses. The weights were tuned once on validation during development and held fixed across all six datasets. Class-weighting, tested on both outputs, lowered both region accuracy and category macro-F1. This is standard fixed-weight joint training [5], kept simple by design so that any improvement over the dedicated single-task models comes from the shared representation, not from an adaptive weighting scheme.

The joint model includes the shared trunk and both task outputs, so it is larger than either dedicated model, and a forward pass costs more compute than running the two small dedicated models: the joint model has about 4.2 million parameters at Alabama against 1.1 million for the two dedicated models combined (5.2 against 2.0 at California). What the single model provides is operational rather than arithmetic: one artifact to train, version, and deploy, and one forward pass whose one set of inputs produces both answers at once.

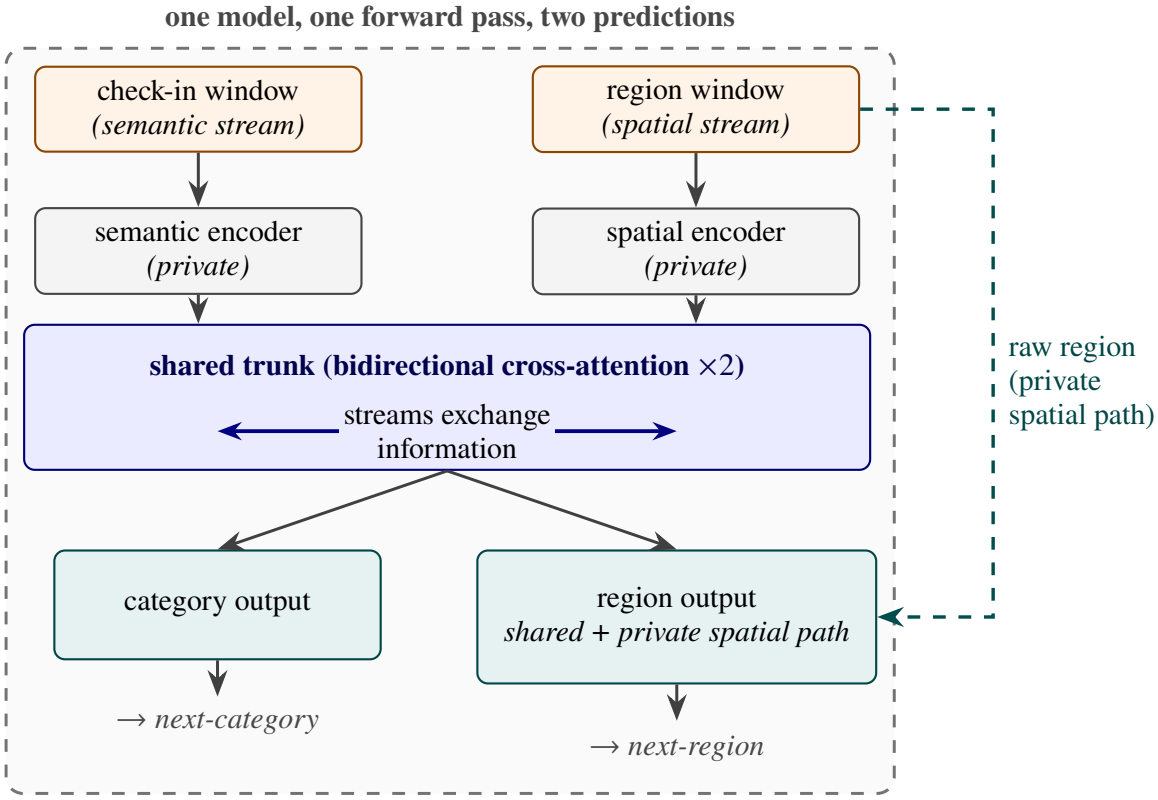


Figure 5 – The single multitask model. A check-in (semantic) window and a region (spatial) window pass through private encoders into the shared trunk, the only place where the two tasks interact. The category output uses the trunk's output alone; the region output combines it with a private spatial path. One model, one forward pass, two predictions.

## 5.5 Experimental Setup

### 5.5.1 Data

Table 8 summarizes our six datasets. Five come from Gowalla, a location-based social network, covering the U.S. states of Alabama, Arizona, Florida, Texas, and California [10]; the sixth is Istanbul, from the Massive-STEPS collection [66], a non-U.S. check on the findings. The states range from about 114,000 check-ins and 1,109 regions in Alabama to about 3.2 million check-ins and 8,501 regions in California. Every dataset uses the seven place categories of Section 5.3; Istanbul's source collection maps its own labels onto them. The region unit is a census tract for the Gowalla states and a mahalle for Istanbul.

Table 8 – Dataset statistics, ordered by region count: five Gowalla states and Istanbul (Massive-STEPS). All share the same seven root categories; the region unit is a census tract (a mahalle for Istanbul). Windows: sliding nine-visit inputs plus the next-visit target; lengths: per-user check-in counts; density: $100 \times \text{check-ins}/(\text{users} \times \text{POIs})$; Majority: share of next-visit labels in the most common category (Food in every dataset).

| Dataset | Source | Check-ins | Users | POIs | Regions | Windows | Max len | Avg len | Density (%) | Majority (%) |
|---|---|---|---|---|---|---|---|---|---|---|
| Istanbul | Massive-STEPS | 462,615 | 23,694 | 29,816 | 520 | 271,666 | 817 | 19.5 | 0.065 | 33.4 |
| AL | Gowalla | 113,846 | 3,858 | 11,848 | 1,109 | 96,326 | 3,835 | 29.5 | 0.249 | 34.2 |
| AZ | Gowalla | 236,450 | 7,869 | 20,666 | 1,547 | 200,895 | 5,589 | 30.1 | 0.145 | 34.0 |
| FL | Gowalla | 1,407,034 | 21,052 | 76,544 | 4,703 | 1,274,418 | 16,679 | 66.8 | 0.087 | 24.7 |
| TX | Gowalla | 4,089,892 | 38,644 | 160,938 | 6,553 | 3,830,414 | 42,300 | 105.8 | 0.066 | 31.0 |
| CA | Gowalla | 3,171,380 | 37,090 | 169,145 | 8,501 | 2,925,466 | 14,855 | 85.5 | 0.051 | 32.7 |

### 5.5.2 Windows, splitting, and the integrity of the representation

*Windows.* For each user with at least ten visits, we build time-ordered overlapping sliding windows of nine visits, one starting at each visit, with the next visit as the target, giving both tasks more examples. Near the end of a user's history, every start position within the last nine visits yields a shorter, padded window whose target is the same final visit; we keep only the full-length window ending there and drop these padded duplicates. The resulting prediction horizon is short but heavy-tailed: the median time from the last visit in a window to its target ranges from 0.4 hours in Florida to 5.5 hours in Istanbul, while 5 to 27 percent of targets lie over 3 days ahead.

*Splitting.* We split by user with stratified five-fold cross-validation, so all of a user's windows fall in the same fold and overlap cannot leak: a test user's visits never appear in training. The held-out fold is the validation data; we reserve no third split. The split is stratified on the next-category label because the seven-class task is imbalanced: Food is the majority class in every dataset, from about 25 percent of visits in Florida to 34 percent in Alabama (Table 8).

*Integrity of the representation.* We train the representation once on the whole dataset and feed it to every model, dedicated and joint. A representation built this way could pass information about a test visit along more than one channel. We therefore bound the channels that we can measure, and state for each what the measurement covers, on four grounds.

First, its training objective is label-free: it contrasts real graph neighborhoods against shuffled ones and never sees a next-category or next-region target. That bounds the training signal and not the inputs, since each visit's own category enters as a node feature (Section 5.4.1); the fourth ground below measures what that feature can carry between visits.

Second, we measured the transductive channel, what a graph fitted over all users passes to the test side that a graph fitted over training users alone would not. Rebuilding the representation per fold, from that fold's training users only, moves both tasks by at most a third of a point (region −0.33 to +0.01; category 0.00 to +0.29, at Alabama, Arizona, and Florida), within fold noise. This measurement covers the visits whose places appear in training (67 to 87 percent); visits to places unseen in training are the one part it cannot reach.

Third, the one component that could pass information between visits, a region-transition prior (a table of how often one region follows another), is built per fold from training data only, after an earlier whole-dataset version inflated region accuracy by 13 to 27 points. Our joint and dedicated models do not use this prior; it appears only in the HMT-GRN baseline (Section 5.5.4).

Fourth, we probe the channel that the graph's own links between consecutive visits open. A visit's node is linked to the visit that follows it, and category is a node input feature, so a per-visit vector could in principle absorb the category of the next visit. A screening audit run during development tests exactly this: it predicts the next category from the last window slot alone. The audit reads each encoder against a clean reference encoder rather than against the category labels directly. At Florida the graph encoder of our own lineage is that reference, at 0.4090 macro-F1 on standardized features and 0.4074 on raw ones, on a zero-to-one scale, and the residual variant that the encoder we ship descends from sits at the same level, 0.4197 and 0.4182; an attention-based encoder screened beside them reached 0.4976 and 0.4863, clear of the reference by more than the screen's margin, and was disqualified on that evidence.

A second reference can be computed directly from the category sequence, with no encoder involved. We call it the label-history benchmark: the best macro-F1 reached by four specified predictors that read only the genuine category history of the input window. It is not an upper bound on what a model may score. The four predictors are a specified set, and a better predictor of the same restricted information could exceed them. The benchmark lies below the clean reference: 0.3617 at Florida, and 0.2800 to 0.3617 across the five datasets where the inputs are available. The clean references therefore lie above the label-history benchmark, by four to six points at Florida, and so does every other candidate screened beside them on the raw run of the probe. One candidate, a relation-typed graph encoder, falls below the benchmark on the

standardized run (0.3328) and above it on the raw one (0.4142); it is not a counter-example because it is the candidate that passed this screen and then leaked under a downstream sequence model, which is the reason the screen bounds candidates against each other rather than any one of them against an absolute standard.

The measurement bounds this channel rather than closing it, and three limits set how far it reaches: the probe is linear, it was run at Florida alone at one random initialization over five user-grouped folds, and it was run on those ancestor builds of the representation rather than on the one that produced the results reported here. The same record shows why the linear form is a screen and not a proof, since one encoder that passed it leaked under a downstream sequence model. The baseline representations meet the same standard: HGI is pre-trained once on the whole dataset, like ours, and CTLE per fold on training users only.

### 5.5.3 Metrics and statistical tests

For the category task, we report macro-averaged F1 (macro-F1), which averages the per-class F1 so each of the seven categories counts equally; its reference point is the majority-class floor, the macro-F1 of always predicting the most common category. For the region task, we report accuracy at ten (Acc@10), the fraction of test visits whose true region appears among the model's ten highest-scoring guesses (a visit whose true region is absent from that fold's training data counts as an error); its reference point is the dedicated model.

A claimed gain and a claimed match require different tests, because a difference that fails to reach significance is not by itself evidence of a match. A written analysis plan, fixed during development and before any result was read, assigned one test to each task, with the two-point margin pinned there: *superiority* for next-category, *non-inferiority* for next-region. It assigned the tests per task, not per dataset, and did not cover next-region superiority, so the four next-region gains of Section 5.6.2 are secondary results outside it. The plan registered a paired Wilcoxon signed-rank test; at four seeds its exact one-sided $p$ cannot fall below 0.0625, so we report a paired $t$ on the per-seed means, with the 90% confidence interval of the paired difference, and the registered test alongside it. Both, and this departure, are released with the code. A *seed* is one complete repetition of the five-fold experiment, over the same folds, with a different random initialization; folds and seeds are the two axes of repetition. On every dataset, both models use four seeds ($4 \times 5 = 20$ measurements) and the tests pair the per-seed means ($n$=4), with a Holm correction [73] across the six next-category comparisons and, separately, across the four next-region ones. The *non-inferiority* claim is that the joint model is no worse than the dedicated model by more than a two-point margin; we test it with the two one-sided tests (TOST) procedure [74]. The two-point margin is fixed in advance as well: a mobility-aware service acts on which region will be busy, not on a single rank position (Section 5.3), so a two-point shift in Acc@10 is below the granularity at which such a service would behave differently. The precision of the equivalence test is measured on this fixed partition: the paired difference's

standard deviation is 0.01 to 0.18 points across the datasets, and the intervals pass a margin as small as one point at Alabama and Arizona (Section 5.6.2).

### 5.5.4 Baselines

The comparisons play three roles. The first role is played by the per-task state of the art, on our data, under user-disjoint five-fold protocols. For next-category, this is POI-RGNN [22], re-implemented from its published architecture and hyperparameters, and a Markov baseline that predicts the category that most often follows the recent ones (Markov-K, best order per dataset); for region, we also compute a first-order Markov floor over region transitions, under the same sliding-window protocol and fold splits as our models. For next-region, the primary comparison is HMT-GRN [19], an end-to-end recurrent model (trained as one piece from the raw check-ins) evaluated on the same data, folds, and initialization. We keep its shared multitask skeleton and add a region-transition prior built per fold from training data. We drop its graph components and hierarchical beam search, which exist to serve its next-place target, a target that we do not predict. The result is a region-native model (region is one of its own prediction targets), not a reproduction of the complete published system. We also run STAN [15], re-implemented from its published architecture and trained from the raw check-ins with its own embeddings and sequence construction under one fixed configuration. We adapt its output layer to rank region candidates, spatial objects that still support its candidate-distance matching; we do not adapt it to category, where distance matching has no meaning. A ReHDM [98] reference is reported under its own published protocol.

The second role is played by two representation controls that attribute the category gain to our specific design, not to contextualization in general or to extra features: CTLE [18], the closest prior contextual check-in embedding, run end to end at Florida and with frozen weights (no fine-tuning) at Alabama, Arizona, and Istanbul; and a feature-concatenation control appending raw per-visit features to a standard place embedding under the same model.

The third role is a comparison between the two ways of coupling the tasks. The dominant published coupling is the directed cascade, predicting the category first and conditioning the place prediction on it [97, 20, 60]. We test the cascade inside our own model rather than re-implementing those systems: we remove the shared trunk, where the two streams exchange information, and feed the predicted category forward into the region pathway, keeping the representation, outputs, and training configuration unchanged. This isolates the coupling topology (chain versus parallel) at no additional parameter or compute cost: the chain form drops the shared trunk and adds a conditioning projection of about 1,000 parameters. A final control, the standard place-level embedding [32], isolates what the per-visit representation adds. All other choices, including the nine-visit window, were fixed during development; the full training configuration for every model is in the released code (footnote 1).

## 5.6 Results

### 5.6.1 The representation makes the category task learnable

The check-in-level representation outperforms a standard place embedding (HGI [32]) on next-category macro-F1 by a wide gap on every dataset (Table 9). The comparison is controlled: target, single-task model, training configuration, windowing, and folds are identical; only the input representation changes (a vector per visit versus a vector per place), so the difference isolates the representation. The check-in-level column keeps one fixed configuration, not the per-dataset-tuned dedicated model of Table 10. The gaps range from +27.63 (Arizona) to +39.62 (Florida), with Istanbul at +28.09. A controlled test isolates this: averaging the per-visit vectors into one vector per place removes roughly 64 to 90 percent of the gain (state-dependent), so most of the gain is the context that each visit carries, not extra training signal (more training vectors per place).

The geometry of the vectors shows why the representation helps this task in particular: the per-visit vectors' silhouette score by category (how tight and well separated the seven labeled groups are, on a $-1$ to 1 scale) is about 0.57 against about 0.00 for the place embedding, and their nearest-neighbor category purity (the share of nearest neighbors with the vector's own category) is about 0.98 against about 0.78 (both averaged over the five U.S. states). The same geometry does not separate regions, so the benefit is category-only (the joint model's spatial stream instead reads the region-level vectors of the same graph, Section 5.4.2). CTLE [18], the closest prior contextual embedding, also gives each visit its own vector, but it pretrains on place identifiers and timestamps alone, so the category vocabulary never enters its training, whereas our graph reads each visit's category and time of day as input features. At Florida, we fine-tune CTLE together with the task model, using its authors' defaults and the same 64 dimensions and windowing as ours. It reaches 33.45 macro-F1 at its best epoch and 29.69 at its final one (an epoch is one pass over the training data), about two points below the place embedding under the same best-epoch rule, and far below the check-in-level representation's 75.15. With frozen weights, fed to the same single-task model, CTLE repeats the ordering at Alabama, Arizona, and Istanbul, with our representation ahead by +37.8, +37.0, and +28.7 macro-F1. A feature-concatenation control joins the place embedding with the same raw per-visit features that our graph reads (the category one-hot plus hour and day-of-week encodings) and feeds the result to the same single-task model. It does not close the gap either: at Alabama, Arizona, and Florida it raises the place embedding by only +2.0, +1.7, and +0.8 macro-F1, under a tenth of the place-to-check-in gap at each state. The gain therefore comes from the hierarchical per-visit representation, not from contextualization alone or feature injection.

Figure 6 shows the same separability contrast graphically.

Table 9 – Next-category macro-F1: the check-in-level representation against a standard place embedding (HGI), same single-task model, training configuration, and folds (mean and fold sd, seed 0).

| Dataset | Check-in level | Place level | Δ |
|---|---|---|---|
| Istanbul | **54.65** ±0.6 | 26.56 ±0.5 | +28.09 |
| AL | **55.87** ±2.4 | 26.56 ±1.0 | +29.31 |
| AZ | **57.13** ±1.3 | 29.50 ±1.0 | +27.63 |
| FL | **75.15** ±0.5 | 35.53 ±0.3 | +39.62 |
| TX | **69.95** ±0.2 | 32.48 ±0.6 | +37.47 |
| CA | **70.26** ±0.3 | 32.31 ±0.4 | +37.95 |

The matching place-level value at Alabama and Istanbul (26.56) is a coincidence of two independent runs; their per-fold values differ.

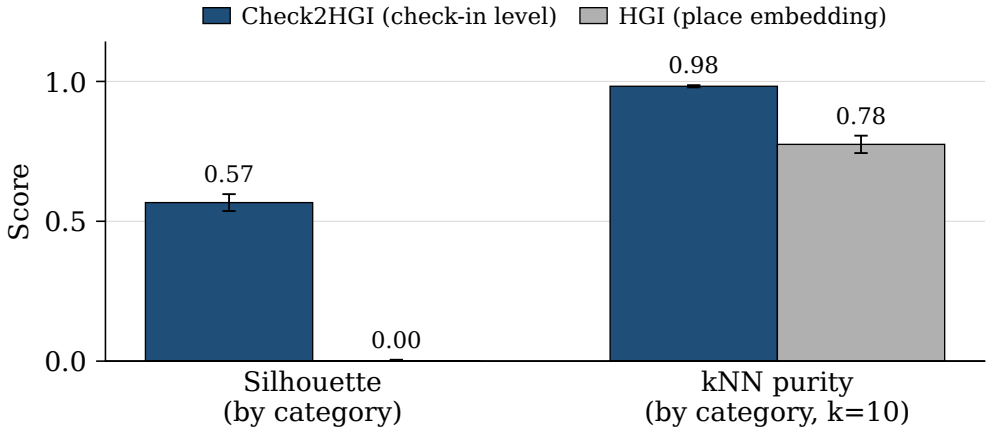


Figure 6 – Class separability of the representations, grouping each vector by its ground-truth category label (cosine silhouette; leave-one-out nearest-neighbor (kNN) purity, $k$=10), averaged over the five U.S. states. The check-in-level representation separates the seven categories; the place embedding does not.

### 5.6.2 One model, two tasks

One model outperforms the dedicated models on next category at all six datasets, and on next region it outperforms them at four of the six and is statistically non-inferior within a two-point margin at the other two. For scale: always predicting the most common category reaches only about 7 percent macro-F1 (the majority-class floor), and a random region top-ten guess is right at most about two percent of the time (Section 5.5.3).

Table 10 reports the single joint model against the dedicated single-task ceilings (the level a joint model is usually expected, at best, to match). The dedicated category model is tuned per dataset over batch size and learning rate (the dedicated region models use the strongest fixed configuration). Throughout, every reported model is one saved artifact per fold, read at its validation-selected epoch: each dedicated model at its task's best epoch, and the joint model at the epoch selected by its joint validation score (the geometric mean of the two task metrics), with both tasks read from that one model. Reading each joint task at its own best epoch instead

would change every joint result by at most 0.06 (category) and 0.11 (region) points.

Table 10 – One model, two tasks: the single joint model against the dedicated single-task models and the external baselines, ordered by region count. The upper block reports next-category (macro-F1) and the lower block next-region (Acc@10); the two share the dataset and region-count columns, and the same six datasets appear in the same order in both. Bold marks a statistically supported improvement over the dedicated model; in the region block, ↑ marks that supported improvement and ≈ a non-inferior match (TOST, ±2 pp; Section 5.6.2).

| | | **Next-category (macro-F1)** | | | |
|---|---|---|---|---|---|
| **Dataset** | **Regions** | **Markov-K** | **POI-RGNN** | **Dedicated** | **Joint (ours)** |
| Istanbul | 520 | 24.55 | 30.12 | 54.74 ±0.09 | **63.32** ±0.02 |
| AL | 1,109 | 20.50 | 23.80 | 56.82 ±0.03 | **64.51** ±0.09 |
| AZ | 1,547 | 23.92 | 27.64 | 56.43 ±0.10 | **65.79** ±0.02 |
| FL | 4,703 | 29.74 | 34.49 | 74.51 ±0.03 | **79.84** ±0.02 |
| TX | 6,553 | 28.67 | 33.03 | 69.79 ±0.08 | **77.24** ±0.01 |
| CA | 8,501 | 27.58 | 31.78 | 70.60 ±0.07 | **77.05** ±0.01 |

| | | **Next-region (Acc@10)** | | | | |
|---|---|---|---|---|---|---|
| **Dataset** | **Regions** | **HMT-GRN** | **ReHDM** | **STAN** | **Dedicated** | **Joint (ours)** |
| Istanbul | 520 | 60.4 | 69.33 | 61.86 | 75.16 ±0.01 | **75.35** ±0.04$^{\uparrow}$ |
| AL | 1,109 | 57.05 | 65.38 | 60.72 | 70.11 ±0.10 | 69.70 ±0.09$^{\approx}$ |
| AZ | 1,547 | 43.70 | 53.00 | 49.86 | 59.46 ±0.08 | 59.46 ±0.04$^{\approx}$ |
| FL | 4,703 | 63.74 | 64.49 | 72.99 | 76.70 ±0.02 | **77.41** ±0.02$^{\uparrow}$ |
| TX | 6,553 | 53.85 | 48.81$^{\ddagger}$ | 61.67$^{\dagger}$ | 64.95 ±0.01 | **67.06** ±0.01$^{\uparrow}$ |
| CA | 8,501 | 49.61 | 50.26$^{\ddagger}$ | 58.52$^{\dagger}$ | 63.49 ±0.02 | **65.69** ±0.02$^{\uparrow}$ |

HMT-GRN (region-native, on our folds, scored on visits whose region appears in training, >99% of test visits in every dataset and fold) is the primary external comparison; STAN (our re-implementation) and ReHDM (its own protocol) are references. Markov-K: strongest order per dataset. $^{\dagger}$STAN partial folds: TX 4/5, CA 2/5 (seed 0). $^{\ddagger}$ReHDM at TX and CA: a single seed. Joint and dedicated entries: four seeds × five folds; ±: sd across seeds.

Figure 7 plots the signed gains ordered by region count. On category, the joint model outperforms the dedicated ceiling on every dataset, by +5.33 to +9.35 macro-F1 (smallest at Florida, largest at Arizona, and +8.58 at Istanbul). On region (Acc@10), the joint model outperforms the dedicated ceiling at Florida, Texas, California, and Istanbul, and stays a non-inferior match (TOST, ±2 pp) at Alabama and Arizona. Across the five U.S. states, the region gain rises with region count, from −0.41 at the smallest count to +2.20 at the largest; region count and corpus size co-vary here, so we read the trend across the points rather than as a precise law.

On category, all four seeds favor the joint model on every dataset, and each gain is significant after a Holm correction across the six datasets (paired $t$, corrected $p < 0.001$); the four next-region gains hold under their own Holm correction as well (corrected $p < 0.001$). The registered Wilcoxon test on the individual fold differences agrees at every dataset, with all

20 folds favoring the joint model (corrected $p < 0.001$). On region, Alabama and Arizona are tested for equivalence (TOST, ±2 pp) and pass with 90% confidence intervals well inside two points (each entry: point estimate; interval): Alabama (−0.41; −0.63 to −0.20) and Arizona (0.00; −0.08 to +0.07). At Arizona, the interval is centered on zero, so we report a match, not a gain. At Alabama, the whole interval lies below zero, a small but statistically significant deficit, still well within the two-point margin. At Florida and Istanbul, the 90% intervals lie entirely above zero (Florida +0.71; +0.67 to +0.76; Istanbul +0.19; +0.15 to +0.23), so the joint model outperforms there, though both gains are smaller than the two-point margin of Section 5.5.3. Texas and California exceed that margin: their 90% intervals lie entirely above it (+2.10 to +2.13 at Texas, +2.19 to +2.21 at California).

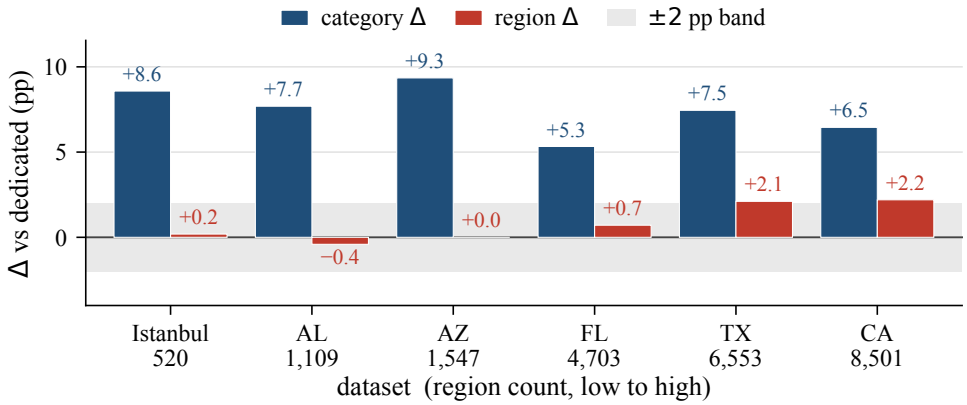


Figure 7 – Signed gains of the joint model over the dedicated single-task models, ordered by region count. The category gain (macro-F1) is positive at every dataset; the region gain (Acc@10) rises across the five U.S. states and is also positive at Istanbul. The ±2-point band applies to region only.

A reader used to multitask learning [5] expects the harder task, here next-region with its hundreds to several thousand classes, to teach the easier one; a control shows otherwise. We fix the region pathway at its initial values at the start of training so it can neither learn nor teach the category task, yet the full category gain survives at Alabama, Arizona, and Florida: the control reaches 63.50, 63.67, and 79.79 macro-F1, above the single-task category score of the one fixed configuration (Table 9) by +7.63, +6.54, and +4.64 points. That comparison is the one the control was built to make, and it is the one to read. The control predates the results of Table 10 and was measured at one random initialization over five folds, so its second comparison is to the joint scores of the development configuration current at the time (63.56, 63.39, 79.82), which it matched to within 0.3, and not to the joint entries reported here. The control therefore rules out one reading and narrows the rest. It rules out the region task teaching the category one, since the gain survives with the region pathway untrained. What it leaves is the joint architecture itself, and the control does not say which part of it: freezing the region stream removes region training, not the category stream's own encoder, the per-stream feed-forward blocks, or the added depth. We therefore report the negative result, that the gain is not cross-task transfer, as a finding, and we attribute the gain to the joint architecture rather than to any named component of it. A

separate ablation in our own development record, at Florida only, removed the cross-attention stack while holding the rest fixed and moved next-category macro-F1 by $-0.04 \pm 0.13$, which a paired test cannot separate from zero. Two limits keep us from reading that as an absence of contribution. The ablation ran on an earlier configuration whose region output was driven by a region-transition prior the models reported here do not use, and the development record that produced it reads the null as a compensation effect, in which the category stream absorbs what the shared stack would otherwise supply, rather than as a component that does nothing. We therefore do not name the shared trunk as the source, and we do not present the ablation as evidence against it either.

Either way the gain requires no second model at serving time (one model, one forward pass).

The joint model is also above every external baseline reported, on both tasks, across all six datasets (Table 10): on region, the primary region-native comparison HMT-GRN [19], STAN [15] trained from the raw check-ins, and a ReHDM reference [98]; on category, POI-RGNN [22] and the Markov-K floor. These externals run on their own embeddings, so this comparison also includes the representation advantage of Section 5.6.1; the controlled comparison for the joint model remains the Dedicated column of Table 10, which uses the same representation, windows, and folds. The first-order Markov region floor of Section 5.5.4, computed under our windows and folds, reaches 51 to 72 Acc@10 across the datasets; the joint model exceeds it by 4.9 to 10.3 points on all six datasets.

That floor is also above the three external region systems at most datasets, and the comparison deserves a direct word rather than being left for the reader to assemble. HMT-GRN falls below it at all six, the ReHDM reference at three, and STAN at four. Two facts about how these numbers were produced bear on that comparison. The first concerns the floor: it is computed under our own sliding-window protocol and fold splits. Those windows advance one visit at a time, so the region of the last visit is a strong predictor of the next one, and a first-order transition table reads exactly that signal. At Alabama the target region is the last visited region in 32.9 percent of windows. The second fact concerns the three systems, which do not meet the floor on equal terms. HMT-GRN is evaluated on the same data, folds, and initialization as our models. STAN runs on the same folds, but builds its own embeddings and its own sequences from the raw check-ins. The ReHDM reference runs under its own published protocol, so its result is not measured on our windows or folds at all.

Neither fact establishes why the floor lies above the three systems, and we do not claim a single explanation. We treat the floor, not the external systems, as the reference the region task has to clear, and we read the external columns as a comparison against published designs rather than as the standard for this task. The dedicated and joint models are above the floor at every dataset.

We prefer coupling the tasks in parallel rather than in a chain: neither target is subordinate to the other, so a chain must pick one to predict first and pass its errors to the other. The choice still had to be checked, because CSLSL reports its chain outperforming a shared-trunk parallel variant on its own benchmarks [60]. Rewired as a cascade (Section 5.5.4), our model reaches the same combined score, the geometric mean of the two task metrics, as its parallel form: at Alabama, Arizona, Florida, and Istanbul, with both forms trained under the same configuration and initialization, the difference is about zero and neither task moves by more than a quarter of a point. We read this as a defense of the parallel design, not a claim that we outperform the cascade: on this representation the coupling topology makes no measurable difference, and the simpler parallel structure loses nothing. Two qualifications bound this reading: the cascade runs under the configuration tuned for the parallel model, and its form was fixed in advance, so tuning the cascade itself remains future work.

### 5.6.3 External validity (Istanbul)

The joint-model results hold for a non-U.S. city under the same representation, sliding-window protocol, and training setup as the U.S. states. At Istanbul, the joint model outperforms the dedicated category ceiling by +8.58 macro-F1 (dedicated 54.74, joint 63.32) and is slightly above the dedicated region ceiling at +0.19 Acc@10 (dedicated 75.16, joint 75.35), a gain with statistical support (the whole 90% confidence interval of the paired difference lies above zero, and TOST, ±2 pp, also passes). The comparable quantity is the gain over the ceiling, not the absolute Acc@10, since region counts differ across datasets (520 mahalle here). The external baselines show the same ordering on both tasks (Table 10). The U.S. result repeats on a different continent and region unit.

## 5.7 Discussion and Limitations

One model serves both tasks. In one forward pass it outperforms the dedicated category model at all six datasets, and on region it outperforms the dedicated model at four of them and matches it within a two-point margin at the other two, with the region output keeping a private spatial path (Table 10; Figure 7). Which part of the joint architecture produces the category gain is not settled by the controls reported here (Section 5.6.2).

A service could treat the model's top-ranked next regions as an anticipatory set: at California, ten regions out of 8,501 contain the true next region 65.69 percent of the time, over 500 times better than picking ten at random. On four datasets (Alabama, Arizona, Florida, and Istanbul; a single seed over five folds), the ten shortlisted regions lie a median of 3 to 8 kilometers from the shortlist's centroid, against 17 to 176 kilometers for ten regions drawn at random from the same candidate set (median over 10,000 draws). This remains motivation, not a measured service result.

Three limits qualify these results. First, our representation is trained once over all places; visits to places never seen during training are the single effect that we cannot fully isolate. A planned follow-up trains it on each fold's training places only and embeds unseen visits. Second, epoch selection consults the fold that the score is then read on (Section 5.5.2), so every absolute score reported here is optimistic. The comparison between the joint model and the dedicated models is affected far less, for two reasons we can state outright: the selection rule is the same for both models on the same folds, an epoch chosen on validation and read on that fold, with each model selected on its own validation objective (Section 5.6.2), and the dedicated model receives the wider search, a per-dataset sweep over batch size and learning rate against one configuration held fixed across all six datasets for the joint model. The residual therefore favors the comparator, which makes the reported difference conservative. It does not follow that the bias cancels exactly. The four seeds also reuse one fixed fold partition, so the reported intervals cover variation across random initializations and not across resampled user splits. Third, we do not build or evaluate a mobility-aware service; it is background motivation, and this chapter's claims are the prediction results themselves.

Our representation is a fixed per-visit vector that any downstream model can consume. Moura et al. [94], whose structural analysis of tourist check-ins reads past visits only, name machine learning as a next step; this study moves in that direction.

## 5.8 Conclusion

We tested whether one model can predict both the category and region of a user's next visit. A check-in-level representation, one vector per visit rather than one per place, makes the next-category task far more learnable than the standard place-level representation does. On that representation, a single multitask model outperforms a dedicated category model on every dataset. On region, it outperforms the dedicated model on four of the six datasets and remains non-inferior (TOST, ±2 pp) on the other two. The gains at Texas and California exceed the two-point margin; those at Florida and Istanbul are statistically supported but smaller. Across the U.S. states, the region gain rises with region count. The joint model also remains above every external baseline in Table 10 on all six datasets, by at least 4 Acc@10 points over the strongest region reference (HMT-GRN, STAN, or ReHDM; the first two on our folds, ReHDM under its own protocol) and by at least 33 macro-F1 points over POI-RGNN on category. One model with a shared semantic context and a private spatial path anticipates both what a user will do next and where.

# 6 Conclusion

This dissertation asked whether multitask learning helps predict the next category and the next region of a visit, and which design choices determine the answer. Three studies addressed this question in sequence. The first developed a joint model that did not consistently outperform its dedicated counterparts (Chapter 3). The second kept the model architecture fixed, changed the input representation, and identified that representation as the main bottleneck in the tested configuration (Chapter 4). Those two studies paired next-category prediction with a static category classification task, and the pair changed in the final study. The third study combined a check-in-level representation with a redesigned sharing topology (Chapter 5). Under this design, the joint model outperforms the dedicated single-task models on the category task at all six datasets and on the region task at four of the six. At the other two, it remains statistically non-inferior within a two-point margin (TOST).

## 6.1 Contributions by chapter

Chapter 3 presents MTLnet, the first joint model developed in this research. It combined static category classification with next-category prediction and shared its lower layers across the two tasks. Both tasks read a place-level graph embedding. In this configuration, the joint model cost more to train and did not consistently outperform the dedicated single-task models on either task. The chapter contributes this negative finding and an analysis of three possible causes: task dissimilarity and the resulting risk of negative transfer, an input representation that may be insufficient for both tasks, and an overly rigid sharing topology. The finding is limited to a place-level input under hard sharing. The next two studies examine what happens when these conditions change.

Chapter 4 kept the MTLnet architecture fixed but replaced its single 64-dimensional place embedding with separate spatial, temporal, and categorical encoders. On the static task, category macro-F1 rose by 20.2 to 22.0 percentage points across the three states tested. This increase exceeded those obtained from the architectural variations evaluated in the first study.

This gain requires two qualifications: First, the static task classifies a place from that place's own representation, so the input already determines the target. The gain therefore provides no evidence about the sequential task. Second, the comparison is not width-matched: the decomposed input has 192 dimensions, whereas the place embedding has 64. Chapter 4 therefore calls for an equal-dimension control to separate the semantic contribution of the encoders from the effect of the additional width.

The sequential task provides the relevant diagnosis because its input does not determine

its target. The decomposed encoders are more effective in most of the tested cases. Because the sharing topology remained fixed, this result identifies the input representation as the main bottleneck in that configuration.

Chapter 5 responds to this diagnosis with Check2HGI, a check-in-level representation that assigns a distinct vector to each visit rather than reusing one vector for every visit to the same place. The joint model exchanges information between two task-specific streams through cross-attention, while a private spatial path serves the region task. The evaluation used user-disjoint cross-validation, with twenty fitted models per configuration: four seeds over one fixed set of five folds. Paired tests used the four per-seed means.

Under this protocol, the joint model outperforms the dedicated single-task models on the category task at all six datasets, by 5.3 to 9.4 macro-F1 points. On the region task, it outperforms them at four datasets: Istanbul, Florida, California, and Texas. At Alabama and Arizona, it remains statistically non-inferior within a two-point margin (TOST). Non-inferiority was the registered primary analysis for the region task; the four superiority findings are secondary results outside that analysis plan.

## 6.2 The consolidated answer

Does multitask learning help point-of-interest prediction? The answer depends on the design. In the configurations studied here, the decisive choices were the input representation and the topology used to share parameters between the tasks. Identifying these conditions is the dissertation's main finding.

The result of each study determined the question of the next. In Chapter 3, a place-level embedding and naive hard sharing did not consistently outperform the two dedicated models. At that stage, two possible explanations were negative transfer between the static and sequential tasks and an input representation that served neither task well. This verdict applied to that configuration. By holding the architecture fixed and changing only the input, Chapter 4 identified the representation, rather than the sharing scheme, as the main bottleneck. This diagnosis motivated a representation built at the level of each check-in.

Chapter 5 combined that representation with a sharing topology designed for it. The resulting model produces both predictions in one forward pass. It outperforms the dedicated single-task models on the category task at all six datasets and on the region task at four of the six. At the other two, it remains statistically non-inferior within a two-point margin (TOST).

One premise did not survive the controls, and it concerns the mechanism rather than the outcome. Section 2.3 states the expectation under which these models were built: related tasks trained together should improve generalization through a shared representation. The two controls and the gradient measurements narrow this explanation of the gain.

The first control tests whether the next-region task's training signal causes the category improvement. Chapter 5 fixes the region pathway at its initial values, which prevents learning on that side but leaves the rest of the joint architecture intact. The control used one random initialization over five folds and supported one matched comparison: the fixed-region joint model against the dedicated category model under the same training configuration. The fixed-region model retains its category advantage at Alabama, Arizona, and Florida. Within this control, the category improvement therefore does not require training transfer from the region task. This finding excludes that explanation but does not distinguish among the category encoder, the feed-forward blocks, the added depth, cross-attention, or a combination of these components.

A separate ablation at Florida examines cross-attention more directly. Removing the cross-attention stack changed next-category macro-F1 by $-0.04 \pm 0.13$, which the paired Wilcoxon test could not distinguish from zero. This finding does not show that cross-attention has no effect on the final model. The ablation used a region-transition prior that is absent from the final models, and the remaining components could adapt to the removal during training. They may therefore have compensated for it. The available evidence supports attributing the category gain to the joint architecture as a whole, not to one component.

The second control tests whether model size alone explains the gain by matching the capacity of a dedicated category model to that of the joint model. At Alabama, widening the dedicated model from about 0.6 million to about 4.2 million parameters gives it the same parameter budget as the joint model. The widened model was fitted under three training configurations, with twenty models per configuration and sixty in total. Its best configuration reaches $56.16 \pm 1.89$ macro-F1, reported as the mean and standard deviation over twenty fitted models. The dedicated model at its tuned width reaches $56.82 \pm 0.03$, while the joint model reaches $64.51 \pm 0.09$.

California shows the same pattern, with a hidden dimension of 752, the widened model has 101.9 percent of the joint model's parameter count and reaches $69.88 \pm 0.26$ macro-F1 over twenty fitted models. The dedicated model at its tuned, narrower width reaches $70.60 \pm 0.07$, while the joint model reaches $77.05 \pm 0.01$.

At both datasets, widening leaves the dedicated model slightly below its tuned, narrow-width result: 0.66 points lower at Alabama and 0.72 at California. It therefore recovers none of the joint model's gain. The widened models were tuned separately, and their best configurations used a lower learning rate than those of the narrow models. However, the search for the narrow models was wider than the three configurations tested at Alabama and the two tested at California for the widened models. Within this scope, increasing the parameter count alone did not improve the dedicated model.

The gradient measurements support this interpretation within their limited scope. An auxiliary analysis on a separate data preparation obtained an average cosine similarity of +0.001 between the task gradients over four seeds and four Gowalla states. Three of these states are

among the five reported in this dissertation. The measurement captures only directional conflict and applies only to this task pair. Appendix D measures the same quantity for the final model over seven datasets: the six reported here and one additional Gowalla state. It also obtains cosine values close to zero. Under the check-in-level representation, these results indicate little directional conflict between the two sequential tasks. They are also consistent with the small effect of gradient-balancing optimizers in this configuration and the effectiveness of a tuned fixed weighting.

Together, the controls exclude two explanations within the settings in which they were evaluated: direct training transfer from the region task and parameter count alone. They do not identify which remaining component of the joint architecture produces the gain. The gradient measurements weigh against a third explanation. Because directional conflict between the tasks is small, resolving such conflict does not explain the benefit of joining them. The relatedness of the two tasks motivated the joint model, but it is not, on this evidence, the source of that model's advantage.

The negative result of Chapter 3 and the positive result of Chapter 5 apply to different designs and therefore do not contradict each other. Multitask learning provides no automatic benefit, and related tasks alone do not guarantee one. This dissertation shows one tested path from the negative result to a positive one: improve the input representation and redesign how the tasks share parameters.

## 6.3 Limitations

Six limitations bound the scope of these conclusions.

1. **Data vintage.** The five state datasets come from Gowalla [10], and the extraction used here spans January 2009 to August 2011 across the five states. The Istanbul dataset [66] is not appreciably more recent for most of its volume: its check-ins fall in two separate periods, 2012 to 2013 and 2017 to 2018, with none in between, and roughly seven in ten belong to the earlier period. Mobility patterns, place inventories, and check-in behavior have changed since.

2. **Taxonomy coarseness.** Next-category prediction uses seven top-level classes. A finer taxonomy may change the effect of joint training.

3. **Transductive representation.** Check2HGI is trained on the check-in graph of each dataset, so it cannot represent unseen places or users without retraining. The place-level representation it builds on [32] is trained the same way, and methods that learn aggregators instead of per-node vectors [28] are the alternative this constraint points toward.

4. **No next-place task.** The experiments do not predict the exact next POI, and their conclusions apply only to next category and next region.

5. **Geographic coverage.** Outside the United States, the evidence rests on a single city, Istanbul.

6. **The task-pair confound.** The task pair changed together with the representation and the sharing topology. Chapters 3 and 4 paired static category classification with next-category prediction, while Chapter 5 pairs two sequential targets, next-category and next-region prediction. No single controlled ablation separates the representation-and-topology change from the task-pair change in the final result. Chapter 4 is the fixed-pair control for the diagnosis. The capacity-matched baseline above addresses the parameter-count explanation only in the setting that it tested: the category task, two of the six datasets, one width point per dataset, and width scaling rather than depth. The greater similarity of the final pair may still contribute to the size of the improvement. The ablation that would separate the two changes, running static category classification under the check-in-level representation, is not clean under that representation: the category of the visited place is an input feature of a check-in node, so that target would be partly readable from its own input. This follows from the design of the representation and was not measured, so the confound is bounded by the fixed-pair control rather than removed.

## 6.4 Future work

Future work follows directly from these limitations. Newer and denser traces would test the conclusions beyond the Gowalla vintage (limitation 1), and finer-grained taxonomies would test them beyond the seven-class division (limitation 2). Because the check-in-level representation is trained on a fixed graph, an inductive variant of it, one that embeds unseen places and users without retraining, would remove the transductive constraint and support deployment in growing cities. Two further changes to the same representation would test how much of its behavior follows from how it is assembled rather than from the hierarchy: its place-vector table is initialized from a pretrained table and then held near it during training, so an ablation that varies that coupling would separate the contribution of the initialization, and a hypergraph formulation, in which one edge joins the several visits of a session rather than only consecutive pairs, would test whether the pairwise graph is the right structure for a visit history (limitation 3).

Adding the exact next place as a third target would extend the joint model beyond the two properties predicted in this work. The cascade architectures reviewed in Chapter 5 suggest that category and region predictions can provide additional structure for this task, and reusing the existing check-in-level representation would require a change to the input pipeline and one additional output rather than a new representation, which is also the route by which this

representation would serve contexts other than the two studied here (limitation 4). Further cities outside the United States would widen the geographic base (limitation 5). Isolating the effect of changing the task pair needs a static target that the check-in-level representation does not already carry as an input feature, since running the earlier pair directly under this representation is not a clean comparison (limitation 6).

## 6.5 Final remarks

The practical output of this dissertation is a single model that predicts two properties of the next visit in one forward pass: what kind of place the user will visit and in which part of the city. The methodological contribution is the sequence of evidence that led to this model: a published negative result, the diagnosis that identified the input representation as the main bottleneck, and a solution designed from that diagnosis. The negative result was not an obstacle to the contribution. It was its first half.

# References


[1] SILVA, V. H. O. et al. An investigation into multi-task learning for point-of-interest category classification and next-POI prediction. In: **Anais do XVII Congresso Brasileiro de Inteligência Computacional (CBIC 2025)**. [S.l.: s.n.], 2025. p. 1–8.

[2] SONG, C. et al. Limits of predictability in human mobility. **Science**, v. 327, n. 5968, p. 1018–1021, 2010.

[3] LUCA, M. et al. A survey on deep learning for human mobility. **ACM Comput. Surv.**, v. 55, n. 1, p. 7:1–7:44, 2021.

[4] XU, R. et al. Tme: Tree-guided multi-task embedding learning towards semantic venue annotation. **ACM Trans. Inf. Syst.**, v. 41, n. 4, p. 112, 2023. ISSN 1046-8188.

[5] CARUANA, R. Multitask learning. **Mach. Learn.**, v. 28, n. 1, p. 41–75, 1997.

[6] KOKKINOS, I. Ubernet: Training a 'universal' convolutional neural network for low-, mid-, and high-level vision using diverse datasets and limited memory. In: **Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.: s.n.], 2017. p. 5454–5463. ArXiv:1609.02132.

[7] LIPTON, Z. C. et al. Learning to diagnose with lstm recurrent neural networks. **arXiv preprint arXiv:1511.03677**, 2015.

[8] WEI, J. et al. Finetuned language models are zero-shot learners. In: **Proceedings of the 10th International Conference on Learning Representations (ICLR)**. [s.n.], 2022. Disponível em: <https://openreview.net/forum?id=gEZrGCozdqR>.

[9] SILVA, T. H. et al. Urban computing leveraging location-based social network data: A survey. **ACM Comput. Surv.**, v. 52, n. 1, p. 17:1–17:39, 2019.

[10] CHO, E.; MYERS, S. A.; LESKOVEC, J. Friendship and mobility: User movement in location-based social networks. In: **Proc. ACM SIGKDD (KDD)**. [S.l.]: ACM, 2011. p. 1082–1090.

[11] LIU, Q. et al. Predicting the next location: A recurrent model with spatial and temporal contexts. In: **Proc. AAAI Conf. Artif. Intell.** [S.l.: s.n.], 2016. p. 194–200.

[12] FENG, J. et al. DeepMove: Predicting human mobility with attentional recurrent networks. In: **Proc. Web Conf. (WWW)**. [S.l.: s.n.], 2018. p. 1459–1468.

[13] KONG, D.; WU, F. HST-LSTM: A hierarchical spatial-temporal long-short term memory network for location prediction. In: **Proc. IJCAI**. [S.l.: s.n.], 2018. p. 2341–2347.

[14] YANG, D. et al. Location prediction over sparse user mobility traces using RNNs: Flashback in hidden states! In: **Proc. IJCAI**. [S.l.: s.n.], 2020. p. 2184–2190.

[15] LUO, Y.; LIU, Q.; LIU, Z. STAN: Spatio-temporal attention network for next location recommendation. In: **Proc. Web Conf. (WWW)**. [S.l.: s.n.], 2021. p. 2177–2185.

[16] LIAN, D. et al. Geography-aware sequential location recommendation. In: **Proc. KDD**. [S.l.: s.n.], 2020. p. 2009–2019.

[17] YANG, S.; LIU, J.; ZHAO, K. GETNext: Trajectory flow map enhanced transformer for next POI recommendation. In: **Proc. ACM SIGIR**. [S.l.: s.n.], 2022. p. 1144–1153.

[18] LIN, Y. et al. Pre-training context and time aware location embeddings from spatial-temporal trajectories for user next location prediction. In: **Proc. AAAI Conf. Artif. Intell.** [S.l.: s.n.], 2021. v. 35, p. 4241–4248.

[19] LIM, N. et al. Hierarchical multi-task graph recurrent network for next POI recommendation. In: **Proc. ACM SIGIR**. [S.l.: s.n.], 2022. p. 1133–1143.

[20] YU, F. et al. A category-aware deep model for successive POI recommendation on sparse check-in data. In: **Proc. Web Conf. (WWW)**. [S.l.: s.n.], 2020. p. 1264–1274.

[21] ZHU, N. et al. Predicting a person's next activity region with a dynamic region-relation-aware graph neural network. **ACM Trans. Knowl. Discov. Data**, v. 16, n. 6, p. 1–23, 2022.

[22] CAPANEMA, C. G. S. et al. Combining recurrent and graph neural networks to predict the next place's category. **Ad Hoc Netw.**, v. 138, p. 103016, 2023.

[23] MIKOLOV, T. et al. Efficient estimation of word representations in vector space. In: **Proc. ICLR (Workshop)**. [S.l.: s.n.], 2013.

[24] PEROZZI, B.; AL-RFOU, R.; SKIENA, S. DeepWalk: Online learning of social representations. In: **Proc. KDD**. [S.l.: s.n.], 2014. p. 701–710.

[25] GROVER, A.; LESKOVEC, J. node2vec: Scalable feature learning for networks. In: **Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining**. [S.l.: s.n.], 2016. p. 855–864.

[26] KIPF, T. N.; WELLING, M. Semi-supervised classification with graph convolutional networks. In: **Proc. ICLR**. [S.l.: s.n.], 2017.

[27] VELIČKOVIĆ, P. et al. Graph attention networks. In: **Proc. ICLR**. [S.l.: s.n.], 2018. ArXiv:1710.10903.

[28] HAMILTON, W. L.; YING, R.; LESKOVEC, J. Inductive representation learning on large graphs. In: **Proc. NeurIPS**. [S.l.: s.n.], 2017.

[29] BELGHAZI, M. I. et al. Mutual information neural estimation. In: **Proc. ICML**. [S.l.: s.n.], 2018. p. 531–540.

[30] HJELM, R. D. et al. Learning deep representations by mutual information estimation and maximization. In: **Proc. ICLR**. [S.l.: s.n.], 2019.

[31] VELIČKOVIĆ, P. et al. Deep graph infomax. In: **Proc. ICLR**. [S.l.: s.n.], 2019.

[32] HUANG, W. et al. Learning urban region representations with POIs and hierarchical graph infomax. **ISPRS J. Photogramm. Remote Sens.**, v. 196, p. 134–145, 2023.

[33] KAZEMI, S. M. et al. Time2vec: Learning a vector representation of time. **arXiv preprint arXiv:1907.05321**, 2019.

[34] SITZMANN, V. et al. Implicit neural representations with periodic activation functions. In: **Advances in Neural Information Processing Systems**. [S.l.: s.n.], 2020. v. 33, p. 7462–7473. ArXiv:2006.09661.

[35] MAI, G. et al. **Multi-Scale Representation Learning for Spatial Feature Distributions using Grid Cells**. 2020. Disponível em: <https://arxiv.org/abs/2003.00824>.

[36] MAI, G. et al. Sphere2vec: A general-purpose location representation learning over a spherical surface for large-scale geospatial predictions. **ISPRS J. Photogramm. Remote Sens.**, v. 202, p. 439–462, 2023. ArXiv:2306.17624.

[37] RUßWURM, M. et al. Geographic location encoding with spherical harmonics and sinusoidal representation networks. In: **Proc. ICLR**. [s.n.], 2024. ArXiv:2310.06743. Disponível em: <https://openreview.net/forum?id=PudduufFLa>.

[38] RUDER, S. **An Overview of Multi-Task Learning in Deep Neural Networks**. 2017. ArXiv:1706.05098.

[39] PEREZ, E. et al. FiLM: Visual reasoning with a general conditioning layer. In: **Proc. AAAI Conf. Artificial Intelligence**. [S.l.: s.n.], 2018. p. 3942–3951.

[40] MISRA, I. et al. Cross-stitch networks for multi-task learning. In: **Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.: s.n.], 2016. p. 3994–4003.

[41] MA, J. et al. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. In: **Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining**. [S.l.: s.n.], 2018. p. 1930–1939.

[42] TANG, H. et al. Progressive layered extraction (PLE): A novel multi-task learning (MTL) model for personalized recommendations. In: **Proc. RecSys**. [S.l.: s.n.], 2020. p. 269–278.

[43] HAZIMEH, H. et al. DSelect-k: Differentiable selection in the mixture of experts with applications to multi-task learning. In: **Proc. NeurIPS**. [S.l.: s.n.], 2021.

[44] YU, T. et al. Gradient surgery for multi-task learning. In: **Proc. NeurIPS**. [S.l.: s.n.], 2020. v. 33.

[45] STANDLEY, T. et al. Which tasks should be learned together in multi-task learning? In: **Proc. Int. Conf. Machine Learning (ICML)**. [S.l.: s.n.], 2020. ArXiv:1905.07553.

[46] SENER, O.; KOLTUN, V. Multi-task learning as multi-objective optimization. In: **Advances in Neural Information Processing Systems (NeurIPS)**. [S.l.: s.n.], 2018. v. 31, p. 527–538. ArXiv:1810.04650.

[47] NAVON, A. et al. Multi-task learning as a bargaining game. In: **Proceedings of the 39th International Conference on Machine Learning**. PMLR, 2022. (Proceedings of Machine Learning Research, v. 162), p. 16428–16446. Disponível em: <https://proceedings.mlr.press/v162/navon22a.html>.

[48] LIU, B. et al. Conflict-averse gradient descent for multi-task learning. In: **Proc. NeurIPS**. [S.l.: s.n.], 2021.

[49] SENUSHKIN, D. et al. Independent component alignment for multi-task learning. In: **Proc. IEEE/CVF CVPR**. [S.l.: s.n.], 2023. p. 20083–20093.

[50] KENDALL, A.; GAL, Y.; CIPOLLA, R. Multi-task learning using uncertainty to weigh losses for scene geometry and semantics. In: **Proc. IEEE/CVF CVPR**. [S.l.: s.n.], 2018. p. 7482–7491.

[51] CHEN, Z. et al. GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks. In: **Proc. ICML**. [S.l.: s.n.], 2018. p. 794–803.

[52] LIU, S.; JOHNS, E.; DAVISON, A. J. End-to-end multi-task learning with attention. In: **Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)**. [S.l.: s.n.], 2019. Introduces Dynamic Weight Average (DWA).

[53] LIU, B. et al. FAMO: Fast adaptive multitask optimization. In: **Proc. NeurIPS**. [S.l.: s.n.], 2023. v. 36.

[54] LIN, B. et al. Reasonable effectiveness of random weighting: A litmus test for multi-task learning. **Transactions on Machine Learning Research**, 2022.

[55] XIN, D. et al. Do current multi-task optimization methods in deep learning even help? In: **Proc. NeurIPS**. [S.l.: s.n.], 2022. v. 35.

[56] KURIN, V. et al. In defense of the unitary scalarization for deep multi-task learning. In: **Proc. NeurIPS**. [S.l.: s.n.], 2022. v. 35.

[57] VANDENHENDE, S. et al. Multi-task learning for dense prediction tasks: A survey. **IEEE Transactions on Pattern Analysis and Machine Intelligence**, v. 44, n. 7, p. 3614–3633, 2022.

[58] YU, J. et al. Unleashing the power of multi-task learning: A comprehensive survey spanning traditional, deep, and pretrained foundation model eras. **arXiv preprint arXiv:2404.18961**, 2024. Disponível em: <https://arxiv.org/abs/2404.18961>.

[59] LIAO, D. et al. Predicting activity and location with multi-task context aware recurrent neural network. In: **Proc. IJCAI**. [S.l.: s.n.], 2018. p. 3435–3441.

[60] HUANG, Z. et al. Human mobility prediction with causal and spatial-constrained multi-task network. **EPJ Data Sci.**, v. 13, n. 1, p. 22, 2024.

[61] ZHANG, L. et al. An interactive multi-task learning framework for next POI recommendation with uncertain check-ins. In: **Proc. IJCAI**. [S.l.: s.n.], 2020. p. 3551–3557.

[62] HALDER, S. et al. Transformer-based multi-task learning for queuing time aware next poi recommendation. In: **Advances in Knowledge Discovery and Data Mining**. Cham: Springer International Publishing, 2021. p. 510–523.

[63] HALDER, S. et al. Poi recommendation with queuing time and user interest awareness. **Data Mining and Knowledge Discovery**, v. 36, p. 2379–2409, 2022.

[64] WANG, Y. et al. Hierarchy aware-based multi-task learning for user location prediction. **The Journal of Supercomputing**, v. 81, n. 11, 2025.

[65] WANG, Y. et al. Iemtlf: Interaction-enhanced multi-task learning framework for next location prediction. **Information Sciences**, v. 661, p. 120153, 2024.

[66] WONGSO, W.; XUE, H.; SALIM, F. D. **Massive-STEPS: Massive Semantic Trajectories for Understanding POI Check-ins — Dataset and Benchmarks**. 2025. ArXiv:2505.11239.

[67] YANG, D. et al. Modeling user activity preference by leveraging user spatial temporal characteristics in LBSNs. **IEEE Transactions on Systems, Man, and Cybernetics: Systems**, v. 45, n. 1, p. 129–142, 2015.

[68] SOKOLOVA, M.; LAPALME, G. A systematic analysis of performance measures for classification tasks. **Information Processing & Management**, v. 45, n. 4, p. 427–437, 2009.

[69] GAMBS, S.; KILLIJIAN, M.-O.; CORTEZ, M. N. del P. Next place prediction using mobility markov chains. In: **Proc. MPM (EuroSys Workshop)**. [S.l.: s.n.], 2012. p. 1–6.

[70] KOHAVI, R. A study of cross-validation and bootstrap for accuracy estimation and model selection. In: **Proc. IJCAI**. [S.l.: s.n.], 1995. v. 14, p. 1137–1143.

[71] PEDREGOSA, F. et al. Scikit-learn: Machine learning in Python. **Journal of Machine Learning Research**, v. 12, p. 2825–2830, 2011.

[72] WILCOXON, F. Individual comparisons by ranking methods. **Biometrics Bulletin**, v. 1, n. 6, p. 80–83, 1945.

[73] HOLM, S. A simple sequentially rejective multiple test procedure. **Scand. J. Statist.**, v. 6, n. 2, p. 65–70, 1979.

[74] LAKENS, D. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. **Soc. Psychol. Personal. Sci.**, v. 8, n. 4, p. 355–362, 2017.

[75] LESKOVEC, J.; KREVL, A. **SNAP Datasets: Stanford Large Network Dataset Collection**. 2014. <https://snap.stanford.edu/data/loc-gowalla.html>. Accessed: 2024-05-21.

[76] CHEN, M. et al. Modeling spatial trajectories with attribute representation learning. **IEEE Transactions on Knowledge and Data Engineering**, v. 34, n. 4, p. 1902–1914, 2022.

[77] ZENG, J. et al. A next location predicting approach based on a recurrent neural network and self-attention. In: **Collaborative Computing: Networking, Applications and Worksharing (CollaborateCom)**. [S.l.]: Springer, 2019. p. 309–322.

[78] ZHANG, Y.; YANG, Q. A survey on multi-task learning. **IEEE Transactions on Knowledge and Data Engineering**, v. 34, n. 12, p. 5586–5609, 2022.

[79] RUDER, S. et al. Sluice networks: Learning what to share between loosely related tasks. **arXiv preprint arXiv:1705.08142**, 2017.

[80] XIA, B. et al. Mtpr: A multi-task learning based poi recommendation considering temporal check-ins and geographical locations. **Applied Sciences**, v. 10, n. 19, 2020. ISSN 2076-3417. Disponível em: <https://www.mdpi.com/2076-3417/10/19/6664>.

[81] DU, J. et al. Beyond geo-first law: Learning spatial representations via integrated autocorrelations and complementarity. In: IEEE. **2019 IEEE international conference on data mining (ICDM)**. [S.l.], 2019. p. 160–169.

[82] HUANG, W. et al. Estimating urban functional distributions with semantics preserved poi embedding. **International Journal of Geographical Information Science**, Taylor & Francis, v. 36, n. 10, p. 1905–1930, 2022.

[83] BAXTER, J. A model of inductive bias learning. **Journal of Artificial Intelligence Research**, v. 12, p. 149–198, 2000.

[84] VASWANI, A. et al. Attention is all you need. In: **Advances in Neural Information Processing Systems**. [S.l.: s.n.], 2017. p. 5998–6008. ArXiv:1706.03762.

[85] PAIVA, T. S. et al. ST-MTLNet: Representações espaço-temporais de pontos de interesse para aprendizado multitarefa. In: **Anais do X Workshop de Computação Urbana (CoUrb 2026)**. [S.l.]: Sociedade Brasileira de Computação (SBC), 2026. p. 323–336.

[86] WU, N. et al. Torchspatial: A location encoding framework and benchmark for spatial representation learning. **Advances in Neural Information Processing Systems**, v. 37, p. 81437–81460, 2024. ArXiv:2406.15658.

[87] FENG, S. et al. POI2Vec: Geographical latent representation for predicting future visitors. In: **Proc. AAAI Conf. Artif. Intell.** [S.l.: s.n.], 2017. v. 31, n. 1, p. 102–108.

[88] RAHMANI, H. A. et al. Category-aware location embedding for point-of-interest recommendation. In: **Proceedings of the 2019 ACM SIGIR international conference on theory of information retrieval**. [S.l.: s.n.], 2019. p. 173–176.

[89] SUN, K. et al. Where to go next: Modeling long-and short-term user preferences for point-of-interest recommendation. In: **Proceedings of the AAAI conference on artificial intelligence**. [S.l.: s.n.], 2020. v. 34, n. 01, p. 214–221.

[90] SUN, Y. Transtarec: Time-adaptive translating embedding model for next poi recommendation. In: IEEE. **2024 5th International Conference on Computer Engineering and Application (ICCEA)**. [S.l.], 2024. p. 647–651.

[91] MIKOLOV, T. et al. Distributed representations of words and phrases and their compositionality. In: **Advances in Neural Information Processing Systems (NIPS)**. [S.l.: s.n.], 2013.

[92] BASTUG, E.; BENNIS, M.; DEBBAH, M. Living on the edge: The role of proactive caching in 5G wireless networks. **IEEE Commun. Mag.**, v. 52, n. 8, p. 82–89, 2014.

[93] VIELHAUS, C. L. et al. Handover predictions as an enabler for anticipatory service adaptations in next-generation cellular networks. In: **Proc. ACM MobiWac**. [S.l.]: ACM, 2022. p. 19–27.

[94] MOURA, D. L. L.; AQUINO, A. L. L.; LOUREIRO, A. A. F. On the design of mobility-aware systems: A tourist's perspective. In: **Proc. IEEE/ACM MSWiM**. [S.l.: s.n.], 2025. p. 667–674.

[95] SUN, Z. et al. A multi-channel next POI recommendation framework with multi-granularity check-in signals. **ACM Trans. Inf. Syst.**, v. 42, n. 1, p. 1–28, 2024.

[96] SUN, K.; XU, M. **Knowledge Graph Tokenization for Behavior-Aware Generative Next POI Recommendation**. 2025. ArXiv:2509.12350.

[97] YE, J.; ZHU, Z.; CHENG, H. What's your next move: User activity prediction in location-based social networks. In: **Proc. SIAM SDM**. [S.l.: s.n.], 2013. p. 171–179.

[98] LI, X. et al. Beyond individual and point: Next POI recommendation via region-aware dynamic hypergraph with dual-level modeling. In: **Proc. IJCAI**. [S.l.: s.n.], 2025. p. 3081–3089.

# Appendix

# APPENDIX A – Other Scientific Contributions

## A.1 Reproducing the reported numbers

This section names, for each part of the protocol, the code that implements it and the file its output lives in. Every path is relative to the working repository of this research. The protocol described is Chapter 5's, and Chapter 2 scopes it there rather than to the whole collection. The public snapshot that Chapter 5 footnotes carries the model, the representation, the baselines and the fold construction; the statistical scripts and their output files listed below are part of the working repository and are supplied on request.

**The fold partition.** Folds are built by <src/data/folds.py>, which calls scikit-learn's `StratifiedGroupKFold` with five splits, shuffling enabled, and a fixed partition seed of 42, grouping by user identifier and stratifying on the task label. Grouping by user is what makes the split user-disjoint: a user's check-ins fall on one side of every fold. The same routine builds the paired multitask folds, where both tasks share one user partition so that a user cannot reach one task's training data and the other task's validation data in the same fold.

**The seeds.** Four random initializations, 0, 1, 7, and 100, each repeating the five-fold experiment over that one fixed partition, which is the twenty fitted models per configuration the chapter reports. The design is recorded in <docs/studies/closing_data/v17_completion/STATISTICAL_PROTOCOL.md>. Only the initialization varies across the four; the folds do not, which is why the reported intervals cover variation across initializations and not across resampled user splits, a limitation Chapter 5 states.

**The region-transition prior.** Rebuilt for each fold from that fold's training partition alone by <scripts/build_phase3_per_fold_transitions.sh>, which writes one file per fold. Only the external region baseline of Chapter 5 consumes it; the joint and dedicated models do not.

**The reported scores.** Read at one saved model per fold, the epoch selected on validation, which is the joint-best convention: both tasks are read at the same checkpoint rather than each at its own best epoch. The post-hoc scorer is <scripts/closing_data/score_joint_best.py> and its results are collected in <docs/studies/closing_data/joint_best/JOINT_BEST_RESULTS.md>.

**The significance tests.** The superiority and non-inferiority tests are implemented in <scripts/closing_data/superiority_wilcoxon.py> and <scripts/closing_data/region_match_tost.py>. The reported analysis was run by <m1_stats_n20.py>, with the pre-registered per-fold variant in <m2_prereg_perfold.py>; both scripts, their console output (<m1_full_output.txt> and <m2_prereg_output.txt>), and the verdict tables that Chapter 5 quotes sit together in <docs/studies/closing_data/v17_completion/stats_n20/>.

**The label-history benchmark.** Computed by <scripts/embedding_eval/autocorrelation_ceiling.py>.

Two qualifications belong with this list. The metric conventions it presupposes are defined where they are used, macro-F1 and accuracy at ten in Chapter 2 and again in Chapter 5, and this section does not restate them. And the list covers the final study alone. The experiments of Chapters 3 and 4 ran under the weaker protocol described earlier in this appendix, the second of them in a separate repository, so a reader reproducing those chapters is reproducing what they report and not what this section describes.

# APPENDIX B – AI-Use Disclosure

This dissertation was written with the assistance of a generative artificial intelligence tool, Claude (Anthropic), in its Opus 4.8, Fable 5, and Opus 5 versions, used as a research and writing assistant under the author's direction. This appendix discloses the scope of that use, as the CNPq integrity policy requires (Portaria nº 2.664/2026) and the UFV/DPE disclosure guidelines of 2026 recommend.

The scope of use, per part of the work, was the following.

**Drafting.** The frame chapters (Chapters 1, 2, and 6), the appendices, and the front matter were drafted by the assistant from author-approved outlines, source documents, and decision records, section by section. The three article chapters (Chapters 3 to 5) are re-typeset reproductions of the articles' own texts; the assistant performed the re-typesetting and the English translation of the Chapter 4 article, which passed a separate sentence-level fidelity check against the published Portuguese text.

**Editing and review.** Assistant-run review passes checked citations, numbers, claims, cross-references, and style, each pass run by an agent that did not write the text under review. Their findings were corrections for the author to accept or reject, not approvals.

**Formatting.** The LaTeX structure, the two build modes, the bibliography consolidation, and figure and table re-setting were produced by the assistant.

**Code.** Analysis and experiment code was written by the author with assistant support; the assistant also wrote verification scripts used in the review gates.

Three rules governed the drafting. Every reference was checked against its source of record, with the cited claim located in the source. Every number was traced to the file it came from and quoted rather than computed. Every verdict verb was bound to the statistical test that supports it. Passages that could not be verified were flagged for the author instead of being smoothed over. The author read, corrected, and approved the final text, and is responsible for every word of it.

# APPENDIX C – Data Ethics and Governance

Every result in this dissertation is measured on check-ins published by location-based social networks, and neither collection used here was assembled by the author. This appendix states where each came from, under what terms, what the pipeline does with it, and what remains unsettled.

## C.1 Where the data came from

The Gowalla check-ins are read from a category-annotated deposit published on Figshare, DOI <10.6084/m9.figshare.22126586.v2>, which carries the Creative Commons CC0 public-domain dedication.[1] Four of its files enter the pipeline: the check-in table, two place tables, and the category structure file. The seven place categories used throughout the dissertation are the top-level names in that structure file, so the taxonomy arrives with the deposit; what the pipeline adds is the mapping of each finer category up to its top-level parent, plus a short local supplement for names the structure file leaves uncovered.

Two qualifications belong with that label, and both come from the record itself. The dedication was applied by the depositor of this copy, identified there by a single name, and not by the party that collected the data. The record also names a further site as the origin of the files, and that address now redirects to an unrelated commercial site, so its terms could not be read. What is supportable is therefore a statement about the copy in hand, which is distributed under CC0; that Gowalla data as such is in the public domain is a broader claim, and nothing that could be opened supports it. A second and older Gowalla release, hosted by the Stanford Large Network Dataset Collection and cited here as the collection reference [10, 75], is a different artifact: it carries no place categories, and its page states no license.

The Istanbul check-ins come from the Massive-STEPS collection [66], distributed on Hugging Face under the Apache License 2.0.[2] The project repository carries the same license. Massive-STEPS is derived material, and its documentation names the Semantic Trails Dataset and Foursquare Open Source Places as its sources; among the records that could be opened, no upstream term more restrictive than the Apache License 2.0 appeared. Two facts qualify that. The Foursquare distribution is additionally access-gated behind an agreement prompt, so a reader who goes to that upstream must accept terms before downloading, while the Massive-

[1] <https://creativecommons.org/publicdomain/zero/1.0/>
[2] <https://huggingface.co/datasets/CRUISEResearchGroup/Massive-STEPS-Istanbul>

STEPS copy used here is not gated. And one check is outstanding: the Foursquare product terms themselves were not read, only the license tag on the distribution.

## C.2 Real people, and how the traces are handled

A user identifier in these files is a pseudonym and not a name, but a sequence of timestamped visits is still a description of one person's movements. Pseudonymity is not anonymity. The mobility literature treats the residual risk as open, and reports that models trained on mobility data raise privacy questions during training as well as at prediction time, because the portions of information a model absorbs cannot be controlled directly [3]. Public availability settles who may hold the files, not what can be inferred from them.

Against that, this work adds no de-identification of its own. No coordinate is perturbed, rounded, generalized, or masked, and no formal privacy mechanism is applied. Latitude and longitude are used at the precision the sources publish, because the prediction target of Chapter 5 is itself spatial, and a coarsened coordinate would change the quantity being measured.

Exposure is limited by restriction instead. The Figshare deposit also contains a social-graph file and a user-profile file; neither is declared as an input, and no code reads either one, so the friendship network and the profile fields never enter the study. User identifiers are carried as opaque integers exactly as the sources supply them, and a non-numeric identifier is replaced by a position index that is not kept. Nothing in the code links a user across the two collections or to any outside source.

No check-in data is redistributed. The public repository publishes code and configuration, and the data directory is excluded from version control, so a reader reproducing the results obtains both collections from the sources named above.

# APPENDIX D – Why the Two Tasks Do Not Compete on the Shared Trunk

A joint model that shares parameters between two tasks can end up worse at both than two dedicated models are at one each [45]. The usual reason is a disagreement, and it shows up in the gradients: when the two tasks ask for opposite updates, one task improves at the other's expense. That has a direct measurement, and this appendix runs it on the joint model of Chapter 5.

The two tasks turn out not to disagree. Their gradients are statistically indistinguishable from orthogonal on every dataset measured, which is a stronger statement than mere absence of conflict, and it carries a direct consequence for that model: a gradient balancer had nothing to balance. The sections below give the measurement, the two findings that qualify it, and the boundary beyond which none of it is claimed.

## D.1 What was measured, and on what unit

At every training epoch, each task loss was back-propagated to the shared cross-attention trunk, and the cosine of the angle between the two resulting gradient vectors was recorded, the quantity the gradient-surgery literature uses to define task conflict [44]. The cosine is the right quantity because it is scale-free: it reads how the two requested updates are aligned and ignores how large they are, which differs between the tasks and drifts during training. A cosine of $+1$ means the tasks ask for the same update, $-1$ for opposite ones, and 0 that the requests are orthogonal, each leaving the other's objective unchanged to first order.

The observations are 4,650 epoch-level cosines from seven datasets. Florida contributes 3,150 across twelve configurations that vary the loss weight, the weighting schedule, and the training procedure; Alabama, Arizona, California, Georgia, Istanbul, and Texas contribute 250 each at the canonical configuration. Every run is five-fold user-disjoint cross-validation over fifty epochs.

Six of the seven are the six Chapter 5 reports on. The seventh, Georgia, is a further Gowalla state the dissertation does not otherwise use, included here as an additional test of the same diagnostic. Chapter 5 also reports its own smaller measurement of this quantity; that one comes from a different run, so the two sets of numbers are not interchangeable.

Those fifty values are not fifty independent measurements: they are consecutive states of one training run. The unit of independence is the fold, and every test below runs on folds: five fold means at each dataset, and at Florida its sixty fold series, five folds for each of its twelve

configurations. Each dataset is measured at a single random initialization, so the variation the tests see is across folds, not across restarts. Where a count of observations appears it describes the data, not a test's sample size.

Equivalence to zero is assessed by two one-sided tests against a margin of ±0.05 [74]. The margin is a judgment about relevance and should be read as one: at $|\cos| < 0.05$ one task's update projects less than five percent of its length onto the other's direction, below the epoch-to-epoch variation in either gradient and below anything a balancing method could act on. Chapter 5's two-point margin for the region task measures a different quantity on a different scale and should not be read against this one.

## D.2 The result

Table 11 – The cosine between the next-region and next-category gradients on the shared trunk, over the six datasets of Chapter 5 and Georgia. The rows are fold-level: every test takes the fold as its unit. Equivalence to zero holds at every dataset (TOST, ±0.05); † marks a dataset where the exact sign test sits at its floor of 0.0625, which five folds cannot go below.

| | | Sample | | Mean cosine | | *p* against zero | | |
|---|---|---|---|---|---|---|---|---|
| Dataset | Unit | *n* | obs. | 95% CI | mean | TOST | *t* / sign | pos. |
| Florida | fold series | 60 | 3,150 | [−0.0010, +0.0015] | +0.0003 | $10^{-62}$ | 0.68 / 0.90 | 31/60 |
| Alabama | fold | 5 | 250 | [+0.0040, +0.0184] | +0.0112 | $10^{-5}$ | 0.013 / 0.063† | 5/5 |
| Arizona | fold | 5 | 250 | [−0.0051, +0.0081] | +0.0015 | $10^{-5}$ | 0.56 / 1.00 | 3/5 |
| California | fold | 5 | 250 | [+0.0000, +0.0014] | +0.0007 | $10^{-9}$ | 0.048 / 0.38 | 4/5 |
| Texas | fold | 5 | 250 | [−0.0024, +0.0018] | −0.0003 | $10^{-7}$ | 0.74 / 0.38 | 4/5 |
| Istanbul | fold | 5 | 250 | [−0.0008, +0.0011] | +0.0001 | $10^{-9}$ | 0.76 / 1.00 | 3/5 |
| Georgia | fold | 5 | 250 | [+0.0016, +0.0061] | +0.0039 | $10^{-7}$ | 0.009 / 0.063† | 5/5 |

† 0.0625 is the smallest value the exact sign test can return at $n = 5$; no five-fold row can reach 0.05 by that test. Florida read at the configuration mean instead ($n = 12$) gives the same +0.0003 and remains equivalent, TOST $p = 1.3 \times 10^{-16}$.

Every dataset is equivalent to zero, and Table 11 gives each one's mean, confidence interval and tests. All seven means lie within one and a half hundredths of zero, against a margin of five hundredths, and equivalence holds at every other level of aggregation as well, including the raw observations, so the conclusion does not depend on the choice of unit.

The distance between that result and a null result is the point of this appendix. A test that merely failed to reject zero would support one statement, that no conflict was detected, which is equally consistent with a conflict too small to see at this sample size. An equivalence test supports the positive claim instead: the mean alignment lies inside a margin fixed in advance, so whatever alignment is there is too small to matter. That is a claim about the tasks, not about the experiment's power.

Figure 8 shows the distributions, and one feature must be stated plainly: the equivalence is a statement about the mean, not about every observation. Of the 4,650 cosines, 92.4 percent fall inside the margin, and the full range runs from −0.34 to +0.58, the largest value lying past the right edge of panel (a). Individual epochs drift out of the margin in both directions, ordinary noise in a quantity computed from one mini-batch. What never happens is a systematic pull to one side.

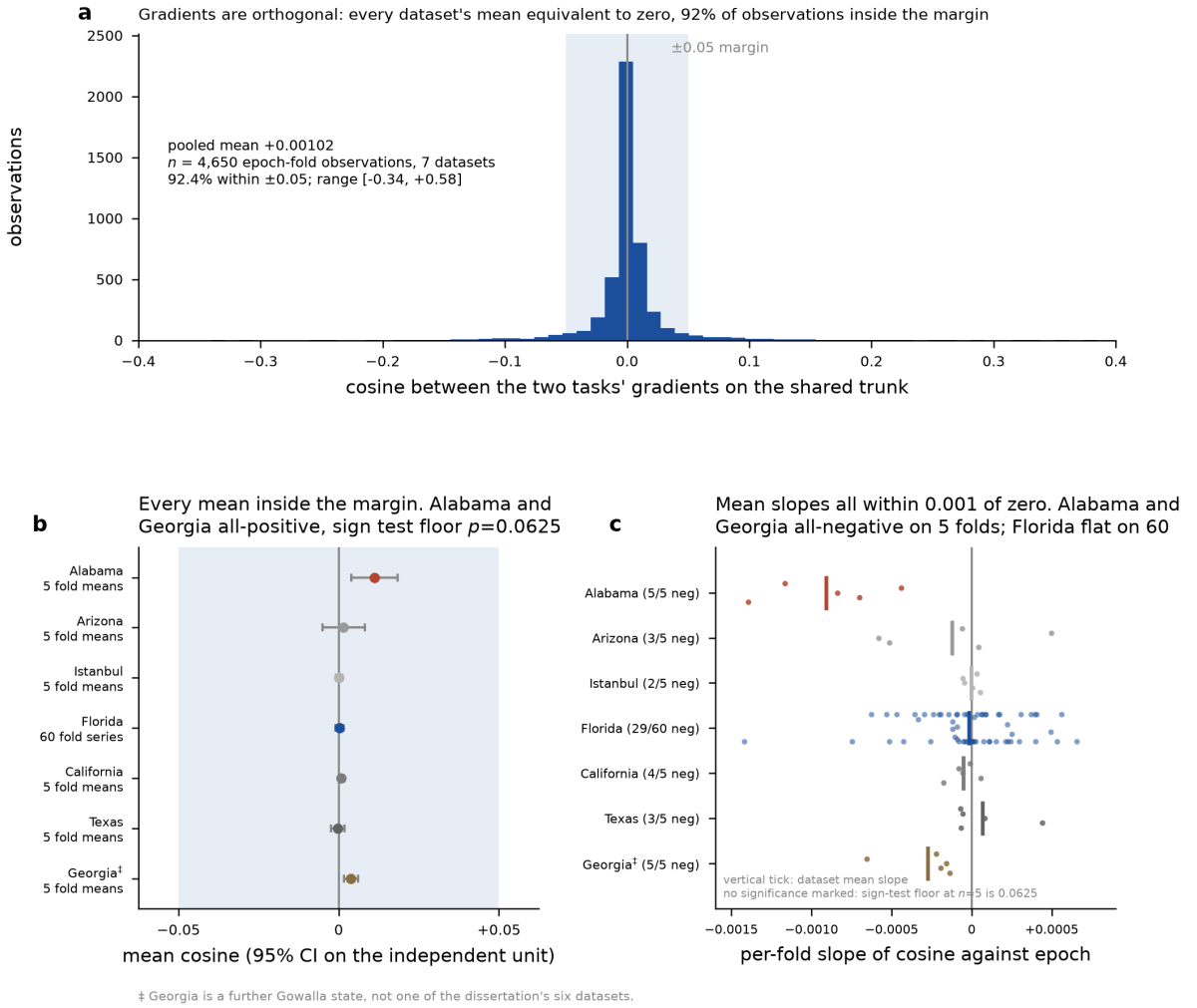


Figure 8 – The two tasks' gradients on the shared trunk are orthogonal, on seven datasets: the six of Chapter 5 and Georgia, marked ‡ because the dissertation does not otherwise use it. The shaded band is the equivalence margin of ±0.05; every mean falls inside it. Panels: (a) the pooled distribution of all 4,650 epoch-level cosines, (b) each dataset's mean with its 95 percent interval on the independent unit, (c) one point per fold for the slope of the cosine against the epoch, with a tick at each dataset's mean slope. No significance is marked anywhere in the figure.

The figure shows two patterns. The first is a positive tendency that recurs on two datasets: all five of Alabama's fold means are positive, averaging +0.0112, and so are all five of Georgia's, averaging +0.0039. No other dataset is unanimous in either direction. The second is a decline over training, on those same two and with the same unanimity, all five per-fold slopes negative

in each; the other four single-configuration datasets are mixed, from two of five at Istanbul to four of five at California, and Florida is flat at twenty-nine of sixty.

Neither pattern is called significant here, and the reason is the design rather than the data. At five folds the exact sign test cannot return less than 0.0625, so no distribution-free test could reach significance however consistent the pattern. A $t$-test does reject at Alabama and at Georgia, for the positive means and for the declines, but with five folds that rejection depends on assuming the fold means are normally distributed, an assumption five values cannot check. Both patterns are therefore consistent tendencies rather than established effects. California shows why the table carries a sign-test column: its $t$-test returns 0.048, below the conventional threshold, while its exact sign test returns 0.375 on four of five positive folds, so what looks like a finding under one test is not even a leaning under the other. Florida, whose sixty independent series are the only estimate in the set with the resolution to settle either question, shows neither pattern.

Neither pattern suggests a conflict. A positive cosine means the two tasks cooperate slightly, and the declining cosines stay inside the margin and move toward zero, not away from it. No mechanism is proposed for either pattern.

## D.3 What orthogonality explains

Orthogonal gradients mean neither task's update reinforces or cancels the other's on the shared parameters, so the trunk can serve both without either improvement being paid for by the other. The main consequence is about gradient balancers. Nash-MTL and the family around it exist to resolve exactly the disagreement measured here, reweighting or reprojecting task gradients so a dominant task cannot overwrite a weaker one. Orthogonality leaves them nothing to resolve. That is why Chapter 5 finds no balancer improving on a fixed loss weighting: the measurement explains that finding. It says nothing about the cost of a common trunk, which this architecture does not use and no arm of that chapter tested. Orthogonal gradients also do not mean that the tasks share no knowledge: the two streams still exchange information through the cross-attention trunk, a sharing mechanism this measurement does not read.

## D.4 How far this extends, and how far it does not

Seven datasets and twelve configurations bound this result along two axes, each answering a different objection.

The first axis is the tuning. Every one of Florida's twelve configurations is equivalent to zero on its own five folds, and their twelve means span $[-0.00261, +0.00457]$ over the observations as recorded. Orthogonality survives every hyperparameter and procedure that was varied, so it is not an artifact of one choice among them.

The second axis is the data. The seven datasets cover every dataset Chapter 5 reports on: the check-in counts run from 113,846 at Alabama to 4,089,892 at Texas and the region counts from 520 at Istanbul to 8,501 at California, with Georgia added under the same protocol. Equivalence holds at both ends: the result is not a quirk of Florida, and it is not an artifact of small data.

Together the two axes suggest orthogonality belongs to the task pair rather than to the tuning. Next category asks which of seven kinds of place a user visits next, next region asks which tract or neighborhood, and both read the same trace to predict a different projection of one event. Those projections appear close to statistically independent given the shared representation, which would explain why each task's update carries so little of the other's.

That is where the evidence stops, and the boundary matters. Every run measured here uses one architecture family, the cross-attention joint model of Chapter 5. The architecture decides where the two tasks meet and how much they share, which makes it at once the factor most likely to change the answer and the one this appendix did not vary. Nothing here says the gradients stay orthogonal in a model that shares more of its depth, couples the tasks in a cascade, or shares an output layer. Answering that question needs the same diagnostic and nothing else changed: the per-epoch cosine on the candidate's shared parameters, under a fixed loss weighting, across one dataset's folds, with the fold means tested for equivalence against a margin chosen in advance. Equivalence there makes orthogonality the task pair's property; otherwise it belongs to this architecture, and Section D.3 applies only to models shaped like this one.

# APPENDIX E – How Check2HGI and the Joint Model Work

This appendix explains the two methods used in the final study as one operational pipeline. The emphasis is on the path followed by the data: what each stage receives, how it changes that input, why the transformation is needed, and what is passed to the next stage. Exact settings that determine tensor compatibility are collected in Table 12; lower-level implementation details remain available in the source directories listed at the end of the appendix.

The pipeline has two separately trained parts. Check2HGI learns representations of individual visits and geographic regions without using the next-category or next-region targets. The joint model then reads sequences of those fixed representations and learns the two forecasting tasks together. Check2HGI therefore answers how a visit should be represented in its temporal and geographic context. The joint model answers how category and region prediction can exchange information while retaining task-specific routes.

## E.1 The complete method at a glance

The complete flow contains five stages:

1. validate the check-in records, order every user's visits by time, and map each place to a geographic polygon;
2. construct temporal, place, and region graphs linked by the check-in–place–region–city hierarchy;
3. train Check2HGI, then export separate 64-dimensional check-in and region vectors;
4. create stride-one windows containing nine observed visits and use the tenth visit as the prediction target; and
5. train one joint model to predict the next category and next region from two separate representation sequences.

The boundary between Stages 3 and 4 is important. Check2HGI is fitted first, and its exported tables remain fixed during supervised training. The joint model does not reconstruct the graph or update the representation encoder. Conversely, Check2HGI never receives the two forecast labels.

## E.2 From check-in records to a heterogeneous mobility graph

Each observation is represented by

$$c_i = (u_i, p_i, l_i, g_i, t_i). \tag{E.1}$$

Here, $c_i$ is check-in $i$, $u_i$ is its user, $p_i$ is its place, $l_i$ contains latitude and longitude, $g_i$ is one of seven place categories, and $t_i$ is the timestamp. A spatial join between $l_i$ and the polygon file assigns the check-in's place to a region. Places outside every polygon are removed, and entity identifiers are mapped to stable contiguous integers. This mapping ensures that a row in an embedding table always refers to the same check-in, place, or region.

Visits are sorted by user and timestamp before any temporal edge or forecast window is created. This order is part of the method. Changing it changes both the neighborhoods processed by Check2HGI and the examples processed by the joint model.

The resulting heterogeneous graph is $\mathcal{G} = (\mathcal{V}, \mathcal{E}, \mathcal{T}_v, \mathcal{T}_e)$. The set $\mathcal{V}$ contains the nodes, $\mathcal{E}$ contains the edges, $\mathcal{T}_v$ contains the node types, and $\mathcal{T}_e$ contains the relation types. The mappings $\phi : \mathcal{V} \to \mathcal{T}_v$ and $\psi : \mathcal{E} \to \mathcal{T}_e$ assign a type to each node and edge. The graph has four node levels and three forms of structure:

**Check-in succession.** Consecutive visits by the same user are connected in both directions. Their edge weight decreases exponentially with the time interval, using a one-hour decay constant. These edges describe the local order of a trajectory rather than geographic proximity.

**Spatial neighborhoods.** A Delaunay triangulation connects nearby places. Region nodes are connected when their polygons intersect. These graphs allow information to propagate between nearby places and adjacent administrative areas.

**Hierarchy membership.** Every check-in belongs to one place, every retained place belongs to one region, and every region belongs to the city. These assignments connect the four resolutions of the representation.

Category co-occurrence is absent from the reported edge set. Repeated visits to the same place also receive no additional check-in edge; they meet through their common place node. Category enters through the initial node features and an auxiliary reconstruction objective.

Each check-in begins with 11 values: seven category indicators, sine and cosine for hour of day, and sine and cosine for day of week. Cyclic encoding makes the endpoints of the daily and weekly clocks adjacent in feature space. Coordinates are not appended to this vector. They determine polygon membership, Delaunay edges, and region adjacency instead.

## E.3 How Check2HGI builds the representations

Check2HGI performs a bottom-up pass through the hierarchy. Graph convolutions mix information among neighbors at the same level. Attention pooling moves information between levels by summarizing a variable number of lower-level nodes. Figure 9 separates the self-supervised representation flow from the two tables exported to the joint model.

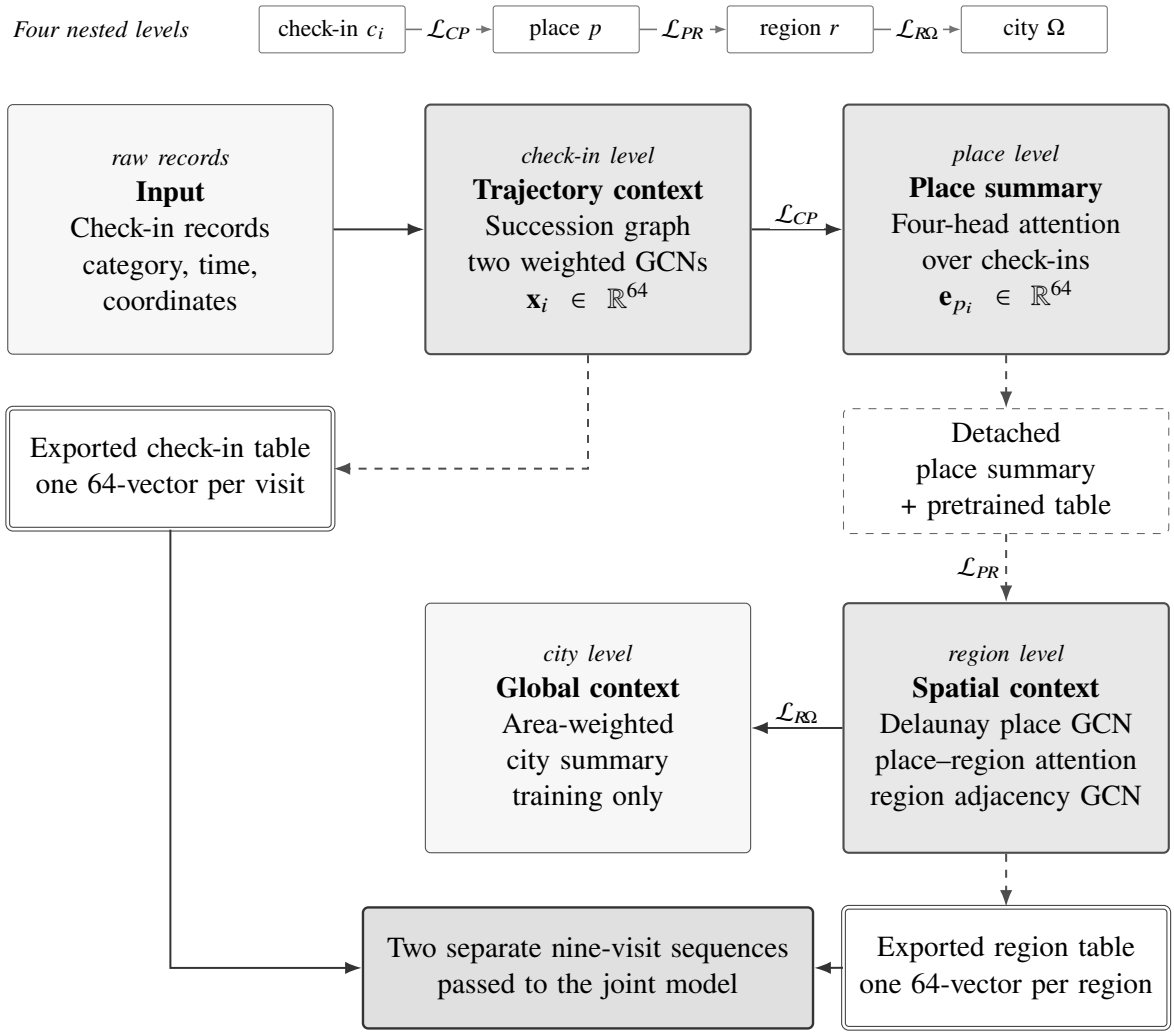


Figure 9 – Check2HGI representation and export flow. The strip at the top names the four nested levels of the graph and the objective that spans each boundary between them. Below it, each processing stage carries a tag naming its own level, and the bottom-up pass is read left to right along the top, then down the right-hand side, then right to left. Dashed arrows mark exported or detached routes. Double-bordered boxes are the representations consumed by the joint model.

### E.3.1 Step 1: place each visit in its trajectory

Two weighted graph-convolution layers process the check-in succession graph. The first layer maps the 11 input features to 64 dimensions and applies layer normalization, PReLU, and dropout. The second layer maps 64 to 64 and is added to the first-layer state as a residual update.

The output for visit $i$ is the check-in representation $\mathbf{x}_i \in \mathbb{R}^{64}$.

This transformation changes what one row means. Before the graph encoder, a row describes only the visit's own category and cyclic time. After the encoder, it also reflects the immediately preceding and following visits in the same user's trajectory. The temporal edge weights determine how strongly those neighbors contribute.

### E.3.2 Step 2: summarize visits at their place

All check-in representations assigned to the same place are pooled with four attention heads. This is a learned summary rather than a fixed mean. The module can assign different importance to routine and unusual visits while producing one 64-dimensional place representation $\mathbf{e}_{p_i} \in \mathbb{R}^{64}$.

The attention module uses one learned query vector, shared across all places. Keys and values come from the visits assigned to the place being summarized, so the query is common while the pooled content is specific to the place. Each head summarizes a 16-dimensional part of the representation, and the four outputs are combined with a residual projection and PReLU. The result represents how a place is used across the observed trajectories, rather than only where it is located.

### E.3.3 Step 3: add the spatial place neighborhood

The higher spatial route combines the pooled place representation with a trainable 64-dimensional place table initialized from the pretrained place representation. A learned scalar, initialized to 1.0, controls the table's contribution. One weighted graph convolution then propagates the combined state over the Delaunay place graph.

The pooled place representation is detached on this route. As a consequence, the place–region and region–city objectives cannot update the check-in encoder through the spatial branch. The check-in and place levels still learn from their own check-in–place objective. This boundary prevents higher geographic losses from rewriting the temporal visit representation while still allowing the spatial path to learn.

### E.3.4 Step 4: form region and city context

Places inside a region are summarized by a second four-head attention module. A $64 \rightarrow 64$ graph convolution then exchanges information between adjacent region polygons. Its output $\mathbf{z}_r \in \mathbb{R}^{64}$ is the representation of region $r$ used downstream.

Finally, the model forms a city summary from an area-weighted combination of all region representations followed by a sigmoid. This city vector provides global context for the

highest self-supervised objective. It is not an input to the joint forecast model; the downstream model consumes the check-in and region tables.

## E.4 How Check2HGI learns and what it exports

Check2HGI learns from true and corrupted relationships instead of next-visit labels [31, 32]. Bilinear discriminators operate at three hierarchy boundaries:

**Check-in to place.** A check-in and its assigned place form a positive pair. A different place provides the negative pair.

**Place to region.** A place and its assigned region form a positive pair. A different region provides the negative pair, with part of the sampling aimed at moderately similar regions to avoid only trivial comparisons.

**Region to city.** Real region representations are compared with the city summary. Negative regions are obtained by permuting the check-in feature rows and repeating the forward pass while preserving the temporal topology.

Two auxiliary objectives regularize the spatial hierarchy. Masked-place reconstruction hides 15% of place representations, aggregates their Delaunay neighbors, and reconstructs the empirical category distribution. The table-anchor term penalizes excessive movement away from the initialized place table. The exact objective is

$$\begin{aligned}\mathcal{L}_{\text{Check2HGI}} = 0.4\mathcal{L}_{CP} + 0.3\mathcal{L}_{PR} + 0.3\mathcal{L}_{R\Omega} \\ + 0.3\mathcal{L}_{\text{mp}} + 0.1\mathcal{L}_{\text{anc}}.\end{aligned} \tag{E.2}$$

Here, $\mathcal{L}_{\text{Check2HGI}}$ is the scalar representation loss. The terms $\mathcal{L}_{CP}$, $\mathcal{L}_{PR}$, and $\mathcal{L}_{R\Omega}$ are the three contrastive losses. $\mathcal{L}_{\text{mp}}$ is masked-place reconstruction, and $\mathcal{L}_{\text{anc}}$ is the place-table anchor. The coefficients are fixed design weights and need not sum to one.

Training uses full-batch Adam for 500 epochs with learning rate $10^{-3}$, no weight decay, and gradient-norm clipping at 0.9. The saved representation state is the epoch with the smallest complete training loss. Seed 42 is used for this stage.

After training, the category stream retrieves one check-in representation per historical visit. For user $u$, the ordered history is

$$\mathbf{H}_u = (\mathbf{x}_1, \mathbf{x}_2, \ldots, \mathbf{x}_k) \in \mathbb{R}^{k \times d'}, \qquad k = 9, \quad d' = 64. \tag{E.3}$$

Here, $\mathbf{H}_u$ is the category-stream history, $\mathbf{x}_j$ is the Check2HGI representation of historical visit $j$, $k$ is the number of observed visits, and $d'$ is the width of each check-in representation. A second

$9 \times 64$ matrix contains the corresponding region representations. The two modalities remain separate; they are not concatenated into one $9 \times 128$ matrix.

Windows use stride one: visits 1–9 predict visit 10, visits 2–10 predict visit 11, and so forth. Users need at least ten visits, and no synthetic final target is added. Complete users, rather than individual windows, determine the supervised folds.

Check2HGI is trained on the complete graph before those folds are fitted. The forecast evaluation is therefore user-disjoint, while representation learning is transductive with respect to the graph. Reproducing the study requires preserving this distinction.

## E.5 How the joint model processes both histories

The joint architecture is more than one shared encoder followed by two output layers. It combines private input encoders, a coupled interaction subsystem, and task-specific heads. Figure 10 shows these routes as parallel lanes so that interaction and private processing remain visually distinct.

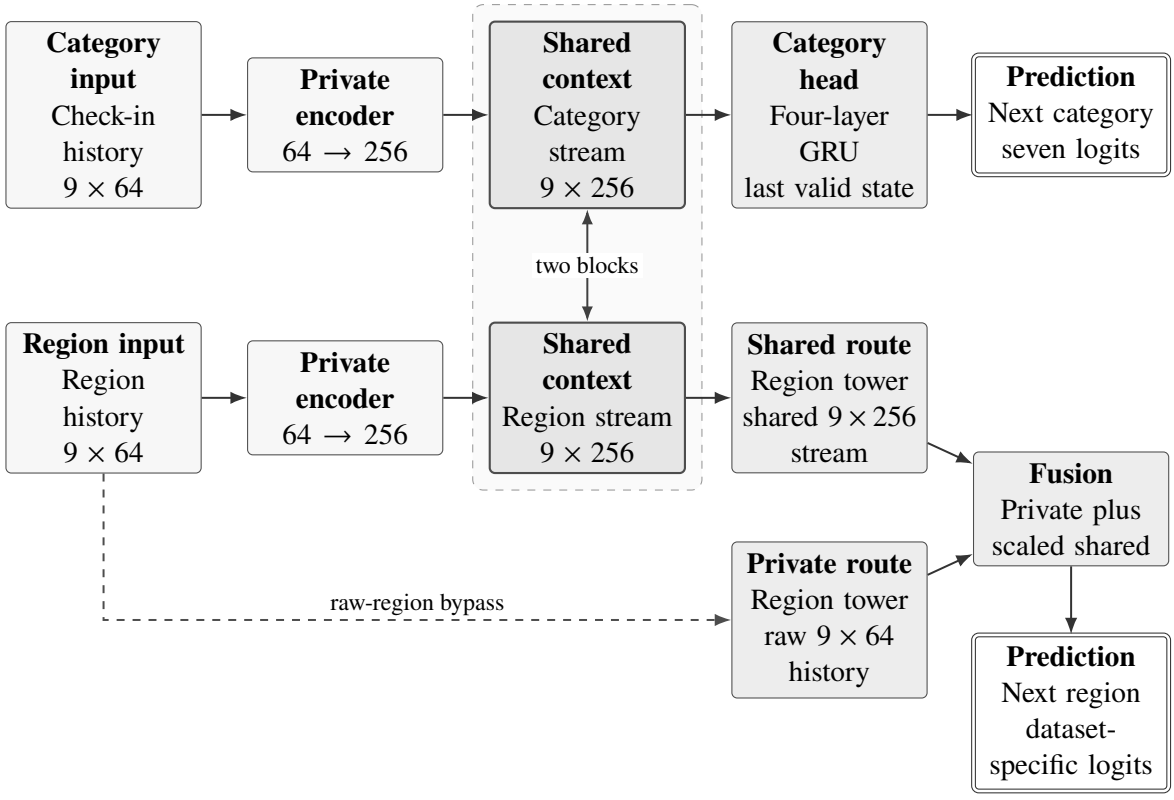


Figure 10 – Joint-model forward path. The two histories have equal shapes but independent encoders. The central states exchange information through two bidirectional cross-attention blocks. The region task also preserves a complete private route from the raw region history.

### E.5.1 Step 1: encode the modalities separately

The check-in and region histories enter independent encoders with the same shape and different parameters. Each encoder applies three linear transformations: $64 \rightarrow 256$, $256 \rightarrow 256$, and $256 \rightarrow 256$. ReLU and layer normalization follow every transformation. Dropout 0.1 follows the first two. The result is one $9 \times 256$ sequence for each task.

Separate encoders are necessary because equal tensor widths do not imply equal meaning. A check-in vector describes one visit in its trajectory, while a region vector describes a geographic area after spatial aggregation. Forcing both through one encoder would assume that the same parameters should interpret these different objects.

### E.5.2 Step 2: exchange context through cross-attention

Two bidirectional cross-attention blocks connect the encoded streams. In each block, the category stream queries the region stream first. The region stream then queries the already updated category stream. Each direction has its own attention projections, residual connections, layer normalizations, and $256 \rightarrow 256 \rightarrow 256$ GELU feed-forward network.

Operationally, every historical category position can retrieve useful information from the region-history positions, and every region position can retrieve category context. Four attention heads perform this exchange in parallel. Padding positions are excluded from the attention weights. After two blocks, separate final layer normalizations produce the category-context and region-context sequences.

The interaction subsystem is jointly optimized, but the directional projections are not tied. The model therefore differs from classical hard parameter sharing: the tasks keep private encoders and heads, while their activations meet in a fixed trainable interaction module that receives gradients from both losses.

### E.5.3 Step 3: produce category and region predictions

The category-context sequence enters a four-layer unidirectional GRU with input and hidden width 256. The head selects the top-layer state at the last valid historical position, applies layer normalization and dropout, and produces seven category logits.

The region head deliberately keeps two routes. The private tower reads the raw $9 \times 64$ region history and processes it with a STAN-style spatio-temporal attention model [15]. The shared-context tower reads the $9 \times 256$ region sequence produced by cross-attention. The private tower uses four heads and dropout 0.3; the shared-context tower uses eight heads and dropout 0.1. Each tower returns one 128-dimensional feature vector.

The two region features are combined as

$$\mathbf{f}_R = \mathbf{f}_{\text{priv}} + \beta \mathbf{W}_{\text{shr}} \mathbf{f}_{\text{shr}}. \tag{E.4}$$

Here, $\mathbf{f}_R \in \mathbb{R}^{128}$ is the fused region feature, $\mathbf{f}_{\text{priv}} \in \mathbb{R}^{128}$ is the private-tower feature, $\mathbf{f}_{\text{shr}} \in \mathbb{R}^{128}$ is the shared-context feature, $\mathbf{W}_{\text{shr}} \in \mathbb{R}^{128\times128}$ is its learned projection, and $\beta$ is a trainable scalar initialized to 0.1. Layer normalization, dropout, and a final linear classifier produce one logit per region in the current dataset. This design lets category context help region prediction without removing the direct spatial sequence model.

The region head contains one further term that stays inactive in the reported configuration. A region-transition prior, a table of how often one region follows another, can be added to the region logits through a scalar weight. That weight is fixed at zero and is not trained, so the prior reaches neither the logits nor the gradients, even when a per-fold transition table is supplied. Two related mechanisms are also off: the same table is never used as a soft training signal for the region output, and the category output is never used as an input to it. Region prediction therefore depends only on the two towers described above.

## E.6 How the joint model is optimized and evaluated

Both heads use mean cross-entropy. Their exact supervised objective is

$$\mathcal{L}_{\text{total}} = 0.75\mathcal{L}_C + 0.25\mathcal{L}_R. \tag{E.5}$$

Here, $\mathcal{L}_{\text{total}}$ is the scalar used for one backward pass, $\mathcal{L}_C$ is the next-category cross-entropy loss, and $\mathcal{L}_R$ is the next-region cross-entropy loss. The weights are fixed. No dynamic task balancing, class weighting, label smoothing, or gradient surgery is active in the reported configuration.

AdamW uses three parameter groups. The category group owns the check-in encoder and GRU head. The region group owns the region encoder and both region towers. The shared group owns the cross-attention blocks and final stream normalizations. This partition makes it possible to apply different peak learning rates while still performing one backward pass through the connected model.

The category and region loaders use the same user-disjoint fold and shuffle independently during training. Consequently, their batch rows have compatible shapes but need not describe the same user or window in one optimizer step; cross-attention operates on that random pairing. The shorter loader cycles until the longer loader is exhausted. Validation rows are record-aligned. This operational detail is unusual, but it is part of the reported training protocol.

Training lasts 50 epochs with batch size 8192 per task, gradient-norm clipping at 1.0, five user-disjoint folds, and seeds $\{0, 1, 7, 100\}$. Early stopping is disabled. A per-parameter-

group OneCycle schedule preserves the category, region, and shared peak learning rates listed in Table 12. The selected checkpoint maximizes the geometric mean of category macro-F1 and region Acc@10, so checkpoint selection depends on both tasks.

## E.7 Implementation settings

Table 12 lists the settings that materially define the reported architecture and training protocol. The region output width is obtained from the region map of each dataset and is therefore not one global constant.

Table 12 – Reproduction-critical settings for Check2HGI and the joint model.

| Stage | Component | Reported setting |
|---|---|---|
| Check2HGI | Check-in input | 7 category indicators + 4 cyclic time values; width 11 |
| Check2HGI | Graph relations | Consecutive-user check-in edges; Delaunay place edges; polygon-adjacency region edges; hierarchy memberships |
| Check2HGI | Encoders and pooling | Two check-in graph-convolution layers, one place graph convolution, one region graph convolution; width 64; four heads at both hierarchy pools |
| Check2HGI | Auxiliary learning | Mask probability 0.15; pretrained place-table anchor; loss weights in Equation E.2 |
| Check2HGI | Optimization | Full-batch Adam; learning rate $10^{-3}$; 500 epochs; weight decay 0; clip norm 0.9; seed 42 |
| Joint model | Examples and inputs | Nine-visit history; tenth visit as target; stride 1; separate check-in and region tensors, each $9 \times 64$ |
| Joint model | Private encoders | Independent $64 \rightarrow 256 \rightarrow 256 \rightarrow 256$ stacks; ReLU and layer normalization; dropout 0.1 after the first two transformations |
| Joint model | Interaction | Two bidirectional cross-attention blocks; four heads; width 256; dropout 0.15 |
| Joint model | Category head | Four-layer GRU; hidden width 256; seven output classes |

| Stage | Component | Reported setting |
|---|---|---|
| Joint model | Region head | Private tower: 4 heads, dropout 0.3; shared-context tower: 8 heads, dropout 0.1; both width 128; additive fusion with $\beta_0 = 0.1$ |
| Joint model | Inactive region paths | Additive region-transition prior with its scalar weight fixed at 0 and not trained; no transition-table training signal; no category-conditioned region output |
| Joint model | Optimization | AdamW; weight decay 0.05; batch size 8192 per task; 50 epochs; clip norm 1.0; OneCycle schedule |
| Joint model | Peak learning rates | Category $10^{-3}$; region $3 \times 10^{-3}$; shared $3 \times 10^{-3}$ for AL/AZ/FL and $10^{-3}$ for CA/TX/Istanbul |
| Joint model | Evaluation | Five user-disjoint folds; seeds $\{0, 1, 7, 100\}$; category macro-F1 and region Acc@10 |

## E.8 Reproduction landmarks and implementation map

Five checks distinguish the reported method from nearby alternatives:

- the check-in graph contains user-succession edges only;
- Check2HGI exports separate check-in and region tables of width 64;
- the joint model receives two $9 \times 64$ histories rather than one concatenated tensor;
- the private region tower reads the raw region history, while the shared-context tower reads the cross-attention output; and
- the supervised objective is the fixed $0.75/0.25$ combination in Equation E.5.

The implementation is organized in four source areas:

**Check2HGI representation:** `<research/embeddings/check2hgi/>`

**Joint-model assembly:** `<src/models/mtl/mtlnet_crossattn_dualtower/>`

**Task heads:** `<src/models/next/>`

**Loss and training:** `<src/losses/>` and `<src/training/>`